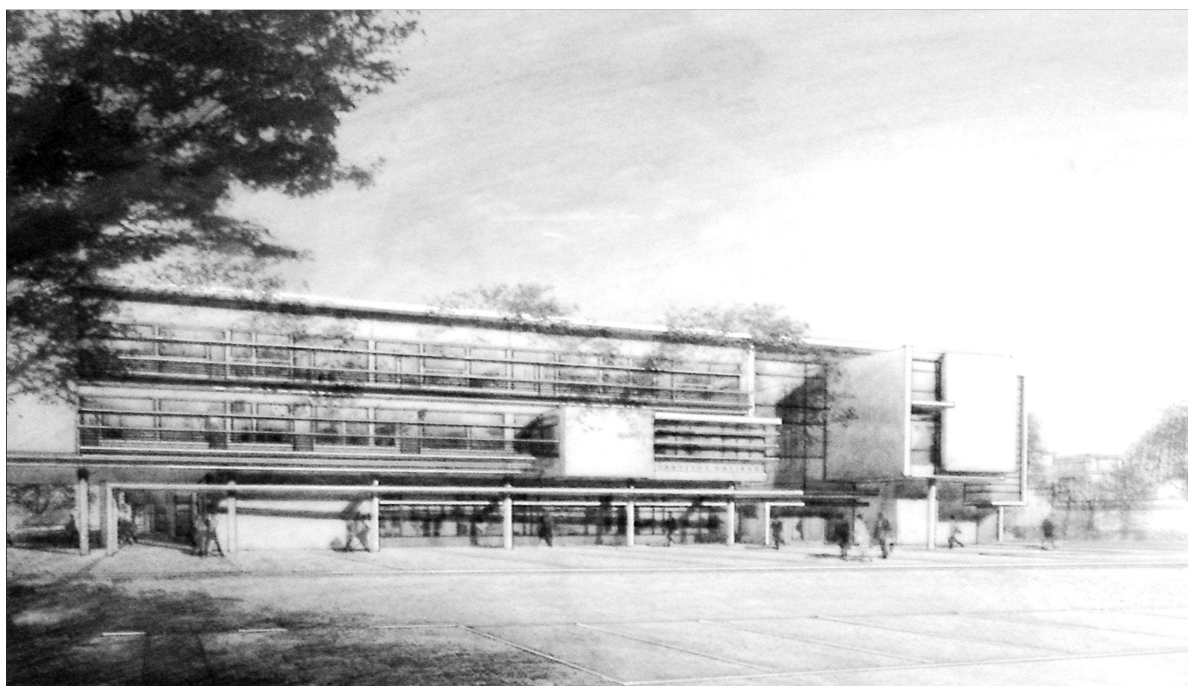




MISE EN ŒUVRE D'UN DSP

Aspects matériels et logiciels de l'outil de développement EZKIT

SOMMAIRE GENERAL

I. INTRODUCTION : LE DSP	02
II. LA FAMILLE 21xx	10
III. LA FAMILLE 21xxx	26
IV. LE DSP UTILISE SUR LA CARTE EZKIT	32
V. LE CODEC AD1847	51
VI. LE SYSTEME D'EVALUATION EZ-KIT	64
VII. MISE EN ŒUVRE EN LANGAGE C	81

THIEBAUT Bernard

I. INTRODUCTION : LE D.S.P. (ou Digital Signal Processor)

I.1. Généralités : qui fait quoi ?

Avant d'expliciter les spécificités d'un **DSP**, essayons rapidement de nous y retrouver dans la grande famille des processeurs (nombreux constructeurs, beaucoup de composants plus ou moins performants et multitudes d'applications diverses, variées et en perpétuelle évolution ...).

On trouvera le plus fréquemment les quatre grandes branches suivantes :

Microprocesseur (μp),

Microcontrôleur (μc),

Processeur spécialisé (μs),

Processeur de traitement du signal (DSP).

Comment s'y retrouver et lequel choisir ?

Cela dépend en fait du type de l'application.

Pour simplifier, voici quelques éléments de base qui pourront nous guider dans le choix.

● **μp** : utilisé surtout pour des applications nécessitant :

- une quantité de mémoire vive importante,
(implantation d'un gros système d'exploitation temps réel ou pas, mono ou multitâches)
- et effectuant des travaux de nature différente (pas de spécialisation à priori) : calculs, transferts de mémoire à mémoire, gestion d'Entrées/Sorties et de périphériques divers (clavier, écran, disques, interface réseau, webcam ...).

Dans la grande majorité des cas, il possède **un seul bus d'adresses** et **un seul bus de données**.

C'est l'architecture dite de **Von Neumann** que l'on voit représentée sur la figure I.1. ci-après.

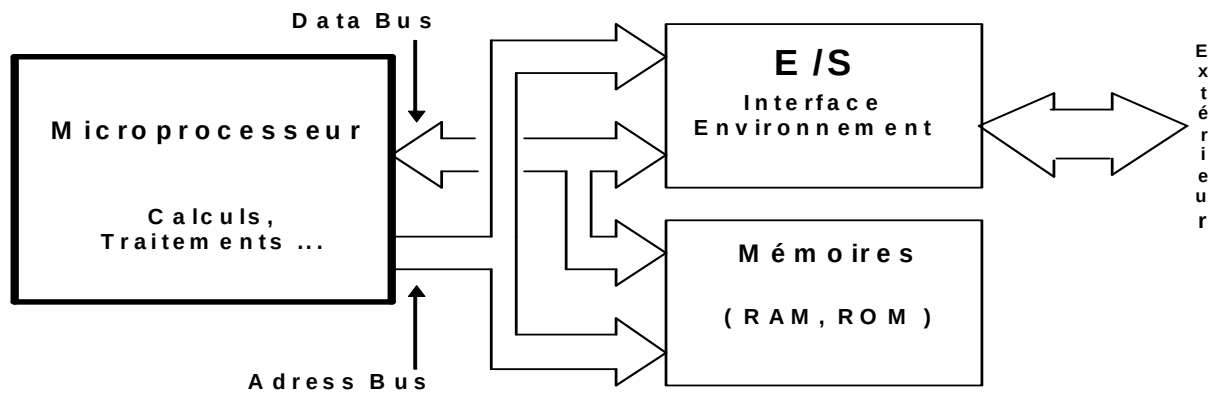


Figure I.1.

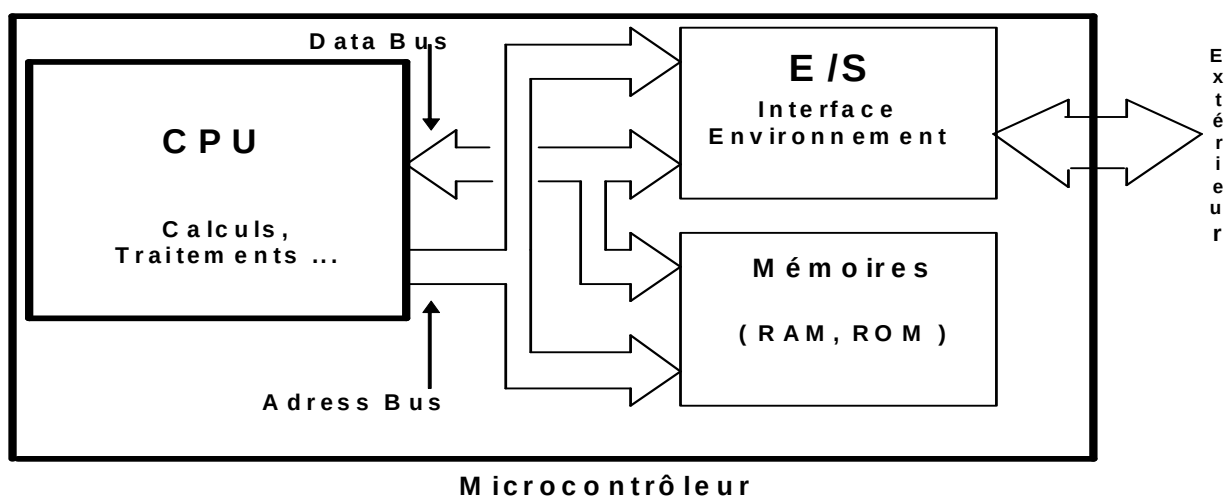
Ex : cœur des microordinateurs, des serveurs, des stations de travail ...

Remarque : le bus de contrôle (synchronisation, horloges, signaux d'acquittement divers, reset ...) n'a pas été représenté sur cette figure.

● **μc** : utilisé pour des applications :

- plus restreintes (moins de mémoire vive, moins de calcul ...),
- mais plutôt orientées vers la gestion d'un grand nombre d'entrées/sorties, de périphériques, d'interfaces avec l'extérieur (divers capteurs et actionneurs).

Pour des raisons d'intégration et de coût, on implémente sur la même puce (et donc dans un seul circuit intégré) toutes les fonctions qui en principe étaient délocalisées dans une architecture à base de **μp**, comme on peut le voir sur la figure I.2. ci-dessous, où on remarque aussi une architecture de type Von Neumann.



CPU : Central Process Unit :là où sont faits les divers traitements
C'est le « cœur » des microcontrôleurs

Figure I.2.

Ex : outil par excellence des petits automatismes délocalisés, électronique portable et/ou embarquée ...

Dans certains cas, grâce à sa mémoire interne, il peut même être en « stand alone » ou « single chip » (pas de mémoire externe supplémentaire) simplifiant considérablement l'étude hardware, limitée à un seul composant à savoir le **μc** lui-même.

● **μs** : processeur utilisé pour effectuer certaines tâches spécifiques d'un système à base de **μp**, en déchargeant complètement ce dernier du traitement correspondant à ces tâches. Tout se passe un peu comme si le **μp** et le **μs** travaillaient en parallèle. Ce qui permet au final et dans la plupart des cas d'accélérer le fonctionnement de l'application complète.

Ex : unité de calcul arithmétique (historiquement série xxx87 chez Intel), contrôleur de clavier, processeur vidéo, contrôleur de **DMA**, contrôleur de mémoire de masse (disque, CD, DVD), contrôleur USB, contrôleur de réseau ...

● **DSP** : avec l'avènement du traitement numérique du signal (traitement du son, traitement d'images, multimédia, techniques de compression, téléphonie numérique, modem hauts débits, cryptage des données ...), les algorithmes de traitement du signal sont devenus de plus en plus complexes, de plus en plus importants et de plus en plus gourmands en capacité de calcul.

D'où la nécessité, pour pouvoir faire du traitement en **temps réel**, d'avoir des architectures adaptées à cette problématique (favoriser la rapidité, faire le maximum de traitements en simultané en développant le parallélisme..).

La première amélioration hardware par rapport aux architectures présentées sur les figures I.1. et I.2. précédentes consiste à utiliser une architecture de **type Harvard**, modification permettant **un gain en vitesse d'un facteur 2** (au prix d'une complexité interne plus importante).

En quoi consiste cette nouvelle architecture (cf. figure I.3. suivante) ?

Le principe de base consiste à séparer la mémoire en 2 blocs **différents** et **indépendants** :

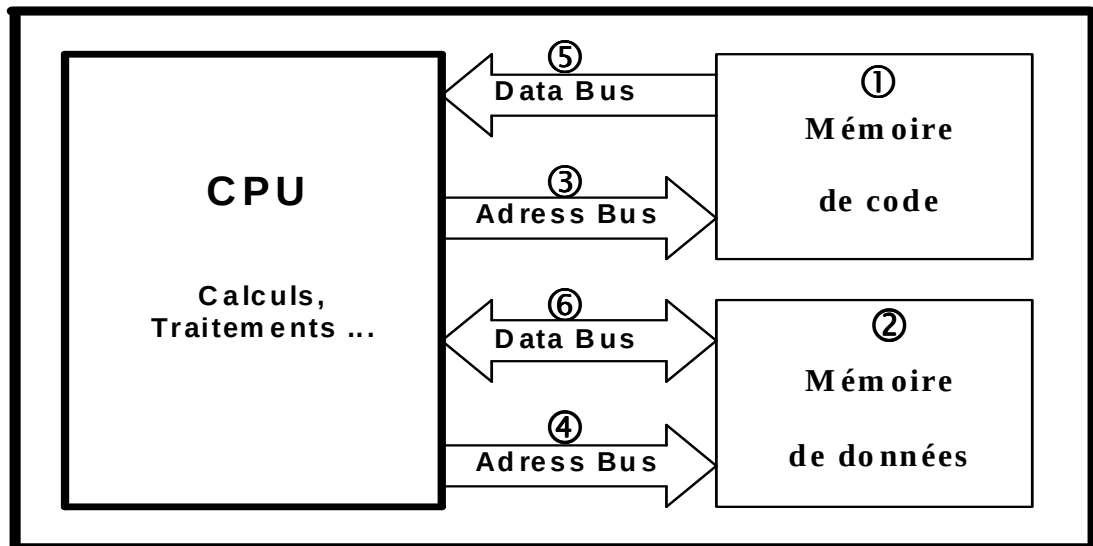
- ① un premier bloc pour contenir le code (c'est-à-dire la liste des instructions constituant le programme qui s'exécute) : c'est la **mémoire de code** (ou **mémoire de programme**),
- ② un second bloc pour contenir les données (c'est-à-dire les opérandes des instructions) : c'est la **mémoire de données**.

Contrairement à une architecture de type Von Neumann, il y aura ici :

- ③ un **bus d'adresses** pour accéder aux différentes cases de la mémoire de code,
- ④ un **bus d'adresses** pour accéder aux différentes cases de la mémoire de données,
- ⑤ un **bus de données** pour lire le contenu des différentes cases de la mémoire de code,
- ⑥ un **bus de données** pour lire / écrire le contenu des différentes cases de la mémoire de données.

Et donc, au final, **4 bus au total**, et qui peuvent fonctionner de manière indépendante, c'est-à-dire que le **DSP** peut, à **un instant donné** lire le contenu de la mémoire de code pour accéder à une instruction et écrire dans la mémoire de données pour modifier une variable.

Le principe de cette architecture de type Harvard est résumé sur le schéma de la figure I.3. suivante :



Digital Signal Processor
Architecture Harvard dans un DSP

Figure I.3.

De plus, pour un DSP, le maximum de traitements est fait en parallèle. Par exemple :

- l'exécution de l'instruction (q) par **l'unité d'exécution**,
- le décodage de l'instruction suivante (q+1) par **l'unité de décodage**,
- la recherche de l'instruction encore suivante (q+2) par **l'unité de recherche en mémoire de code**,

sont faits **en même temps** pour le code, c'est-à-dire en **un seul coup (ou tick) d'horloge (réalisée par un quartz)** repéré par l'entier q ($\cong$ le temps qui s'écoule), tout en laissant possible un accès simultané pour les données dans la mémoire de données grâce à l'architecture Harvard.

Ce qui est résumé sur le schéma de principe de la figure I.4. où sont représentés :

- l'horloge du DSP (fournie généralement par un quartz) sur laquelle sont représentés les différents ticks,
- le cadacement décalé des différentes unités : exécution, décodage et recherche.

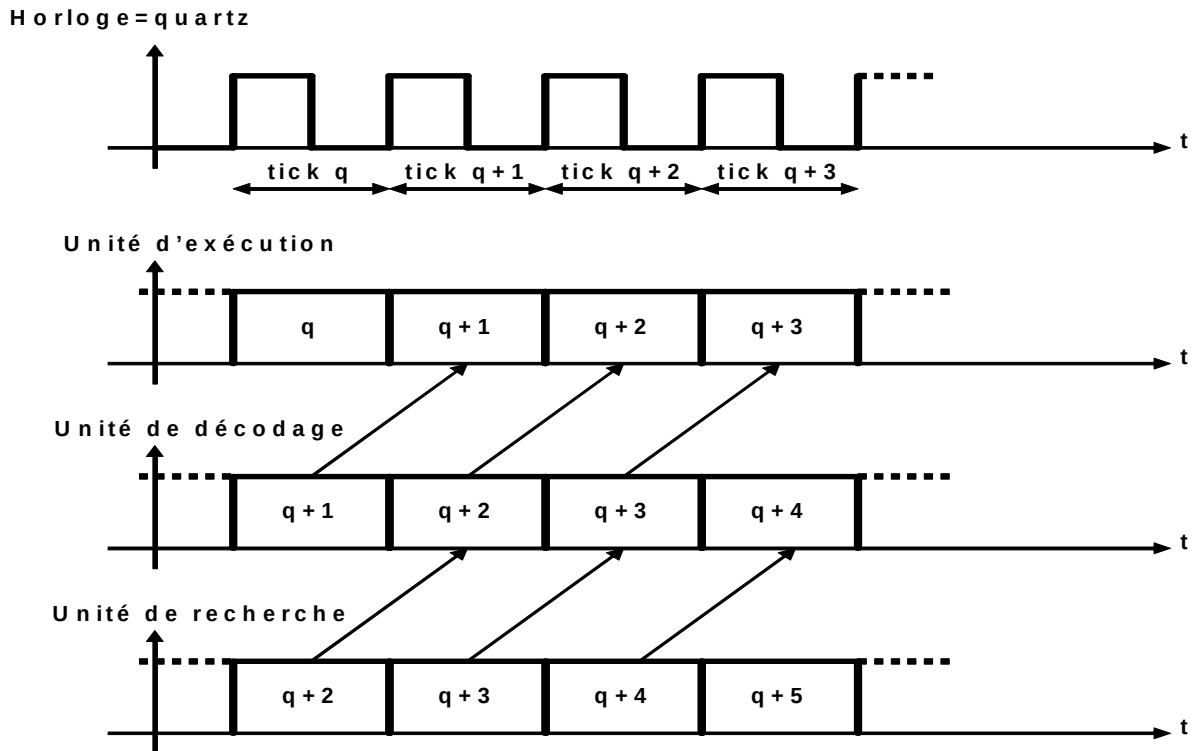


Figure I.4.

Nous verrons tout cela plus en détail dans la suite de ce polycopié en présentant les différentes unités concernées (unité de recherche, unité de décodage et unité d'exécution) par ce fonctionnement en parallèle. Ce fonctionnement est aussi appelé en « **pipe-line** ».

Finalement, si on prend suffisamment de recul, c'est quoi faire du traitement numérique du signal ?

On peut dire que faire du traitement numérique du signal n'est rien d'autre que de travailler et d'appliquer des algorithmes sur les échantillons prélevés sur ce signal, c'est-à-dire au final travailler avec des nombres (les valeurs numériques de ces échantillons prélevés à chaque instant d'échantillonnage).

Et donc, le **DSP** doit être capable de d'effectuer les 4 opérations arithmétiques de base avec ces nombres (en fait, surtout des multiplications et des additions), et ceci le plus rapidement possible.

Pour s'en convaincre, regardons l'exemple de l'équation au récurrence d'un filtre numérique à **Réponse Impulsionnelle Infinie (RII)** :

$$y(n) = \sum_{j=0}^{N-1} a(j).x(n-j) + \sum_{j=1}^M b(j).y(n-j)$$

(équation à laquelle on fera référence plusieurs fois dans le chapitre **II.** pour faire le lien entre le hardware et le software)

Cette équation permet de calculer la valeur numérique du signal de sortie **y(n)** de ce filtre à l'instant t_n (en se rappelant que $t_n = n.T_e$, avec T_e période d'échantillonnage $= \frac{1}{F_e}$ où F_e représente la fréquence d'échantillonnage, celle qui intervient dans le théorème de Shanon) en fonction des valeurs du signal d'entrée :

- **x(n)** à l'instant t_n ,
- **x(n-1)** à l'instant t_{n-1} ,
- **x(n-2)** à l'instant t_{n-2}

et des valeurs du signal de sortie :

- **y(n-1)** à l'instant t_{n-1} ,
- **y(n-2)** à l'instant t_{n-2} ...

(⚠ ⚠ : ne pas confondre l'entier n représentant ici les multiples de la période d'échantillonnage T_e (qui vaudra par exemple 125μs si on échantillonne à $F_e=8$ kHz) et l'entier q de la page précédente représentant les multiples de la période du quartz du DSP, qui est de l'ordre de quelques ns)

En gros, il faut pouvoir **multiplier** deux opérandes (par exemple $a(0)$ et $x(n)$ pour $j=0$) en un **seul coup** d'horloge et **accumuler** le résultat obtenu par cette multiplication (pour construire au fur et à mesure le « $\sum_{j=0}^{N-1}$ ») dans le **même** coup d'horloge, **tout en préparant** la recherche des opérandes suivants ($a(1)$ et $x(n-1)$ pour $j=1$) ... et idem avec les $b(j)$ et les $y(n-j)$.

Grâce à l'architecture Harvard, on verra que le découpage de la mémoire en 2 blocs indépendants permet de répondre à cette problématique et de façon assez efficace.

Remarque : dans la pratique, les grandes caractéristiques que l'on vient de présenter ici pour chacun des membres de la famille des principaux processeurs ne seront pas toujours aussi nettes : par exemple, certains **μc** peuvent présenter à la base une architecture de type Harvard (pour être plus performants comme dans les **PIC** par exemple) ou certains **μp** peuvent être conçus à la base pour faire du traitement numérique du signal (technologie **MMX** (**M**ulti**M**édia **e**Xtensions) chez les premiers Pentium de chez Intel par exemple).

Le **DSP** et sa carte d'évaluation que nous allons présenter dans ce polycopié nous permettront de faire du traitement numérique du signal en temps réel, avec de « vrais » signaux analogiques fournis par exemple par un GBF. Ces signaux seront échantillonnés et convertis en numérique par un Convertisseur Analogique Numérique (**CAN**), puis traités par un programme s'exécutant sur le **DSP** et enfin restitués en sortie via un Convertisseur Numérique Analogique (**CNA**).

Et ceci contrairement à l'approche que l'on fait avec **Matlab** où l'on définit en machine des signaux « virtuels » sur lesquels on vient tester des traitements particuliers, bref où l'on effectue des **simulations** sur ces signaux, en étant donc **un peu plus éloigné des aspects temps réel**.

Par contre, on pourra être amenés à utiliser Matlab comme outil nous permettant d'effectuer ou d'automatiser certains calculs (qui seraient fastidieux si on les faisait « à la main »).

Au final cette série de TP se trouvera donc au croisement de plusieurs « chemins » :

- le **cours de TNS** où les **aspects plus théoriques** liés au traitement numérique du signal vous ont été présentés, comme par exemple la transformée en z , les techniques de conception des filtres numériques à **Réponse Impulsionnelle Finie** ou **Infinie**, la transformée de Fourier Rapide (**FFT**) ...
- les **TP de Matlab** où, comme on vient de le dire, les aspects **simulation** étaient privilégiés,
- mais aussi de **l'informatique** et de la **programmation en langage C**, car les programmes que vous implanterez sur le DSP seront écrits à 99% en langage C, ce qui permet de développer des applications plus rapidement qu'en langage natif assembleur et d'éviter à avoir à apprendre le langage assembleur du DSP.
On verra quand même dans ce polycopié quelques petits exemples en assembleur car, comme d'habitude le langage assembleur étant plus proche du hardware, il permet de mieux faire le lien avec les aspects purement électroniques (architecture interne, blocs fonctionnels, registres internes de programmation...) mais également de comprendre certains principes de fonctionnement, voire certaines techniques de programmation.

Ce polycopié, orienté tout d'abord sur les aspects hardware (chapitres **II.** à **V.**), puis dans un deuxième temps sur des éléments software (chapitres **VI.** et **VII.**), **ne se prétend pas être exhaustif** quant aux multiples possibilités de développement et d'applications à partir du **DSP** présenté et utilisé en TP.

Il se veut cependant être un outil **efficace, condensé et concentré** sur l'essentiel pour pouvoir comprendre le fonctionnement de l'outil présenté et démarrer une première approche et une prise en main dans une **relative autonomie**, sans avoir à assimiler les milliers de pages de la documentation constructeur. (aspects matériels et environnement logiciel dans pas moins d'une dizaine d'ouvrages écrits en anglais et de 400 pages chacun ! ! !).

Vous aurez d'ailleurs l'occasion en TP d'aller « fouiller » dans cette documentation en Anglais pour y extraire des informations utiles pour comprendre et mettre en œuvre certains mécanismes et interpréter des résultats expérimentaux.

I.2. Les DSP chez Analog Devices.

Analog Devices est un constructeur américain bénéficiant d'une bonne réputation et d'une solide expérience dans le domaine du traitement numérique du signal, de la conception et de la fabrication de DSP.

Il existe 2 grandes familles de DSP chez Analog Devices :

- la famille **ADSP 21xx** : sur **16 bits** avec calcul en virgule **fixe**,
- la famille **ADSP 21xxx** : sur **32/40 bits** avec calcul en virgule **flottante**.

(des éléments sur la virgule fixe ou flottante seront fournis plus loin dans ce polycopié).

Le tableau de la figure I.5. ci-dessous résume les principales caractéristiques pour quelques éléments de ces 2 familles. Ce tableau n'est pas du tout exhaustif ni en ce qui concerne le nombre d'éléments de chaque famille, ni en ce qui concerne les caractéristiques techniques de chaque composant : il faudra donc toujours se référer à la documentation complète en cas de choix de composant ou d'informations plus précises sur un composant particulier ...

Famille	Type	RAM	ROM	Instruc. Time(ns)	Serial Port	Program. Timer	External Interrupts	CAN CNA	Volts	Pin Count
16 bits Fixed Point	ADSP2101	3K	-	60	2	1	3	0	5	80
	ADSP2103	3K	-	100	2	1	3	0	3,3	80
	ADSP2105	1,5K	-	100	1	1	3	0	5	68
	ADSP2106	0,5K	1K	100	1	1	3	0	5	68
	ADSP2111	3K	-	60	2	1	3	1	5	100
	ADSP21msp50/51	3K	2K	77	2	1	3	1	5	144
32/40 Bits Floating Point	ADSP21010	32K	-	80	2	1	4	0	5	304
	ADSP21020	64K	-	40	2	1	4	0	5	223

Figure I.5.

La présentation que nous allons faire dans un premier temps est basée, pour :

- des raisons **historiques** : des plus anciens aux plus récents,
- des raisons de **simplicité** et de **pédagogie** : des plus simples aux plus complexes,

sur la famille 21xx, plus ancienne, mais la plupart des principes qui y seront abordés, se retrouveront dans les DSP 21xxx, pour y être le plus souvent affinés et améliorés.

Les développements pratiques en TP seront faits sur un **ADSP 21061** dont nous verrons les principales différences avec les 21xx plus loin dans ce polycopié.

II. LA FAMILLE 21xx.

II.1. Eléments sur l'architecture et sur les bus internes.

On présente sur la figure II.1. ci-dessous, les principaux blocs fonctionnels présents dans l'architecture interne de **tous les DSP** de la famille 21xx. C'est leur partie commune.

Ces principaux blocs sont au nombre de six, repérés de ① à ⑥ :

- ① **ALU** : unité de calculs arithmétiques et logiques,
- ② **MAC** : unité de multiplication et d'accumulation,
- ③ **SHIFTER** : unité de décalage,
- ④ **DATA ADDRESS GENERATOR 1** : unité de gestion des adresses pour la mémoire de données,
- ⑤ **DATA ADDRESS GENERATOR 2** : unité de gestion des adresses pour la mémoire de code,
- ⑥ **PROGRAM SEQUENCER** : unité assurant le déroulement du programme.

Ces différents blocs et leurs rôles seront présentés dans les parties suivantes, mais notons dès à présent, l'existence des **4 bus internes** caractéristiques comme on l'a vu d'une architecture de type Harvard où le code et les données ont été dissociés : il existe une mémoire pour le code **PM** (et éventuellement les données) et une mémoire pour les données uniquement **DM**. Chacune de ces mémoires possède son propre bus d'adresses et son propre bus de données. Il existera donc **en interne** du DSP, 4 bus :

- un bus d'adresses pour la mémoire de code **PM** : le **PMA (Program Memory Addressbus)**,
- un bus de données pour la mémoire de code **PM** : le **PMD (Program Memory Databus)**,
- un bus d'adresses pour la mémoire de données **DM** : le **DMA (Data Memory Addressbus)**,
- un bus de données pour la mémoire de données **DM** : le **DMD (Data Memory Databus)**.

Ces bus, fonctionnant de manière indépendante, agissent ainsi en parallèle : en un seul cycle horloge, le DSP peut accéder à la mémoire de code **et** modifier ou lire des données de la mémoire de données.

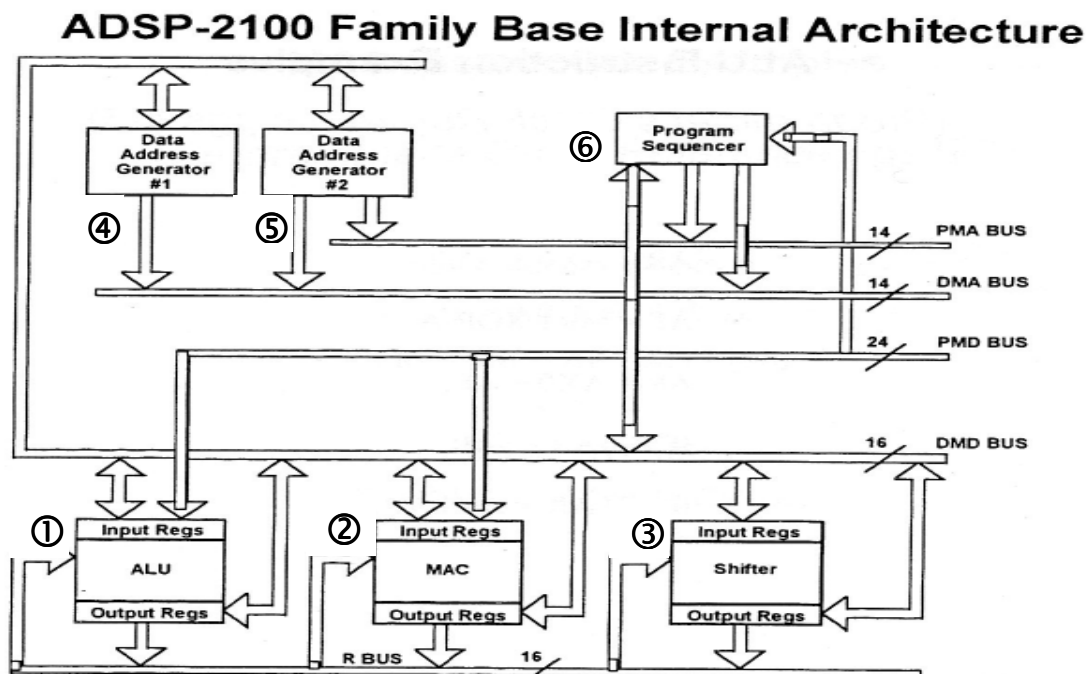


Figure II.1.

Sur la figure II.1., on voit également (en bas) un **RBUS (Result BUS ou bus de résultats)** utilisé pour « reboucler » des résultats intermédiaires obtenus en sortie d'une quelconque des 3 unités de calculs (**ALU**, **MAC** ou **SHIFTER**) vers l'entrée d'une autre unité (ce qui permet les transferts « inter-unités »).

II.2. Les unités de calculs internes.

Ces unités, au nombre de trois fonctionnent en parallèle pour optimiser les temps de calcul.

II.2.1. ALU : Arithmetic and Logic Unit.

C'est un bloc que l'on trouve dans tous les micro/processeurs/contrôleurs.

En fait, c'est véritablement leur cœur qui permet de réaliser toutes les opérations **arithmétiques** de base (**addition, soustraction...**) et toutes les opérations **logiques** (fonctions **OR, AND, XOR...**, **complément, comparaison ...**) sur les opérandes, c'est-à-dire les variables du programme.

Rien d'étonnant alors qu'un DSP possède aussi sa propre **ALU**.

Comme on peut le voir sur la figure II.2. ci-dessous, l'**ALU** ① possède 2 entrées d'opérandes (**X** et **Y**) et une sortie pour le résultat (**R**) correspondant à une opération arithmétique ou logique entre **X** et **Y**. Deux paires de registres (**AX0** et **AX1** ② pour l'entrée **X** et **AY0** et **AY1** ③ pour l'entrée **Y**) permettent de récupérer les opérandes sur les bus de données **PMD** et **DMD** (avant traitement). Ce qui prouve que l'ALU effectue bien des calculs sur des données (et non sur des adresses). Un registre de sortie (**AR** ④) permet de stocker le résultat avant de l'envoyer soit sur le **RBUS**, soit sur le **DMD** bus, soit même sur l'entrée **X** de l'**ALU** pour un autre traitement. Sur le schéma de la figure II.2., les blocs référencés **MUX** (comme **MU**lti**P**le**X**eur) permettent d'assurer le cheminement des données entre les différentes parties que l'on vient de décrire.

ALU Block Diagram

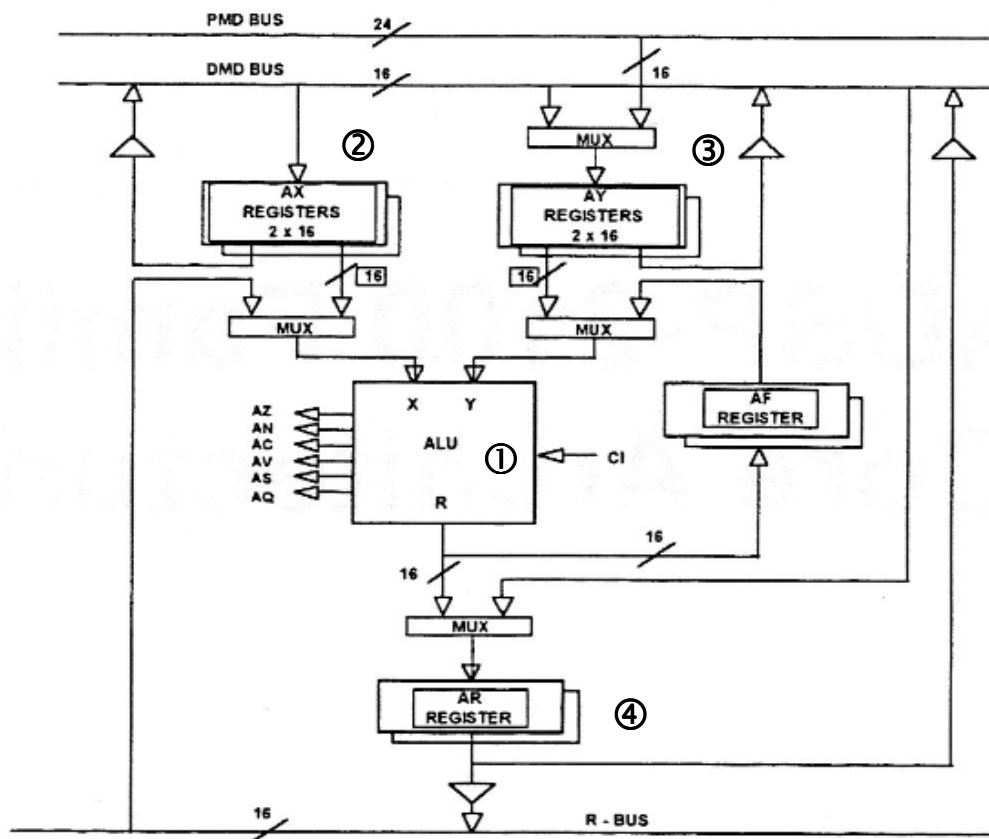


Figure II.2.

II.2.2. MAC : Multiplier / ACcumulator Unit.

Ce bloc est plus intéressant car il est plus spécifique à un DSP.

Remarquons déjà en regardant la figure II.3. que lui aussi travaille sur les données fournies en haut de cette figure soit via le **PMD BUS**, soit via le **DMD BUS**.

Ce **MAC** est constitué en son centre d'un **multiplieur** ① avec 2 entrées **X** et **Y** (sur 16 bits chacune) et une sortie **P** (sur 32 bits) fabricant le produit de **X** et de **Y**. Ce multiplieur est dit « câblé » car il fournit le résultat de cette multiplication en un seul coup d'horloge. Il est alimenté en entrée par la paire de registres **MX** ② (**MX0** ou **MX1**) pour son entrée **X** et la paire de registres **MY** ③ (**MY0** ou **MY1**) pour son entrée **Y**.

En sortie de ce multiplieur, on remarque un bloc **ADD/SUBTRACT** ④ qu'on appelle aussi un accumulateur, au sens où il permet, grâce à ses registres de sortie **MR2**, **MR1** et **MR0** en ⑤ d'accumuler (c'est-à-dire d'additionner) le résultat de plusieurs multiplications successives, résultat dont la taille est sur 5 octets max : **MR2** (de taille 8 bits) servant pour son octet de poids fort et **MR1** et **MR0** (de taille 16 bits chacun) pour ses 4 octets de poids faible.

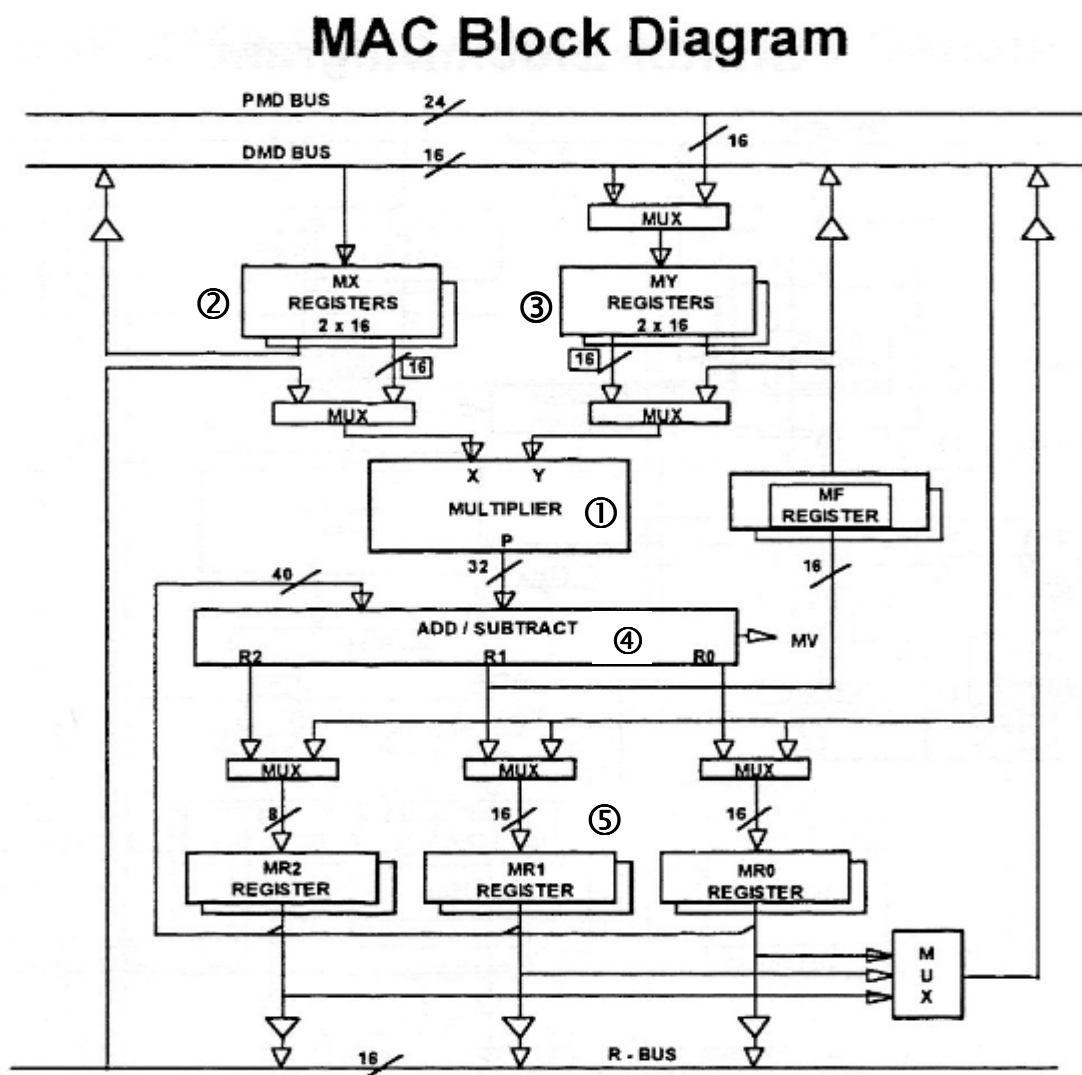


Figure II.3.

Par exemple, examinons une instruction assembleur DSP du type :

$$\mathbf{MR} = \mathbf{MR} + \mathbf{MX0} * \mathbf{MY0}$$

dont le détail est représenté sur la figure II.4. ci-dessous.

Elle s'exécute en un seul cycle d'horloge et met en œuvre le **MAC**.

Elle permettra d'implanter facilement un calcul comme $y(n) = \sum_{j=0}^{N-1} a(j).x(n-j)$ dans une boucle répétée N fois (de j=0 à N-1).

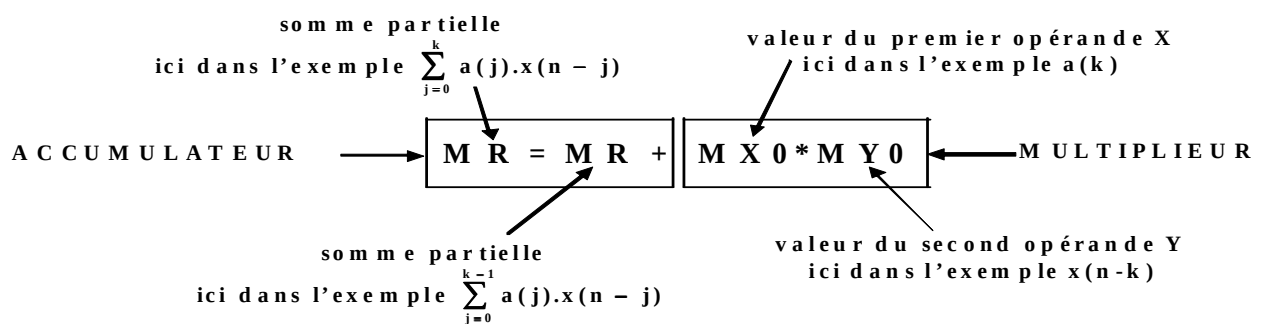


Figure II.4.

Notons toujours la présence du **RBUS** sur le bas de la figure II.3. pour récupérer les résultats des multiplications/additions successives.

II.2.3. SHIFTER : SHIFTER Unit.

Cette unité permet de faire des décalages (à droite ou à gauche) d'un nombre donné de bits **en un seul tick d'horloge**. Ce procédé permet de simplifier les **multiplications** (ou les **divisions**) par 2 et plus généralement par toutes les puissances entières de 2.

Certains algorithmes de traitement numérique du signal, comme par exemple le calcul de la transformée de Fourier rapide (**Fast Fourier Transform**) utilisent beaucoup cette possibilité.

Comme on peut le voir sur la figure II.5. ci-dessous, le **SHIFTER** ① est connecté sur le bus de données **DMD** en entrée (via le registre **SI** ②) et sur le **RBUS** en sortie (via la paire de registres **SR0** et **SR1** ③).

Le bloc **EXPONENT LOGIC** ④ permet de faciliter le traitement en virgule flottante par émulation logicielle (normalisation, dé normalisation) au prix d'une diminution de la rapidité de traitement.

Car rappelons le, à la base, la famille 21xx travaille en virgule fixe.

Shifter Block Diagram

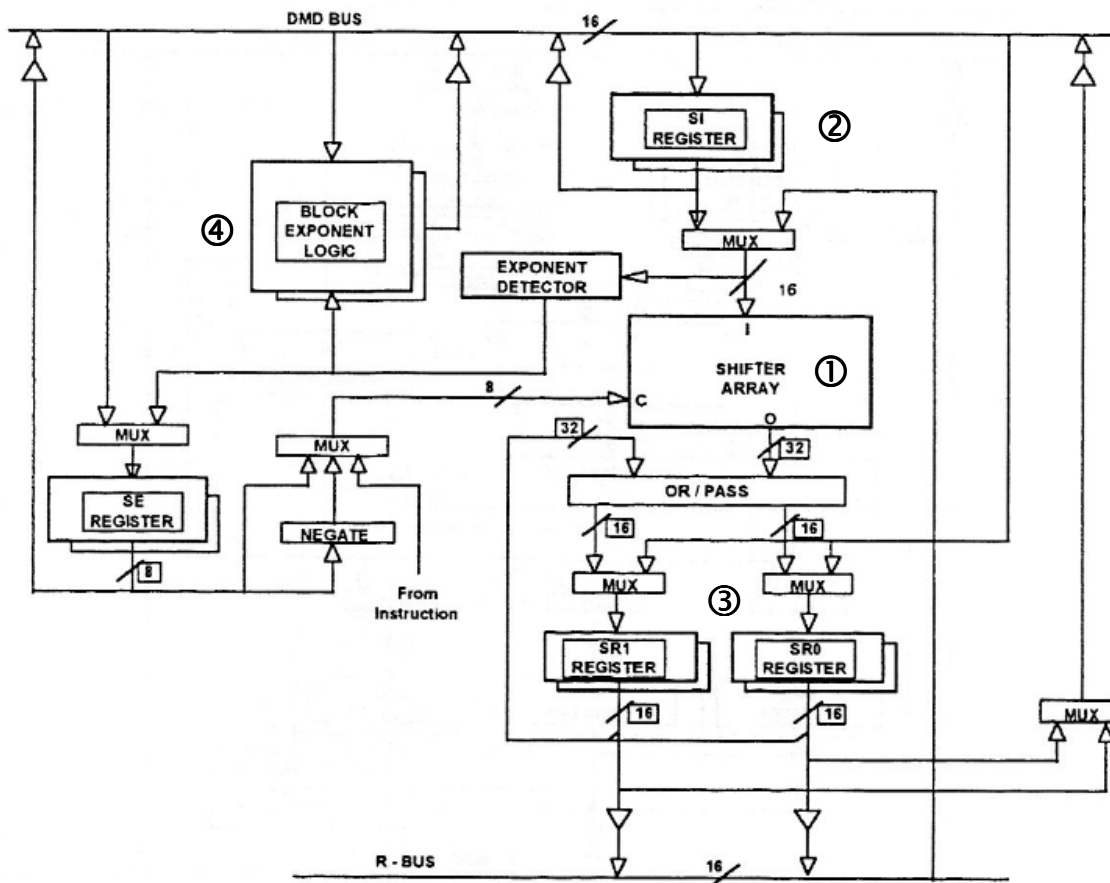


Figure II.5.

II.3. Les DAG : Data Address Generator.

Contrairement aux 3 unités de calculs que l'on vient de présenter et qui travaillent sur les données, les unités que l'on va maintenant voir agissent sur les adresses (en fait sur des pointeurs) pour faciliter les accès en mémoire de données (**DM**) et en mémoire de code (**PM**).

Elles permettent d'accélérer et d'optimiser le calcul de ces adresses pour les bus **DMA** et **PMA** en fonction de la structure des données en mémoire (par incrémentation/décrémentation de pointeurs pour notamment faciliter la gestion de buffers circulaires ou pas).

Les **DAG** sont au nombre de deux comme on a pu le voir sur la figure II.1. précédente.

- **DAG1** pour les données dans la mémoire de données **DM**,
- **DAG2** pour les données dans la mémoire de code (ou mémoire de programme) **PM**.

La figure II.6. montre le synoptique interne de chacun de ces deux **DAG**.

L'adresse, en sortie d'un **DAG** est calculée à partir de 3 groupes de registres :

- 4 registres de type **I** (comme **Index**) : ① (**I0** à **I3**) : ils contiennent un pointeur,
- 4 registres de type **M** ② (**M0** à **M3**) : pour **Modifier** les registres **I** (chaque registre **M** contient en fait un offset qui est rajouté à un registre **I** (**I+M** en signé) comme le montre le bloc **ADD** ④ de la figure II.6.), ce qui permet par exemple de pointer sur l'élément suivant ou précédent d'un buffer pour la recherche de cet élément (**I** est incrémenté/décrémenté par **M** de la taille de chaque élément de ce buffer),
- 4 registres **L** ③ (**L0** à **L3**) : pour la gestion de buffers circulaires par une arithmétique en modulo **Li** (cf. bloc **MODULUS LOGIC** ⑤ sur la figure II.6.) ils contiennent alors la **Longueur** (taille) de ce buffer.

Exemple :

si **L0** = 32 alors buffer **circulaire** de 32 positions mémoire.

Du coup si **I0** = 31 et **M0** = 1 alors **I0+M0** = 32 $\equiv$ 0 [32] (accès au premier élément de ce buffer circulaire).

Par contre, si **L0** = 0, alors il y a une gestion **linéaire** du buffer et la gestion modulo est inhibée.

Data Address Generator Block Diagram

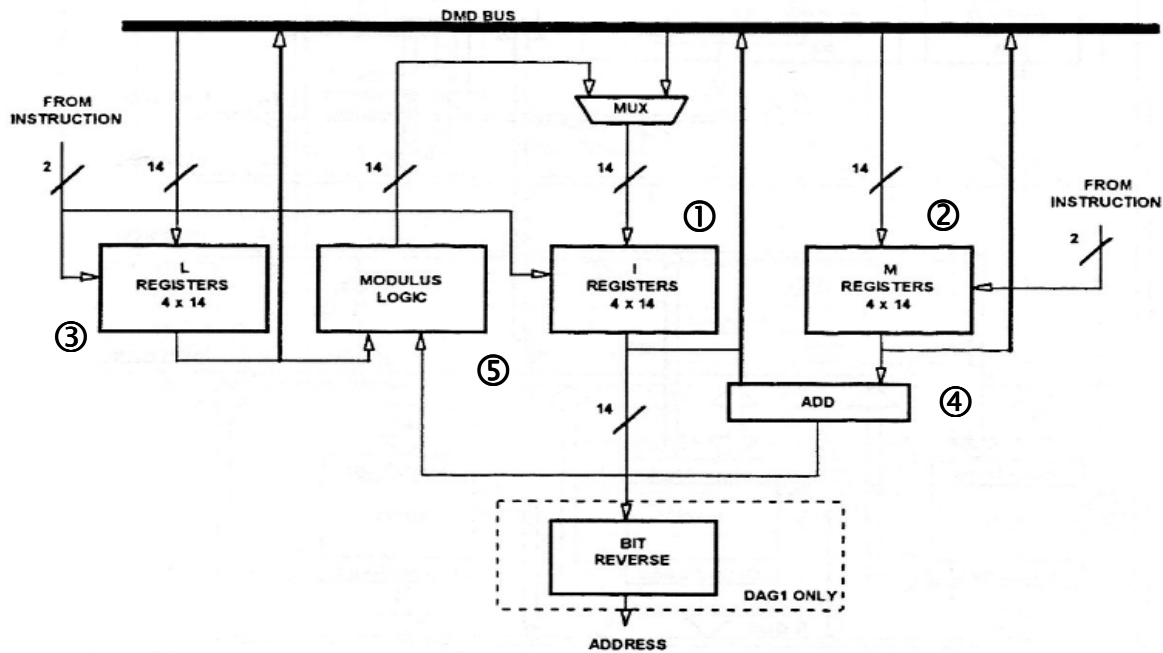


Figure II.6.

Pour comprendre l'utilité et le fonctionnement des **DAG**, on peut le voir sur un exemple qui prolongera celui déjà détaillé précédemment sur la figure II.4.

L'intérêt, pour faire le lien avec le hardware que l'on vient de présenter, est d'examiner une instruction assembleur DSP (en TP on programmera le DSP en langage C, mais dans ce cas, il serait difficile d'être très proche de ces aspects hardware). Rassurez-vous cependant, vous n'aurez pas à apprendre cet assembleur DSP.

Voici la ligne assembleur présentée :

MR=MR+MX0*MY0,

MX0=PM(I0,M0),

MY0=DM(I1,M1)

On a l'impression qu'il y a en fait 3 instructions assembleur, comme si elles allaient se dérouler les unes après les autres. Mais il faut bien comprendre qu'en fait elles se dérouleront en même temps et en parallèle **pendant le même tick q d'horloge**, car elles font travailler des unités différentes.

Ce type de syntaxe, très compacte, est une caractéristique de la programmation des DSP.

La figure II.7. ci-après explique quels sont les blocs internes du DSP qui seront sollicités pendant l'exécution de cette ligne de code.

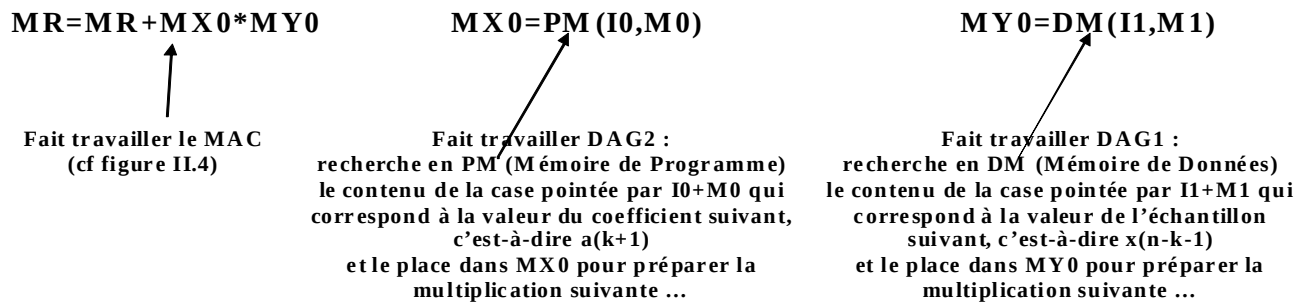


Figure II.7.

Cette architecture hardware (où les différentes unités de calcul sur les données ou sur les adresses peuvent travailler en parallèle) permettra une implémentation optimale, compacte et très rapide en temps d'exécution pour les algorithmes de calculs comme ceux des filtres numériques par exemple. Mais aussi de façon très générale pour tout type de traitement numérique du signal qui souvent, à la fin des fins, se réduisent, comme on l'a déjà souligné, à des opérations arithmétiques sur des valeurs numériques prélevées sur les signaux.

Remarque : dans cet exemple, on remarque que l'exécution de cette instruction nécessite entre autre un accès vers la mémoire de programme **PM** (pour y lire un des coefficients du filtre). Si on se rappelle que (cf. figure I.4.), pendant l'exécution d'une instruction quelconque (q), le DSP allait rechercher en mémoire de programme **PM** l'instruction (q+2), il y a conflit possible entre ces 2 accès à des adresses différentes dans la mémoire **PM** **pendant le même tick d'horloge**. Les concepteurs du DSP ont trouvé la parade pour éviter ce conflit et faire en sorte qu'il ait le minimum de conséquences sur le temps d'exécution des programmes : en effet, on verra plus loin que grâce au concept de **mémoire cache**, ce conflit n'aura lieu qu'une seule fois dans une boucle (la première fois où il se produit ...).

II.4. Le séquençage du programme.

Le séquenceur de programme a été conçu pour optimiser le déroulement du programme.

Son synoptique général est donné en figure II.8.

Il doit fournir, à chaque tick d'horloge, l'adresse de la case mémoire où se trouve l'instruction suivante à exécuter.

Cette adresse doit correspondre à de la mémoire de programme **PM**, et c'est pour cette raison qu'on trouve en sortie du séquenceur de programme (et en bas de la figure II.8.) le **PMA BUS**.

On voit qu'il joue en quelque sorte le rôle du **PROGRAM COUNTER (PC)** chez Motorola) ou de l'**Instruction Pointer (IP)** chez Intel). On rappelle que le **PC** pointe sur la case mémoire qui contient la prochaine instruction à exécuter.

Ici, le séquenceur de programme joue en quelque sorte le rôle d'un « super **PROGRAM COUNTER** » architecturé autour du **PROGRAM COUNTER** de base que l'on peut voir sur la figure II.8.

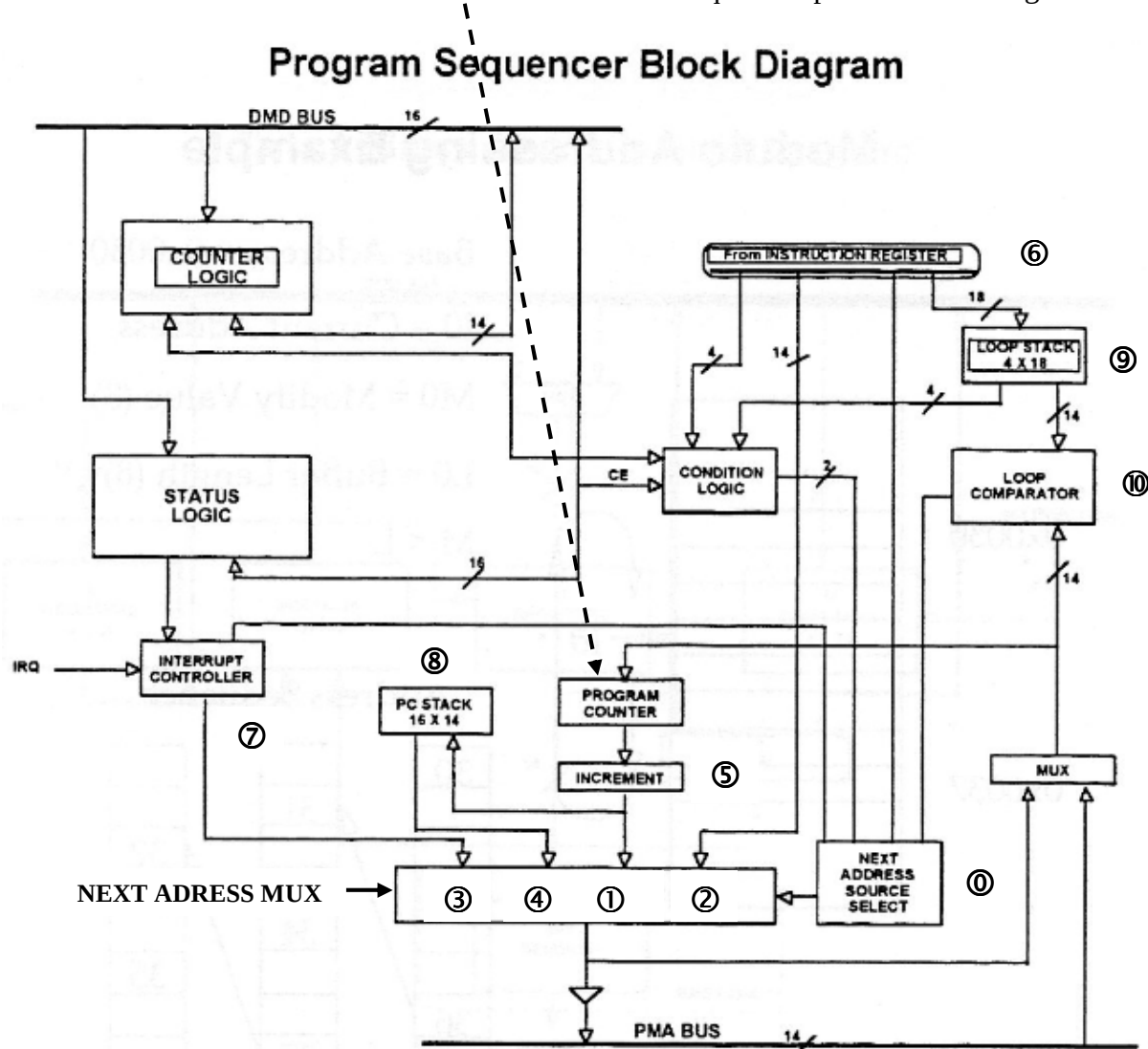


Figure II.8.

Pour déterminer l'adresse où se trouve la prochaine instruction à exécuter, il y a dans le cas général, 4 possibilités, et c'est le rôle du **NEXT ADDRESS MUX ⑩** du bas de la figure II.8. de déterminer cette adresse en fonction de chacune de ces 4 possibilités.

Cas ① : l'adresse de la prochaine instruction se trouve dans la case mémoire contigüe suivante (c'est en fait un cas très fréquent, peut-être 99% du temps, voire plus).

Pour la déterminer il suffit d'incrémenter la valeur courante du **PROGRAM COUNTER (PC)** : d'où le bloc **INCREMENT ⑤**.

Ce cas correspond au déroulement séquentiel du programme.

Cas ② : l'adresse de la prochaine instruction est issue de l'instruction en cours : sur la figure II.8. elle est fournie par l'**INSTRUCTION REGISTER ⑥**.

Par exemple en cas de branchement conditionnel ou inconditionnel à une autre adresse en mémoire ou en cas de saut à un sous-programme...

Ce cas correspond à ce qu'on appelle une rupture de séquence sous contrôle logiciel.

Cas ③ : l'adresse de la prochaine instruction correspond à celle d'un vecteur d'interruption fourni par un contrôleur d'interruption (**INTERRUPT CONTROLLER ⑦**).

Cela correspond en fait un saut à la première instruction de la routine de traitement d'une interruption matérielle par exemple.

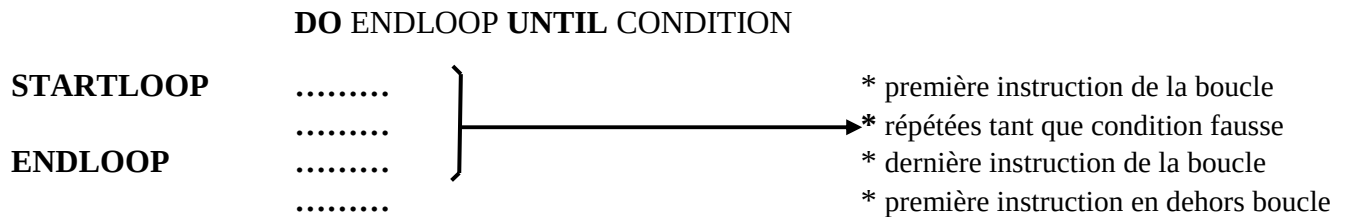
Ce cas correspond à ce qu'on appelle une rupture de séquence sous contrôle matériel.

Cas ④ : l'adresse de la prochaine instruction provient de la **PC STACK ⑧** : c'est en fait le cas le plus intéressant mais aussi le plus dur à comprendre. Il met en œuvre la **PC STACK ⑧** (pile de PC), la **LOOP STACK ⑨** (pile de boucle) et la fonction **LOOP COMPARATOR ⑩** (comparateur de boucle) que l'on peut voir sur la figure II.8.

Ces éléments permettent, comme on va le voir, une gestion optimale des boucles du type jusqu'à ce qu'une condition de sortie soit vraie (qui sont souvent utilisées en traitement numérique du signal).

En général, dans ce type de boucle, il y a toujours un test de sortie de boucle : tant que la condition de sortie est fausse, on répète la boucle une nouvelle fois. Par contre, cette procédure de test peut entraîner un ralentissement dans le temps d'exécution de la boucle car il faut rajouter des instructions pour vérifier si cette condition de sortie est vraie ou fausse (tests couplés avec des sauts conditionnels).

Ici, pour ne perdre aucun cycle (tant dans les phases d'exécution que dans les phases de décodage et de recherche d'instruction en mémoire), on optimise le fonctionnement. Voyons cela sur une instruction assembleur de gestion de boucle de la forme :



Cette syntaxe parle d'elle-même : faire toutes les instructions suivantes jusqu'à l'adresse **ENDLOOP** (ce qu'on appelle en général le « corps de boucle ») jusqu'à ce que la condition de sortie de boucle **CONDITION** soit vraie : ainsi toutes les instructions de ce corps de boucle, comprises entre les adresses **STARTLOOP** (adresse de l'instruction qui suit le **DO** précédent et qui correspond donc au début de la boucle) et **ENDLOOP** (fin de la boucle), seront exécutées tant que la condition de sortie de boucle **CONDITION** est fausse.

Lorsque la condition de sortie **CONDITION** est vraie, on sort de la boucle en passant à l'adresse qui suit l'adresse **ENDLOOP** (et qui correspond à la première instruction en dehors de la boucle).

Nous allons appliquer cette structure pour réaliser un petit algorithme permettant de calculer l'équation au récurrence d'un filtre numérique avec 10 coefficients $a(j)$ (pour $j = 0$ à 9) :

$$y(n) = \sum_{j=0}^9 a(j).x(n-j) \quad (1)$$

Le code aurait une structure de la forme :

ADRESSES	INSTRUCTIONS	COMMENTAIRES
	CNTR=10	initialisation du compteur de boucle : ici la boucle sera répétée 10 fois car 10 coefficients pour l'équation au récurrence de ce filtre.
	DO ENDLOOP UNTIL CE	à chaque boucle CNTR est automatiquement décrémentée. Quand CNTR=0 le flag CE (Counter Expired $\equiv$ compteur à 0) passe à 1 et on sort de la boucle (condition de sortie vraie). Mais tant que CE reste à 0, on refait la boucle une fois de plus.
STARTLOOP	MR=MR+MX0*MY0, MX0=PM(I0,M0), MY0=DM(I1,M1)	Première instruction de la boucle. Ici, corps de boucle pour le calcul de $y(n)$ (dans MR) et la recherche du coefficient $a(j)$ et de l'échantillon $x(n-j)$ suivant (ces instructions ont été présentées auparavant sur la figure II.7.)
ENDLOOP	Yn=MR	Dernière instruction de la boucle. Mise à jour de la variable courante Yn (sortie du filtre) qui se calcule au fur et à mesure des 10 passages dans la boucle.
	Sortie de Yn vers le CNA	Première instruction en dehors de la boucle. Envoi de la valeur Yn (correspondant à la formule (1) ci-dessus) vers le CNA
	Instruction suivante	...

Comment optimiser ce fonctionnement pour ne perdre aucun cycle ?

Au moment de l'exécution du « DO ENDLOOP UNTIL CE », l'adresse **ENDLOOP** est empilée dans la **LOOP STACK** et l'adresse **STARTLOOP** est empilée dans la **PC STACK**.

Ces 2 opérations étant faites automatiquement et de façon transparente pour le programmeur (en fait l'architecture hardware interne du DSP est prévue pour cela...).

Ainsi quand la dernière instruction de la boucle (à l'adresse **ENDLOOP**) est exécutée (en fait 2 instructions avant à cause du pipe : n'oublions pas que pendant que l'instruction (q) est exécutée, l'instruction (q+2) est recherchée en mémoire), ce que le séquenceur sait grâce au **LOOP COMPARATOR** câblé (qui compare l'adresse de l'instruction recherchée avec le sommet de la **LOOP STACK**, c'est-à-dire **ENDLOOP**), le séquenceur vérifie si la condition de sortie est satisfaite (ici si CE vaut 1) :

- si ce n'est pas le cas (**CE=0**), il dépile de la **PC STACK** l'adresse **STARTLOOP** de la première instruction de la boucle et la place sur le bus **PMA** via le **NEXT ADDRESS MUX** de sortie (et ce qui explique le cas ④) : la boucle est exécutée une fois de plus,

- si c'est le cas (**CE=1**), il saute à l'instruction suivante (première instruction en dehors de la boucle), et tout ceci pendant l'exécution de la dernière instruction de la boucle, donc sans perdre un seul cycle machine. Impressionnant non ???

Remarque :

Notons également que le fait d'utiliser des structures de type pile et donc **LIFO** (Last In First Out) pour la **PC STACK** et la **LOOP STACK** permettra d'**imbriquer** une deuxième boucle dans la première boucle.

Tout ce mécanisme est résumé dans les deux tableaux de la figure II.9. suivante :

CAS D'UNE RE-ITERATION DE LA BOUCLE (CE=0):

Sens de déroulement des cycles machine : $\longrightarrow$

Exécution instruction	f-2	f-1	f	d
Décodage instruction	f-1	f	d	d+1
Recherche instruction	f	d	d+1	d+2
	condition de fin de boucle fausse (CE=0) (testée deux instructions avant la fin de boucle)	recherche dans la PC STACK de l'adresse de début de boucle		on exécute une fois de plus la boucle

CAS D'UNE TERMINAISON NORMALE DE LA BOUCLE (CE=1):

Sens de déroulement des cycles machine : $\longrightarrow$

Exécution instruction	f-2	f-1	f	f+1
Décodage instruction	f-1	f	f+1	f+2
Recherche instruction	f	f+1	f+2	f+3
	condition de fin de boucle vraie (CE=1) (testée deux instructions avant la fin de boucle)	recherche de la première instruction en dehors de la boucle		on exécute l'instruction qui suit la boucle

f = adresse de l'instruction de fin de boucle, c'est-à-dire ENDLOOP,

d = adresse de l'instruction de début de boucle, c'est-à-dire STARTLOOP.

Figure II.9.

Finalement, et dans le cas général, on utilisera deux buffers circulaires de N valeurs (via les registres de modulo **Li** des **DAG**) pour stocker les échantillons et les coefficients d'un filtre numérique. Le calcul de la valeur de sortie **y(n)** du filtre sera effectué grâce à une boucle de ce type entre 2 prises d'échantillons successives de la valeur d'entrée **x(n)** du filtre (entre t_n et t_{n+1}). Souvent, un sous-programme d'acquisition fonctionnant sous interruption par exemple (voir comment plus loin), viendra mettre à jour le buffer des échantillons acquis.

Il faudra bien sûr que le temps de calcul de la sortie courante **y(n)** du filtre (durée d'exécution de la boucle multiplié par le nombre de passage dans la boucle) soit inférieur à l'intervalle de temps séparant deux acquisitions successives de l'entrée **x(n)**.

$(t_{n+1} - t_n = T_e = \frac{1}{F_e})$ avec F_e représentant la fréquence d'échantillonnage) car sinon le filtre ne sera pas réalisable.

Mais on voit que tout a été fait au niveau du hardware du DSP pour que le temps de calcul ($\equiv$ durée totale de la boucle) soit le plus court possible pour que la période d'échantillonnage T_e puisse être choisie la plus petite possible afin qu'on puisse travailler avec une fréquence d'échantillonnage F_e la plus grande possible dans le but bien sûr de respecter les limites imposées par le théorème de Shannon.

II.5. En résumé.

Le synoptique de la figure II.10. reprend les différents éléments qui viennent d'être présentés.

C'est l'architecture interne de base commune à tous les DSP de la famille 21xx.

ADSP-2100 Family Base Architecture

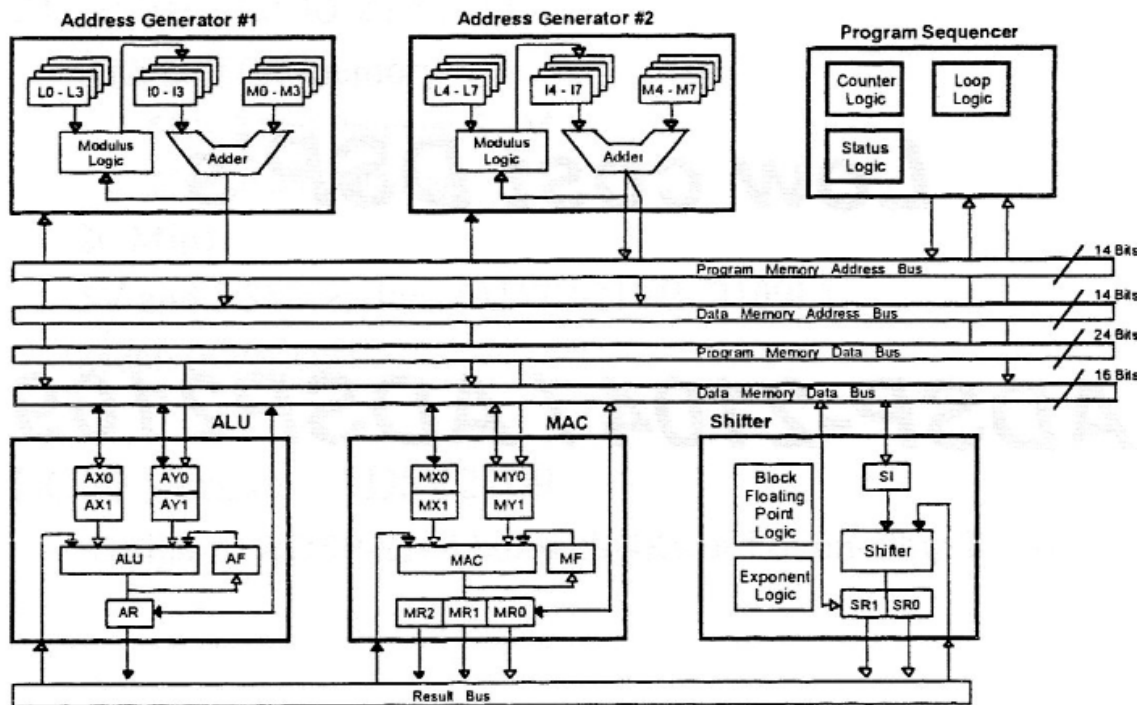


Figure II.10.

II.6. Les différents éléments de la famille 21xx.

Avant d'en terminer avec la famille 2xx, essayons d'en présenter quelques uns de ses éléments, et notamment ceux qui avaient été introduits sur le tableau de la figure I.5. (liste qui n'était pas du tout exhaustive comme on l'avait dit).

Voici donc les principales caractéristiques de ces différents composants qui se rajoutent à l'architecture de base présentée sur la figure II.10. précédente.

- ADSP 2101 :

La figure II.11. ci-dessous montre l'architecture interne d'un ADSP 2101.

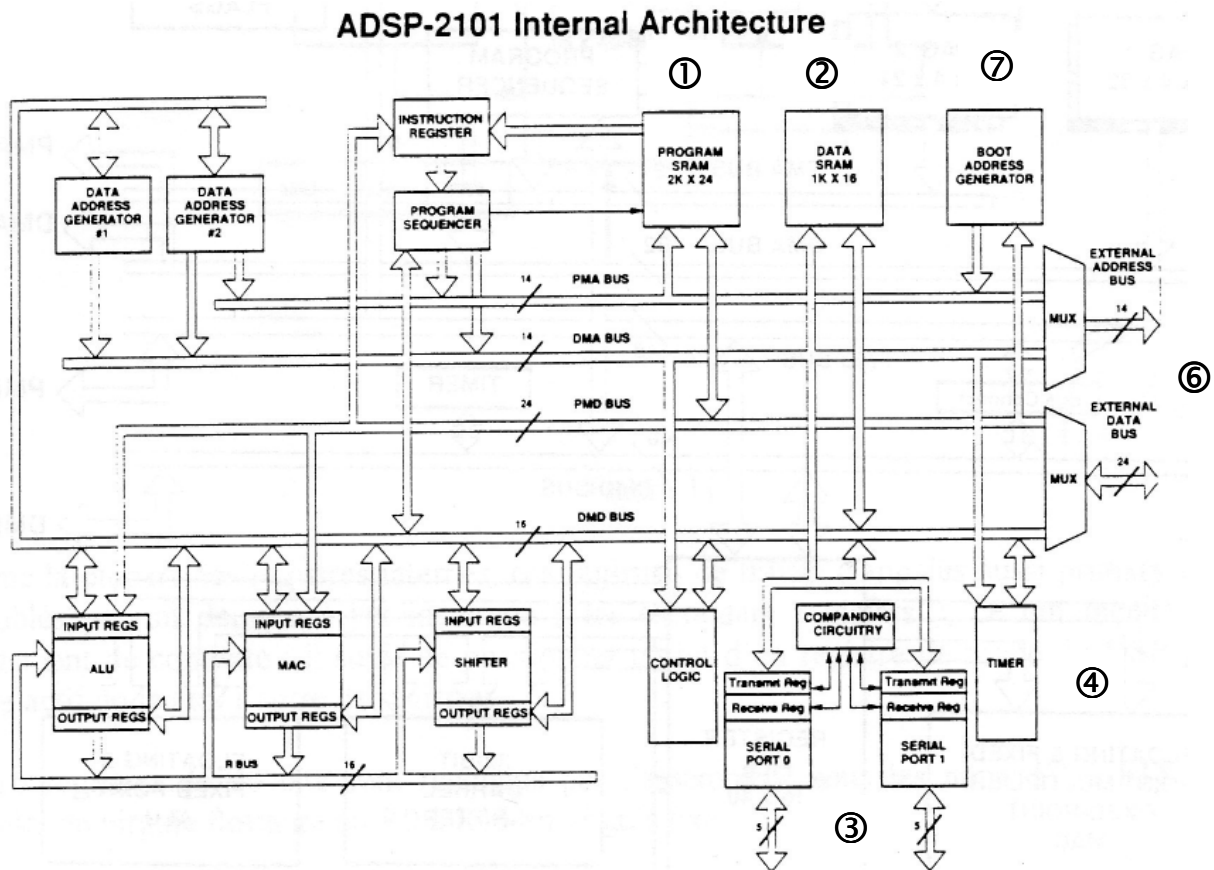


Figure II.11.

On peut y distinguer, entre autre (voir les points ① à ⑦ sur la figure II.11.) :

- ① 2 K x 24 bits de mémoire **SRAM** interne de programme (pour la mémoire de code **PM**),
- ② 1 K x 16 bits de mémoire **SRAM** interne de données (pour la mémoire de données **DM**),
- ③ deux liaisons séries (**SPORT0** et **SPORT1**) bidirectionnelles et indépendantes fonctionnant sous interruption (voir plus loin pour le détail de ce type de fonctionnement) : elles permettent d'assurer le dialogue avec un **CODEC** (**CAN** en entrée et **CNA** en sortie),
- ④ un **timer programmable** fonctionnant aussi sous interruption : pour définir par exemple une fréquence d'échantillonnage F_e ,

⑤ **trois signaux d'interruptions matérielles** : $\overline{\text{IRQ0}}$, $\overline{\text{IRQ1}}$ et $\overline{\text{IRQ2}}$

(non représentés sur la figure II.11.)

Notons que les 6 sources d'interruptions précédentes
(2 pour les SPORT + 1 pour le timer + 3 interruptions matérielles)
peuvent au choix être :

- soit imbriquables avec notion de niveau de priorité,
- soit non interruptibles (c'est-à-dire masquables).

⑥ possibilité de rajouter des mémoires externes (pour du code ou des données supplémentaires ou même pour le rajout de circuits périphériques programmables) des signaux externes sur le composant (mais non représentés sur la figure II.11.) permettant de sélectionner l'une ou l'autre de ces mémoires externes (pilotage par les signaux $\overline{\text{PMS}}$ pour **Program Memory Select** et $\overline{\text{DMS}}$ pour **Data Memory Select**),

⑦ un **BOOT ADDRESS GENERATOR** pour initialiser au reset les RAM internes avec le programme d'initialisation du DSP, les vecteurs d'interruptions ... via une **EPROM** externe (sélectionnée par le signal $\overline{\text{BMS}}$ pour **Boot Memory Select**) : au reset, et si on le souhaite (en fonction de la tension sur l'entrée **MMAP** du DSP) une phase de chargement automatique permet d'initialiser la RAM interne de programme du DSP avec le contenu d'une zone située dans une EPROM externe, ce processus de chargement se terminant par un saut à l'adresse \$0000 de cette RAM interne où doit bien sûr se trouver la première instruction du programme.

- ADSP 2103 :

idem que le **ADSP 2101** mais alimenté en 3,3 V pour des applications portables et donc nécessitant une faible consommation. En effet, si la tension d'alimentation diminue, les durées de commutation entre les états haut et bas diminuent et les pertes aussi (car elles ont essentiellement lieu pendant ces commutations et non pendant les états haut et bas).

- ADSP 2105 :

version compatible avec le **ADSP 2101** mais néanmoins simplifiée (et donc moins chère) avec moins de mémoire interne, une seule liaison série ...

- ADSP 2106 :

version **ROMée** (masquée) de l'**ADSP 2105**, pour des applications fabriquées en grande quantité : production directe de ce composant en usine avec son programme déjà présent sur les masques de conception et de fabrication (donc plus de programmation à postériori).

- ADSP 2111 :

même composant que le **ADSP 2101** avec en plus une interface **HIP** (**Host Interface Port**) permettant une initialisation supplémentaire en plus de celle vue précédemment via une **EPROM** externe : ce qui permet par exemple de télécharger du code via un PC (très utile pendant les phases de développement).

- ADSP 21msp50/51 :

notons également l'existence des **ADSP 21msp50/51** (**Mixed Signal Processor**) intégrant sur la même puce les convertisseurs **AD** (**CAN**) et **DA** (**CNA**) et leurs filtres associés.

III. LA FAMILLE 21xxx.

L'architecture au niveau du cœur du processeur est comparable avec la famille 21xx, comme on peut le voir en examinant le schéma de la figure III.1.

On y retrouve :

- les 3 unités de calculs internes : **ALU**, **MAC** et **SHIFTER**,
- les 2 **DAG** : **DAG1** et **DAG2**,
- le **PROGRAM SEQUENCER**.

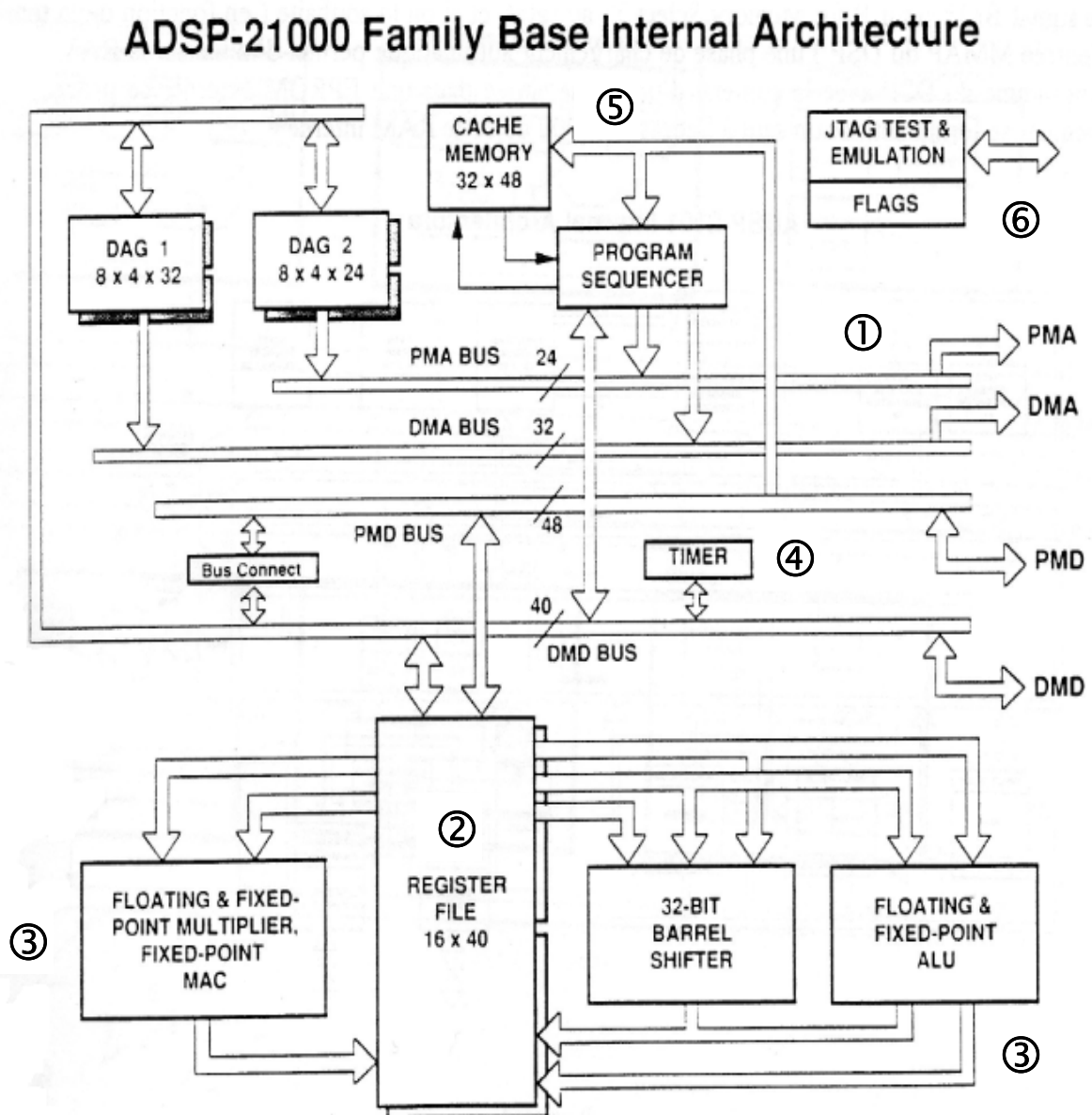


Figure III.1.

Voici cependant les quelques différences notables, qui permettront d'améliorer les performances de cette famille, et qui sont repérées par les points ① à ⑥ sur le schéma de la figure III.1.

- ① L'augmentation de la taille des bus internes par rapport à ceux de la famille 21xx permettant d'améliorer la vitesse de traitement et les tailles des mémoires de code et de données,
- ② Les registres de travail des différentes unités de calcul internes ont été **banalisés** en un ensemble de 16 registres (**Register file 16X40 Bit**) disponibles pour ces différentes unités de calcul (ALU, MAC ou SHIFTER).

Cette nouvelle architecture permet les échanges d'informations entre elles en facilitant et en fluidifiant les mouvements de données et les stockages de résultats temporaires ou intermédiaires, comme le montre la figure III.2. ci-dessous :

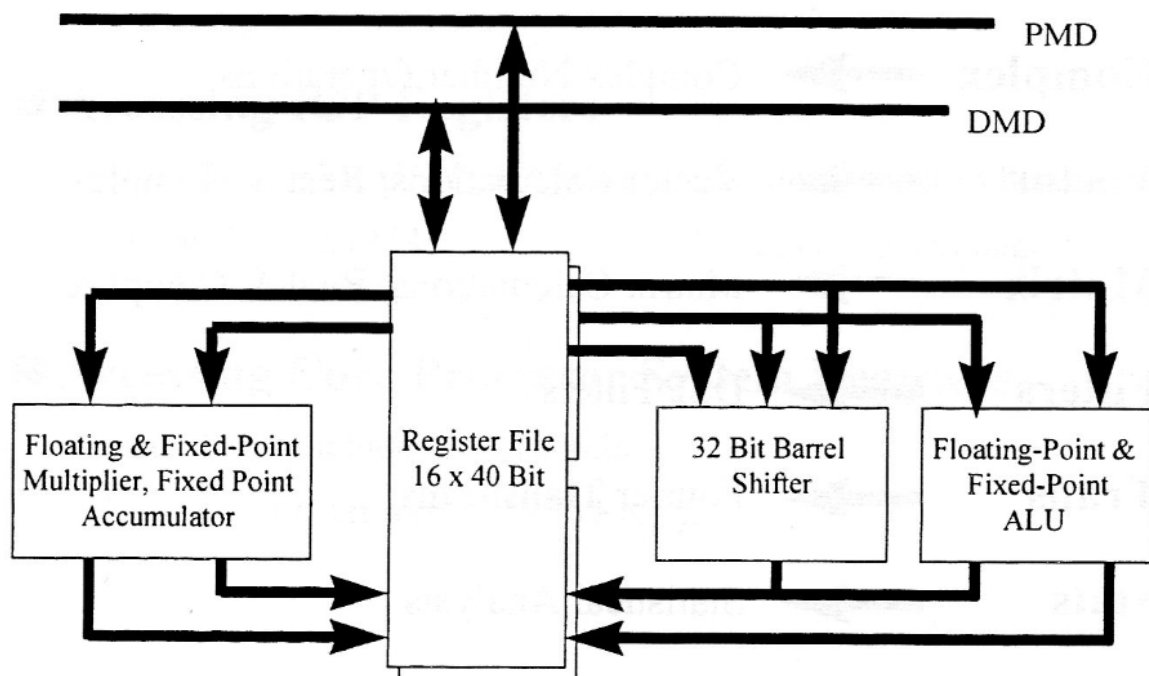


Figure III.2.

Comme la plupart des registres internes, ces registres de travail (appelés aussi « **primary registers** ») sont dédoublés par un deuxième jeu identique (les « **secondary registers** »), ce qui facilite et accélère le changement de contexte : il suffit de positionner un bit d'un registre de mode du DSP pour valider et rendre actif un jeu ou l'autre de ces registres.

Remarque : ces 16 registres de 40 bits sont reconnus par l'assembleur sous les noms **F0** à **F15** si on souhaite travailler en virgule flottante ou **R0** à **R15** pour les calculs en virgule fixe.

- ③ Contrairement à la famille 21xx, la nouvelle famille 21xxx fonctionne en virgule flottante sur 32 bits (**Single Precision** : standard IEEE 754/854) ou sur 40 bits (**Extended Precision**), avec possibilité bien sûr de travailler quand même en virgule fixe si on le souhaite : format entier ou fractionnaire.

On peut ici ouvrir une parenthèse pour rappeler et préciser à ce sujet quelques éléments sur le format et le codage des nombres supportés par la famille 21xxx :

Les entiers sur 32 bits :

- **entiers non signés** : la virgule est implicitement à droite du **LSB** (bit 0) (cf. figure III.3.) :

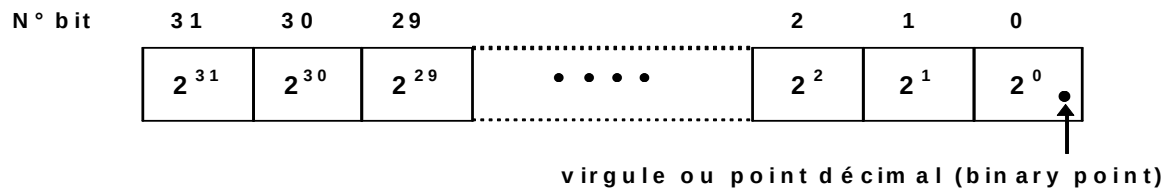


Figure III.3.

- **entiers signés** : le **MSB** (bit 31) est le **bit de signe s**. Il vaut **0** pour les entiers **positifs** et vaut **1** pour les entiers **strictement négatifs**. La virgule est aussi implicitement à droite du **LSB** (bit 0) (cf. figure III.4.) :

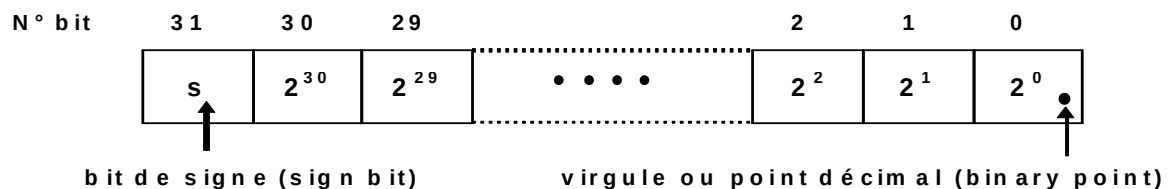


Figure III.4.

Les fractionnaires en virgule fixe sur 32 bits :

Ce codage correspond à la partie fractionnaire (à droite de la virgule) des nombres réels.

- **fractionnaires non signés** : la virgule est implicitement à gauche du **MSB** (bit 31) (cf. figure III.5.) :

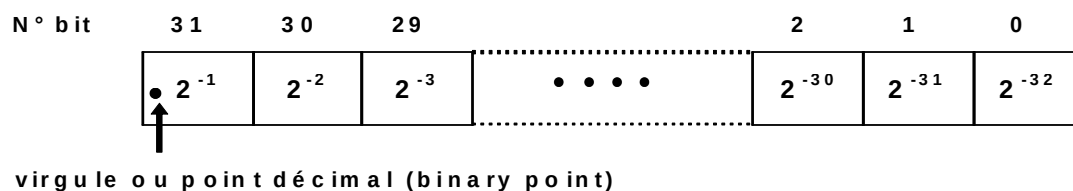


Figure III.5.

- **fractionnaires signés** : le **MSB** (bit 31) est le **bit de signe s**. Il vaut **0** pour les fractionnaires **positifs** et vaut **1** pour les fractionnaires **strictement négatifs**.

La virgule est implicitement à droite du **MSB** (bit 31) (cf. figure III.6.) :

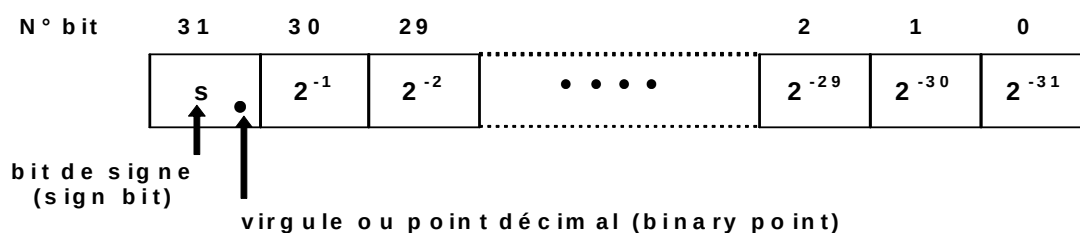


Figure III.6.

Les réels en virgule flottante :

Dans ce type de format, proche de la notation scientifique avec exposant, le réel x est représenté par la donnée d'une **mantisse signée m** (appelée aussi **significande**) et d'un **exposant e** . Ceci permet d'écrire x sous la forme $x = m.2^e$. En faisant varier e , on fait « flotter » la virgule.

Les tailles de m et de e dépendent de la précision.

- **sur 32 bits : format simple précision (Norme IEEE 754/854)** (cf. figure III.7.) :

La mantisse m est codée sur 23 bits et l'exposant e est codé sur 8 bits.

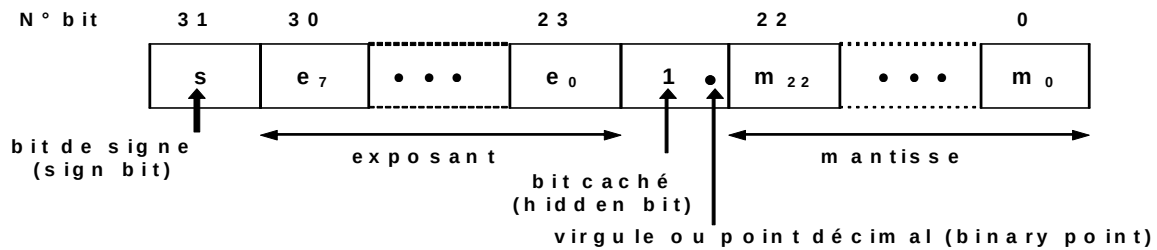


Figure III.7.

s : bit de signe du réel x ,

$e_7 \dots e_0$: codage de l'exposant e en non signé ($1 \leq \text{exposant codé} \leq 254$) avec un offset de 127 rajouté. L'exposant réel est obtenu en enlevant 127 à l'exposant codé.

$m_{22} \dots m_0$: codage de la mantisse m sur 23 bits (la virgule est entre m_{22} et le hidden bit).

Les différents cas possibles sont résumés dans le tableau ci-dessous :

Type	Valeur exposant	Mantisse	Valeur
NAN : Not A Number	255	différente de 0	Indéfinie
Infini	255	nulle	$\pm\infty$ suivant bit de signe s
Normal	$1 \leq \text{exposant} \leq 254$	quelconque	$(-1)^s 1, m_{22} \dots m_0 2^{e-127}$
Zéro	0	nulle	± 0 suivant bit de signe s

- **sur 40 bits : format double précision** (cf. figure III.8.) :

La mantisse m est sur 31 bits et l'exposant e toujours sur 8 bits.

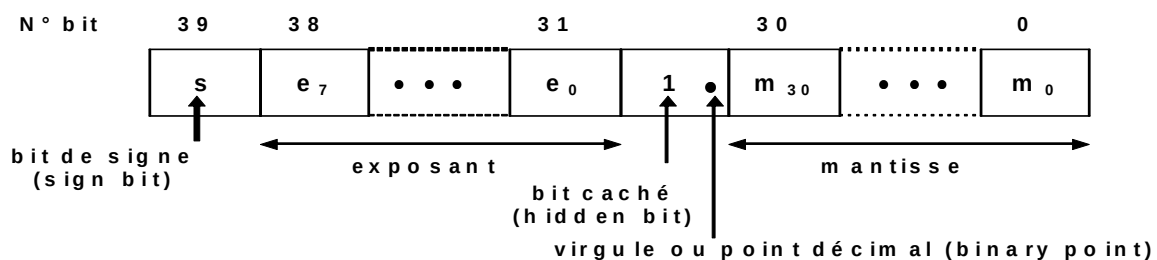


Figure III.8.

- **sur 16 bits : format court** (cf. figure III.9.) : la mantisse m est sur 11 bits et l'exposant e sur 4 bits.

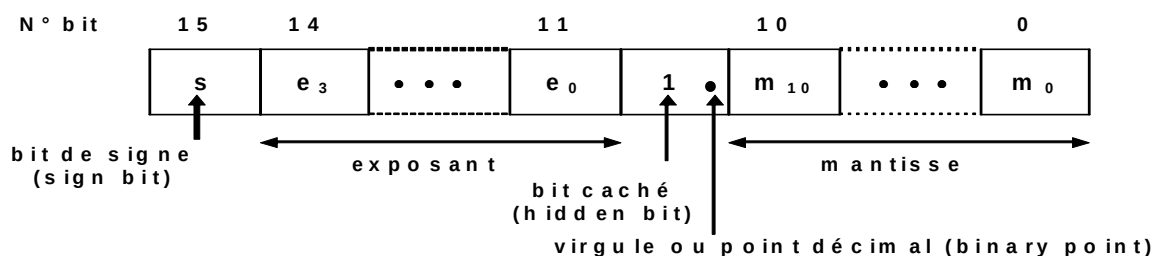


Figure III.9.

Revenons au schéma de la figure III.1. pour indiquer les 3 dernières différences notables entre la famille 21xx et la famille 21xxx.

④ Existence d'un timer sur 32 bits permettant de générer des interruptions périodiques de fréquence programmable, ce qui sera bien utile par exemple pour définir la période d'échantillonnage T_e et donc la fréquence d'échantillonnage F_e .

Car rappelons le, un DSP sert à faire du traitement numérique du signal : le choix et le paramétrage de la fréquence d'échantillonnage sont des aspects fondamentaux, notamment vis-à-vis du théorème de Shanon.

⑤ Un cache instructions (**INSTRUCTION CACHE**) a été rajouté au séquenceur de programme pour augmenter la rapidité de traitement.

Ce cache peut contenir 32 instructions.

En principe son action est transparente pour le programmeur (comme la plupart des caches par ailleurs), mais il est intéressant de comprendre son algorithme de fonctionnement, car il s'agit ici d'un cache **sélectif** : seules **certaines** instructions sont mises en cache (par opposition à un cache **associatif** où ce sont les instructions **les plus fréquemment utilisées** qui sont mises en cache)

Mais lesquelles ?

Pour comprendre cela, il faut encore revenir à l'exemple vu précédemment sur la figure II.7. et qui représentait le « cœur » de la plupart des algorithmes en traitement numérique du signal à savoir effectuer un calcul du type :

$$y(n) = \sum_{j=0}^p a(j).x(n-j) \text{ (multiplication et accumulation par addition)}$$

Cet exemple correspondait, on l'a déjà vu, à l'exécution simultanée de :

MR=MR+MX0*MY0, MX0=PM(I0,M0), MY0=DM(I1,M1)

Normalement le DSP recherche une instruction en mémoire à chaque tick q d'horloge.

Si on se rappelle que les phases de recherche de l'instruction (q+2) (instruction fetch), de décodage de l'instruction (q+1) (instruction decode) et d'exécution de l'instruction (q) (instruction exécution) ont lieu en parallèle, et donc pendant le même tick d'horloge q (fonctionnement en pipeline), il se peut, comme dans le cas ci-dessus, que l'exécution de l'instruction (q) demande un accès (en lecture ou en écriture) pour des données situées dans la mémoire de code (ici pour charger **MX0**) et donc dans la même mémoire que celle demandée par la phase de recherche de l'instruction (q+2) : on dit qu'il y a **conflit** sur la mémoire **PM** (car **double accès simultané** à la **même** mémoire **PM**).

Sans le cache, on perd un cycle. Et comme en général ce type d'instruction est, comme on l'a vu, située dans une boucle du type **DO ENDLOOP UNTIL CONDITION**, ce conflit se produira autant de fois que cette boucle sera exécutée, ce qui n'est pas acceptable, car on perdra un cycle à chaque fois.

Le cache sert à réduire (en moyenne) le délai de traitement de ce type de conflit : la première fois que ce conflit se produit (et donc à la première exécution de la boucle) le DSP attend un cycle pour rechercher l'instruction (q+2) car la priorité est donnée à l'exécution de cette instruction (q).

Et il place ensuite l'instruction recherchée (q+2) dans le cache pour éviter que ce conflit ne se reproduise à nouveau, à la prochaine itération de la boucle. C'est en ce sens que le cache est sélectif.

Ainsi, à chaque cycle, le DSP (en fait son séquenceur de programme) vérifie si l'instruction (q+2) n'est pas dans le cache : si elle y est, il peut y accéder et en même temps peut se dérouler l'exécution de ce type d'instruction nécessitant un accès en mémoire de programme : le conflit a disparu grâce au cache.

On voit finalement que tout se passe comme si on rajoutait un troisième bloc de mémoire (en plus de PM et de DM) auxiliaire à la mémoire de code.

Pour vérifier la présence d'une instruction dans le cache, le principe est de comparer l'adresse à rechercher avec les adresses présentes dans le cache : si c'est bingo, l'instruction est fournie par le cache (on dit qu'il y a cache **hit**), sinon, c'est cache **miss** : l'instruction est lue en mémoire puis mise dans le cache (et dans ce cas, le délai et la perte d'un cycle est inévitable, mais, comme on l'a dit précédemment, il ne se produira **qu'au premier passage** dans la boucle).

Il existe des instructions permettant de geler (**freeze**) le cache dans un état donné (plus de stockage supplémentaire dans le cache) ou même de stopper (**disable**, comme ne plus utiliser) le cache.

⑥ Notez également l'existence d'une interface **JTAG** (Joint Test Action Group) pour faciliter l'émulation, les tests et la mise au point en temps réel.

IV. LE DSP UTILISE SUR LA CARTE EZKIT.

La carte **EZKIT** du système de développement utilisé en TP possède un **ADSP 21061** dont le synoptique est donné en figure IV.1.

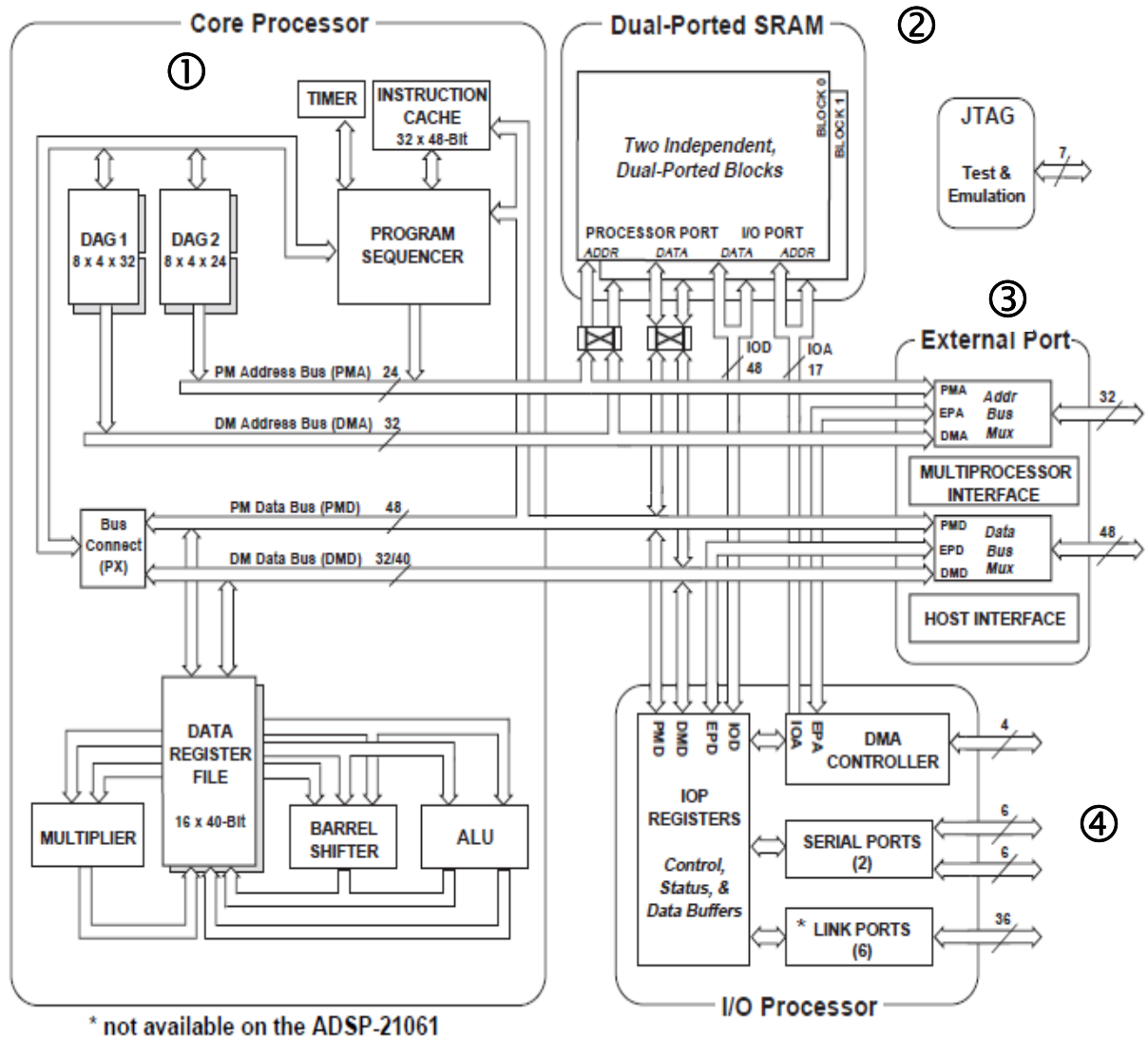


Figure IV.1.

On remarque sur cette figure les blocs numérotés ① à ④ que l'on va maintenant détailler.

IV.1. Le cœur du processeur.

① Le cœur du processeur (le block **CORE PROCESSOR**) qui est bien celui, décrit plus haut, de la famille 21xxx.

IV.2. La mémoire interne et le mapping mémoire.

② L'existence d'une mémoire **interne** (**DUAL-PORTED SRAM**) organisée en 2 blocs indépendants nommés **BLOCK0** et **BLOCK1**, dont les tailles dépendent du type de DSP, comme indiqué sur la figure IV.2.

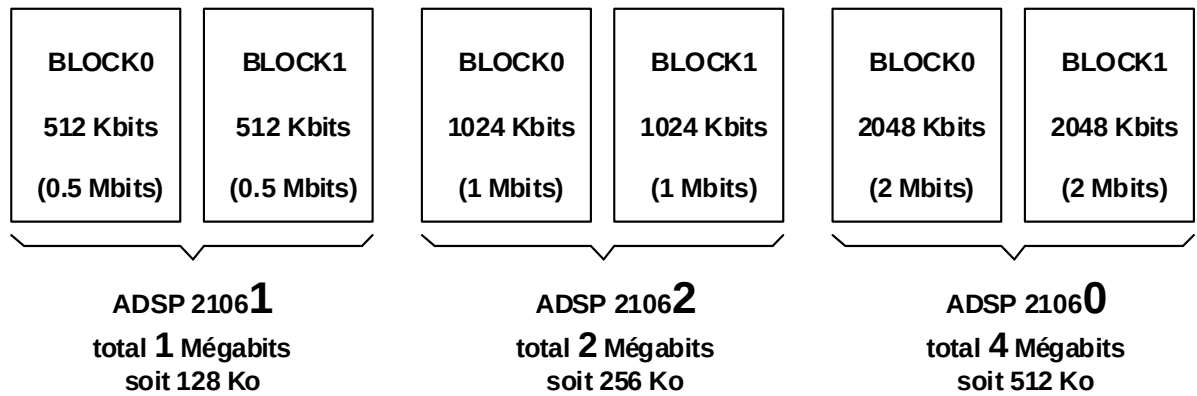


Figure IV.2.

Pour optimiser la gestion de ces 2 blocs (et permettre l'exécution d'instructions complexes utilisant les trois unités de calcul et la recherche de données en mémoire), la solution optimale consiste à ce que :

- le **BLOCK0** soit piloté par le **DAG2** via le **PMA Bus** pour servir de **PM** (pour les données de la mémoire de programme). Rappelons que c'est le séquenceur de programme qui permet d'accéder aux instructions situées aussi dans la mémoire de programme.
- le **BLOCK1** soit piloté par le **DAG1** via le **DMA Bus** pour servir de **DM** (pour les données de la mémoire de données),

Le mapping mémoire **complet** pour les ADSP 2106x est fourni sur la figure IV.3. ci-dessous.

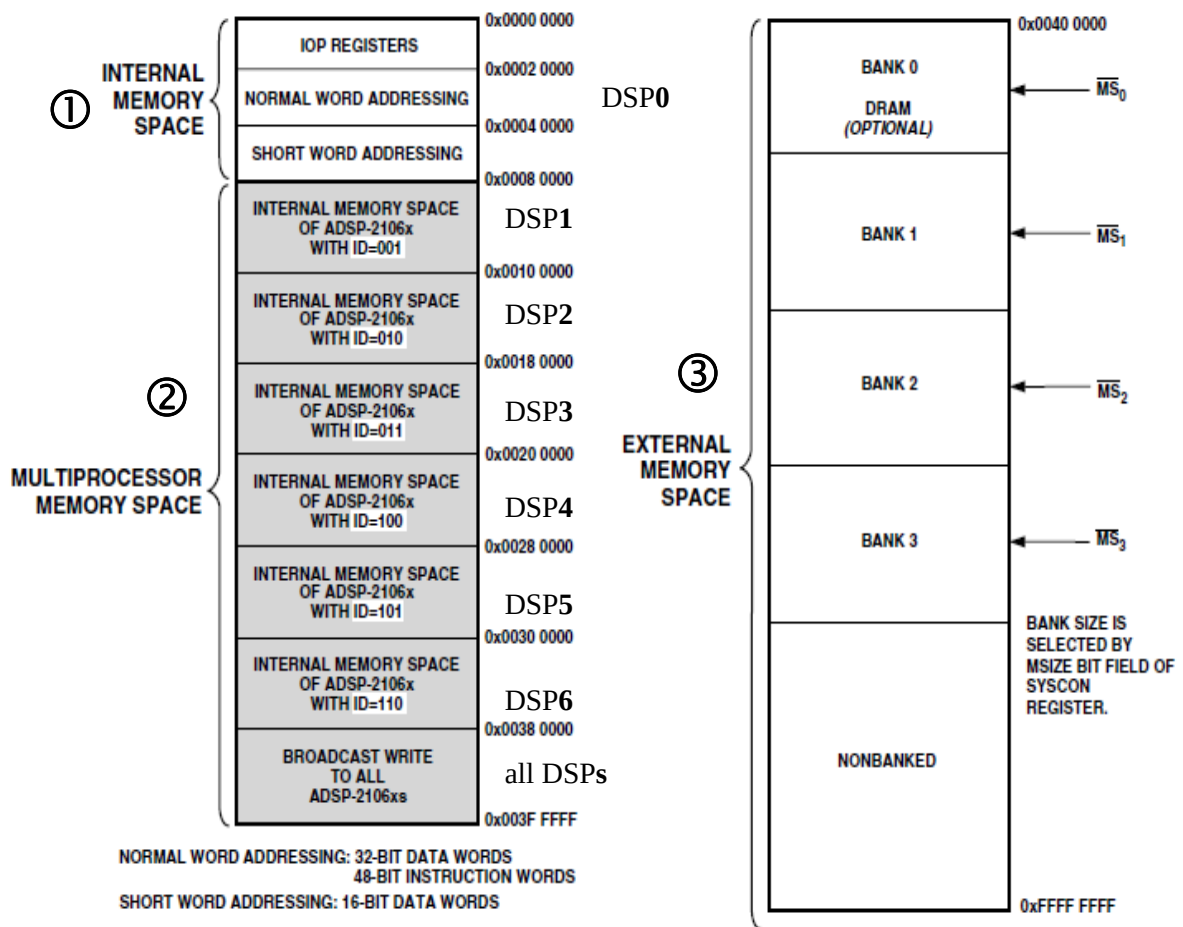


Figure IV.3.

Comme on peut le voir sur cette figure IV.3., ce mapping est constitué de 3 parties principales :

① : La mémoire **interne** du DSP0 (entre **0x0000 0000** et **0x0007 FFFF**)

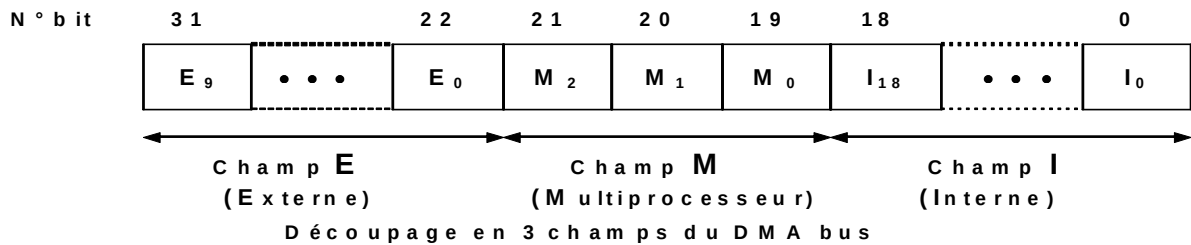
② : Les mémoires **internes** des autres DSP, dans le cas des systèmes **multiprocesseurs** (entre **0x0008 0000** et **0x003F FFFF**) : pour des applications plus complexes, on peut associer dans le même système jusqu'à 7 DSP (DSP0 à DSP6) qui peuvent travailler en parallèle dans une architecture de type maître/esclave.

A tout instant, un des DSP est le maître du système, les autres étant ses esclaves.

Cet espace, constitué des mémoires et ressources internes de tous les DSP du système appartient au mapping mémoire de chaque DSP, et est donc accessible par chaque DSP.

③ : La mémoire **externe** que l'on peut rajouter, pour augmenter la capacité mémoire totale du système. (entre **0x0040 0000** et **0xFFFF FFFF**)

Le découpage entre ces différentes zones est effectué en examinant la configuration du bus d'adresses de taille 32 bits de la mémoire de données **DM**, divisé en 3 champs comme indiqué sur la figure IV.4.



Champ **I** : bit 0 à bit 18 : sélection adresses **I**nternes (si champ **E** = 0).

Champ **M** : bit 19 à bit 21 : sélection **M**ulti-DSP (000 pour DSP0 à 111 pour all DSPs et **E** = 0)

Champ **E** : bit 22 à bit 31 : sélection adresse **E**xterne (quand champ **E** ≠ 0).

Figure IV.4.

Le principe est le même pour le bus d'adresses **PMA** de la mémoire de programme **PM**.

En développant le bus d'adresse en binaire de son **MSB** à son **LSB**, on peut retrouver les adresses du mapping de la figure IV.3.

- la mémoire interne du DSP0 (mettre les champs **E** et **M** à 0) ira de :

Adresse de début :

Champ E										Champ M			Champ I																		
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0										0			0																		

Adresse de fin :

Champ E										Champ M			Champ I																		
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0				0			0			7				F				F				F				F					

soit de **0x0000 0000** à **0x0007 FFFF** (en hexadécimal).

Ce qui correspond bien à une taille de 512 Koctets, correspondant à la capacité de l'**ADSP 21060** qui possède comme on l'a vu la plus grande capacité de mémoire interne.

Dans le cas où on utilise un **ADSP 21061** ou un **ADSP 21062**, le mapping restera le même, mais il y aura soit 3, soit 1 zone fantôme (décodage incomplet).

- la mémoire interne du DSP1 (mettre les champs **E** à 0 et **M** à 001) ira de :

Adresse de début :

Champ E										Champ M			Champ I																		
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0				0				0				8				0				0				0				0			

Adresse de fin :

Champ E										Champ M			Champ I																		
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0				0				0				F				F				F				F				F			

soit de **0x0008 0000** à **0x000F FFFF** (en hexadécimal).

La même méthode nous permettrait de retrouver les plages d'adresses pour les autres DSP.

Remarquons la zone de mémoire comprise entre les adresses **0x0038 0000** et **0x003F FFFF** qui correspond à une zone dite de « **broadcast** » :

en écrivant dans cette zone, le DSP maître peut communiquer des informations à tous les autres DSP en même temps au lieu d'avoir à écrire ces informations dans chacun des DSP à tour de rôle, ce qui permet de gagner du temps.

- la mémoire **externe** (mettre dans le champs **E** une valeur non nulle) ira de :

Adresse de début :

Champ E										Champ M			Champ I																		
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0				0				4				0				0				0				0				0			

Adresse de fin :

Champ E										Champ M			Champ I																		
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
F				F				F				F				F				F				F							

soit de **0x0040 0000** à **0xFFFF FFFF** (en hexadécimal).

Ce qui porte à **4 Giga octets** ($= 2^{32}$) la capacité d'adressage maximale d'une telle architecture

Nous terminerons ces éléments sur l'architecture de la mémoire par ces 3 remarques :

- les registres internes (**IO REGISTER**) qui servent pour la programmation de certaines fonctions du DSP comme le timer, les liaisons séries, les interruptions... se trouvent dans le mapping entre les adresses : **0x0000 0000** et **0x0000 00FF**. Ils jouent un peu le même rôle que les registres IO du PIC qui permettaient aussi de piloter toutes les interfaces de ce microcontrôleur.
- la table des vecteurs d'interruption, si elle est implantée en RAM interne doit débiter à l'adresse **0x0002 0000**.
- compte tenu de la différence de taille entre le bus de données **PMD** pour la mémoire de programme et le bus de données **DMD** pour la mémoire de données, les accès en mémoire peuvent se faire :
 - soit sur 48 bits (6 octets) : taille d'une instruction en **PM**,
 - soit sur 16 ou 32 bits (2 ou 4 octets) : taille d'une donnée en **DM** ou **PM**.

Ce qui implique des accès mémoire un peu particuliers (sur 2, 4 ou 6 octets) et une organisation et un découpage interne permettant de mixer les accès sur 16, 32 ou 48 bits dans la même mémoire. L'idée étant d'optimiser la mémoire sans perdre de position. En effet, si on s'y prend mal, par exemple en définissant des zones de taille constante de 48 bits (c'est la manière la plus simple qui permet de stocker des informations de taille 16 bits, 32 bits et 48 bits en prenant la taille maximum de ces différentes informations), on risque de perdre de la mémoire quand on placera 16 bits ou 32 bits dans un « récipient » pouvant contenir 48 bits (un peu comme si la mémoire devenait alors une espèce de « gruyère » avec beaucoup de « trous »..., et donc beaucoup de place perdue...).

Nous ne détaillerons pas ici ces spécificités et les solutions permettant d'éviter de transformer la mémoire en gruyère. Vous pouvez toujours vous reporter à la documentation constructeur pour en savoir plus...

Mais tous ces aspects purement électroniques seront pris en compte par le LINKER du compilateur C grâce au fichier d'architecture matérielle que l'on présentera plus loin et qui permet de détailler la cartographie mémoire complète du système.

Revenons maintenant au schéma de la figure IV.1. pour présenter les 2 derniers blocs (③ et ④) de l'architecture interne du **DSP 21061**.

IV.3. Interfaçage avec l'extérieur.

③ Une interface avec l'extérieur (**EXTERNAL PORT**).

IV.3.1. Les bus externes.

Cette interface permet de fabriquer les **bus d'adresses sur 32 bits (ADDR BUS MUX)** et de **données sur 48 bits (DATA BUS MUX) externes** qui piloteront la mémoire externe.

IV.3.2. Multi-processeurs.

Cette interface permet aussi le couplage et le contrôle des différents DSP dans un système multiprocesseurs (**MULTIPROCESSOR INTERFACE**).

Dans ce cas, plusieurs DSP partagent les mêmes bus externes, chacun pouvant, comme on l'a vu, devenir à son tour, le maître de ces bus (et contrôler les bus externes d'adresses et de données).

Le maître peut lire ou écrire dans les mémoires ou les registres internes des esclaves ou bien faire du « broadcast » dans la zone mémoire prévue à cet effet (zone all DSPs comprise entre les adresses **0x0038 0000** et **0x003F 0000**)

IV.3.3. Le processeur hôte.

Il y a aussi une possibilité pour l'interfaçage avec un processeur hôte (**HOST PORT**) : ce pourra être un PC pour écrire, compiler, linker, simuler, télécharger et débbugger les softs de traitement du signal qui tourneront sur le DSP du système d'évaluation.

Notons cependant que sur le système **EZ-KIT** que nous utiliserons en TP, la liaison avec le PC hôte est faite par un **UART** (voir plus loin).

IV.4. Un processeur d'entrées / sorties.

④ L'existence d'un processeur d'entrées / sorties (**I/O PROCESSOR**), indépendant du Core Processor, et qui comprend :

IV.4.1. Les registres de contrôle pour les IO.

Ces registres de contrôle pour les IO (**IOP REGISTERS**) sont mappés dans la mémoire interne comme on l'a vu plus haut entre les adresses **0x0000 0000** et **0x0000 00FF**.

IV.4.2. Les deux ports série SPORT0 et SPORT1.

Notons l'existence de deux ports série **SPORT0** et **SPORT1** (**SERIAL PORTS**).

Il s'agit de 2 liaisons **séries synchrones** et **indépendantes**.

Ces liaisons ont la partie réception et la partie transmission indépendantes, autorisant le fonctionnement en vrai full duplex. Elles peuvent également fonctionner sous interruption (voir plus loin comment).

Une de ces liaisons (SPORT0 sur l' EZ-KIT) permet ici d'assurer la liaison avec un **Codec** (ici un **Sound Port Stéréo Codec** de référence **AD 1847** orienté vers le **traitement du son**) que nous détaillerons plus loin.

Le protocole de liaison entre le DSP et le Codec est le **TDM** (**T**ime **D**ivision **M**ultiplexed) dont nous parlerons également dans la suite de ce polycopié.

IV.4.3. Un contrôleur de DMA (Direct Memory Access)

La notion de **DMA** est liée à celle de transfert de données dans un système informatique.

Faire du transfert de données entre les mémoires internes et l'extérieur (mémoire externe, liaisons série, périphériques externes, processeur hôte ...) ou même entre les mémoires internes elles-même, est une tâche relativement fréquente dans le cas du traitement du signal : transférer de fichiers de sons traités ou à traiter (voix, musique...) pour des applications audio, ou de fichiers d'images traités ou à traiter (dessins, photos, séquences animés...) pour des applications vidéo...

Pour réaliser ces mouvements de données, il existe différentes techniques, plus ou moins faciles à implanter mais aussi plus ou moins efficaces en temps.

L'idée de base est toujours de diminuer le temps pris par ces transferts de données (qui est du temps perdu par le DSP sur le temps nécessaire au traitement numérique de ces données proprement dit).

Nous allons en présenter trois, de la plus simple (mais aussi la moins efficace en temps) à la plus compliquée (mais aussi la plus performante), ce qui nous permettra d'aboutir au principe de la **DMA**.

❶ par **scrutation** (ou **pooling**),

❷ par **interruption**,

❸ par **DMA**.

dans le cas d'un transfert d'une zone de mémoire sur la liaison série **SPORT0** en **transmission**.

Ce transfert, en pratique, peut permettre d'interfacer un **Convertisseur Numérique/Analogique** sur la liaison série **SPORT0** (c'est ce qui sera fait en TP) et correspond à l'envoi de données numériques ayant été traitées vers ce **CNA** en sortie.

Le principe général est d'écrire les données à transmettre les unes derrière les autres dans le **registre de transmission TX0** de la liaison **SPORT0**, ce registre se charge ensuite de les **sérialiser** pour les envoyer sur la liaison série **bit à bit** au rythme de la **vitesse choisie** (le registre de transmission est donc un registre à décalage à **entrée parallèle** et à **sortie série** comme le montre la figure IV.5. ci-dessous).

Mais attention, comme la vitesse de la liaison série est très faible (de l'ordre de la dizaine de μ s par élément à transmettre) par rapport au temps de chargement du registre de transmission (de l'ordre de la dizaine de ns, soit un facteur 1000), il faut toujours attendre que l'élément en cours de transfert soit entièrement transmis avant de placer l'élément suivant dans le registre de transmission, car sinon on écraserait les éléments les uns sur les autres et le message émis serait entièrement corrompu.

Il est important d'avoir bien compris cela (et qui est résumé sur la figure IV.5. suivante) avant de regarder les trois exemples suivants...

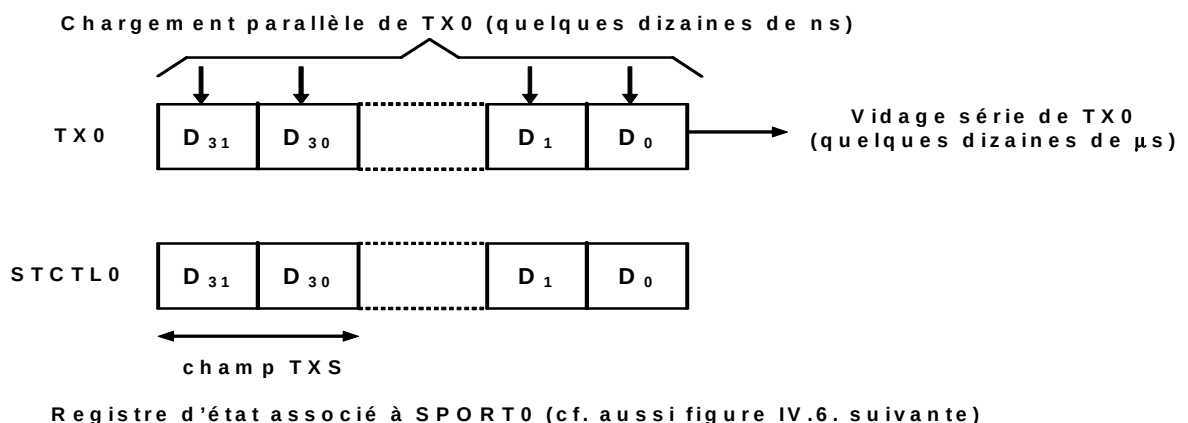
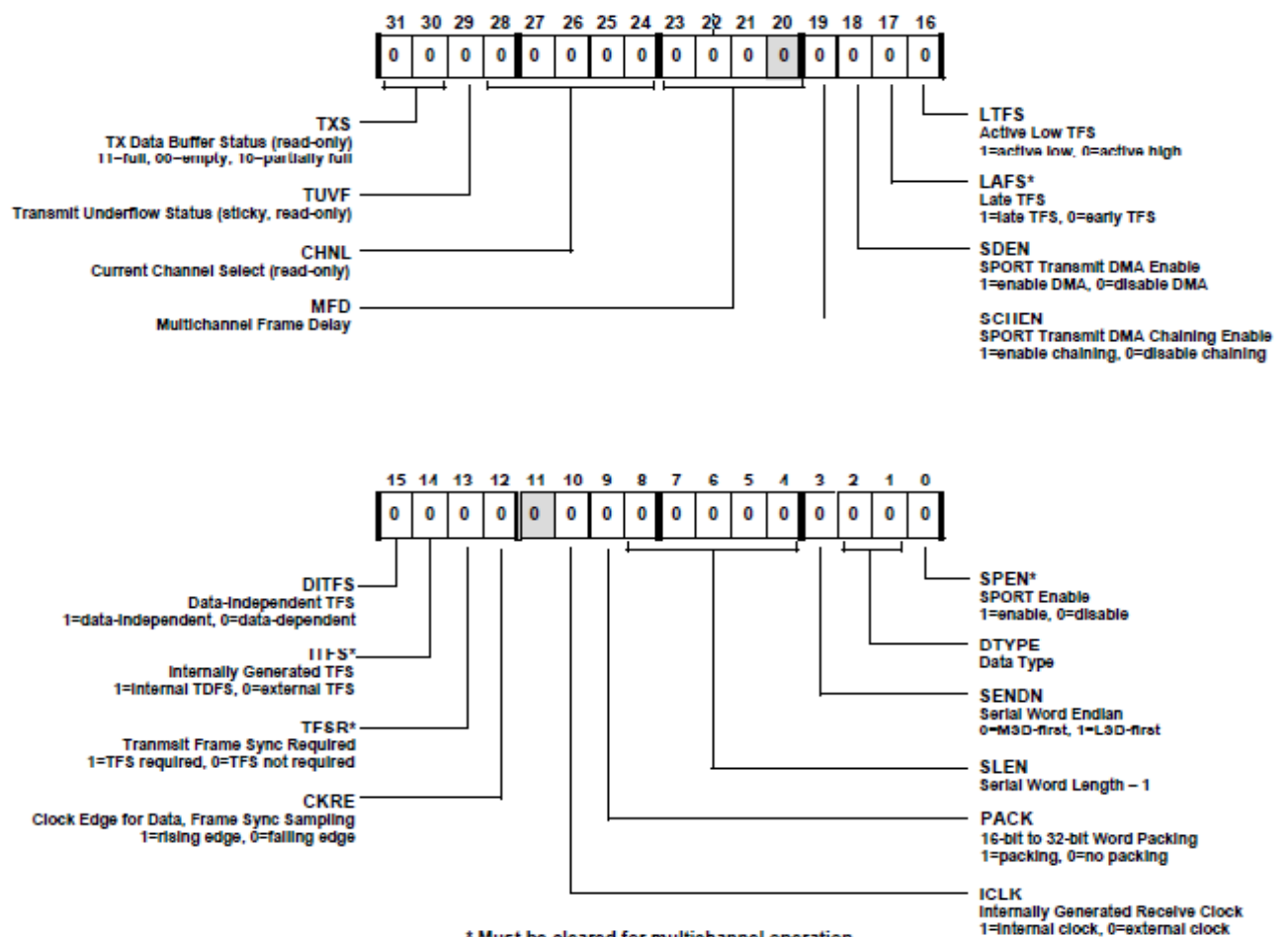


Figure IV.5.

Comment savoir si ce registre de transmission est **vide** (donc prêt à recevoir une nouvelle donnée à transmettre) ou **plein** (donc encore en train d'émettre la donnée précédente) ?

Pour cela, il faut regarder les **bits 31 et 30** (champ **TXS**) du registre **STCTL0** (registre d'état associé à la liaison **SPORT0** situé dans la zone des **IOP REGISTERS**) comme indiqué sur la figure IV.6. décrivant la signification de tous les bits de ce registre d'état et de contrôle de la liaison **SPORT0**.



STCTL0, STCTL1 Transmit Control Registers

Figure IV.6.

Si champ TXS de STCTL0 = 11 : registre de transmission **plein**,
Si champ TXS de STCTL0 = 00 : registre de transmission **vide**.

Remarque : on présentera ici 3 méthodes pour mettre en œuvre une transmission de données vers l'extérieur mais notons que les principes pour établir une réception de données via la liaison série **SPORT0** seraient comparables...

IV.4.3.1. Transfert par « pooling » ou scrutation.

C'est la technique la plus simple à comprendre, à programmer et à mettre au point. Mais c'est aussi celle qui est la moins performante car c'est elle qui consomme le plus de temps pour le transfert et donc qui laisse le moins de temps pour des éventuels traitements numériques. En gros, tout le temps est pris par la transmission car le DSP ne peut rien faire en parallèle : il passe tout son temps à attendre (on dit aussi à scruter d'où le nom de cette technique : la scrutation) que le transfert complet soit terminé. Le principe est le suivant :

Quand le registre de transmission est plein, le Processor Core se met en attente qu'il soit vide avant de le remplir avec le prochain élément à transmettre :
la conséquence est qu'il ne peut rien faire en parallèle, d'où les limites de cette méthode...

Exemple de structure de programme pour une implantation de transfert de données sur SPORT0 par scrutation (pooling)

```

/* fichier de définition des registres internes et de certains de leurs champs de bits */
#include <def21060.h>
/* les registres et champs de bits des SPORT0 et 1 */
#include <sport.h>
/* taille du fichier à transmettre : ici on a pris 4 mais le fichier pourrait être plus long... */
#define      N      4

/* le tableau ou buffer des données (entiers sur 32 bits) à transmettre */
/* ce tableau source sera rangé en mémoire de données par dm */
long      dm      source[N]={ 0x11111111, 0x22222222, 0x33333333, 0x44444444};
int      k = 0;

/* Le programme de transfert suivi d'un éventuel traitement */
void main ( void )
{
    TDIV0 = 0x00270007 ;    /* initialisation du registre TDIV0 de SPORT0 */
                           /* permet de fixer le débit de la liaison série */
                           /* détail de la synchronisation des trames */
                           /* se reporter à son contenu exact pour plus d'infos */

    STCTL0 = 0x000064F1 ;  /* initialisation du registre STCTL0 de SPORT0 */
                           /* STCTL0 contient des informations de contrôle */
                           /* comme le format des données, la validation de */
                           /* SPORT0, l'utilisation ou pas de la DMA ... */
                           /* se reporter à son contenu exact pour plus d'infos */

    /* début du transfert */
    for ( k=0 ; k<N ; k++ )
    {
        TX0 = source[k];    /*écriture de l'élément courant du buffer dans le */
                           /* registre de transmission TX0 */

        /* utilisation du masque TXS (bits 31 et 30) qui vaut en binaire %11000....000 = 0xC000 0000
        /* tant que le registre de transmission est plein, on attend */
        while ( STCTL0 & TXS );    /* tant que TX0 est plein, on attend */
                                   /* quand TX0 est vide, ce qui signifie que l'élément */
                                   /* courant a été totalement transmis, alors on peut */
                                   /* envoyer l'élément suivant du buffer */

    }

    /* à partir de là toutes les données ont été transmises sur SPORT0 */
    /* appel de la fonction de traitement du signal */

    while ( 1 )                / boucle sans fin de traitement */
        traitement() ;         /* on ne peut faire du traitement que quand toutes */
                                /* les données ont été transmises */

}

```

IV.4.3.2. Transfert par interruption.

Le principe ici est totalement différent.

Au lieu d'attendre que le registre de transmission soit vide avant d'envoyer l'élément suivant, c'est la liaison **SPORT0** qui va générer une interruption de transmission, signalant que le registre de transmission est vide (et donc que la donnée précédente a été entièrement transmise).

Cette interruption va entraîner l'exécution d'une routine d'interruption venant charger dans **TX0** la donnée suivante à transmettre.

Par rapport à la méthode précédente, le Processor Core peut effectuer autre chose en parallèle, puisqu'il n'est dérangé qu'entre chaque donnée à transmettre et qu'il ne passe pas son temps à attendre que la donnée courante soit transmise, comme dans le cas précédent.

La structure du programme est un peu la même, mais se rajoute un bloc supplémentaire.

Ce bloc n'existait pas dans la technique par scrutation et correspond à la routine de traitement de l'interruption.

En général ce type de programme sous interruption est plus difficile à concevoir et à écrire que celui correspondant à la technique précédente de pooling, car il faut penser :

- à initialiser le vecteur d'interruption (utilisation de la fonction **interrupt** du C),
- à valider le fonctionnement sous l'interruption correspondante pour qu'elle soit générée par la liaison série **SPORT0**,
- à autoriser la prise en compte des interruptions par le DSP (démasquage des interruptions),
- à écrire la routine de traitement de l'interruption (c'est quand même le minimum !!!),
- à acquitter, dans cette routine de traitement, chaque interruption pour éviter des appels intempestifs liés à la même cause (ici l'écriture dans le registre **TX0** permet de faire automatiquement cet acquittement)....

ce qui fait en pratique beaucoup de choses importantes à ne pas oublier !!!

Ce type de programme sera aussi toujours plus délicat à mettre au point.

**Exemple de structure de programme pour une implantation
de transfert de données par interruption sur SPORT0 en transmission.**

```

/* fichier de définition des registres internes et de certains de leurs champs de bits */
#include <def21060.h>
/* les registres et champs de bits des SPORT0 et 1 */
#include <sport.h>
/* taille du fichier à transmettre : ici on a pris 4 mais le fichier pourrait être plus long... */
#define      N      4

/* le tableau ou buffer des données (entiers sur 32 bits) à transmettre */
/* ce tableau source sera rangé en mémoire de données par dm */
long      dm      source[N]={ 0x11111111, 0x22222222, 0x33333333, 0x44444444};
int      k = 0;

/* Le programme de transfert avec un éventuel traitement en parallèle */
void main ( void )
{
    TDIV0 = 0x00270007 ;    /* idem cas précédent par scrutation*/

    STCTL0 = 0x000064F1 ;    /* idem cas précédent par scrutation */

    interrupt ( SIG_SPT0I,it_transmit );    /* installation de l'interruption en émission */
                                           /* it_transmit est le nom de la routine */
                                           /* d'interruption qui sera exécutée chaque fois */
                                           /* que le registre TX0 sera vide */

    SPT0I = SPT0I | IMASK;    /* mise à 1 du bit 12 IMASK du registre SPT0I de validation */
                             /* des interruptions sur registre de transmission vide */
    IRPTEN=IRPTEN | MODE1; /* démasquage de l'interruption */

    /* début du transfert*/

    TX0 = source[k];          /*écriture du premier élément du buffer dans le */
                             /* registre de transmission TX0 */

    /* appel de la fonction de traitement du signal */

    while ( 1 )                / boucle sans fin de traitement */
        traitement() ;        /* on peut faire du traitement pendant (en même temps) */
                             /* que les données sont transmises */
}

/* routine d'interruption appelée chaque fois que le registre de transmission TX0 est vide*/
/* c'est-à-dire entre l'envoi de chaque élément du buffer source */
void it_transmit ( int sig_num )
{
    if ( k==N-1 )              /* dernier élément : buffer entièrement transmis */
    {
        SPT0I=SPT0I & ~IMASK;    /* mise à 0 du bit 12 IMASK du registre SPT0I */
                                /* pour invalider les interruptions sur TX0 vide*/
        IRPTEN=IRPTEN & ~MODE1; /* masquage de l'interruption */
    }
    k++;                      /* passage à l'élément suivant */
    TX0 = source[k];          /* et l'émettre */
}

```

IV.4.3.3. Transfert sous DMA.

Avec la technique précédente, on a vu que le DSP pouvait effectuer des traitements pendant le transfert puisqu'il n'est dérangé par une interruption qu'entre chaque élément du buffer à transférer.

Dans notre exemple, il y aura 4 interruptions, donc c'est acceptable, mais imaginons le cas de gros buffers source à transférer (plusieurs milliers d'éléments à transférer, voire plus), le temps perdu par ces milliers d'interruptions peut commencer à devenir important.

L'idée, pour améliorer encore cette technique, consiste à dire :

l'idéal serait qu'il n'y ait qu'une seule interruption à la fin du transfert, quelque soit la taille du buffer à transférer !!!

Cela revient, pour le DSP, à complètement « sous-traiter » la tâche du transfert.

Et c'est là qu'intervient le contrôleur de **DMA (DMA CONTROLLER)** qui apparaissait sur le schéma de la figure IV.1.

En fait, c'est un bloc spécialisé capable de prendre en compte de « A à Z » et en totale autonomie une opération de transfert qu'on appelle alors une **DMA (Direct Memory Access)**.

Bien sûr, cette prise en charge du transfert nécessitera avant tout une phase d'initialisation (on dit aussi une phase de paramétrage) pour fournir tous les éléments nécessaires pour que ce transfert ait lieu en toute autonomie et sous le seul contrôle du **DMA CONTROLLER**, sans « déranger » le DSP (sauf une seule fois, pour signaler la fin du transfert).

Ainsi une **DMA** s'avère être une méthode très rapide et quasiment transparente pour le programmeur pour transférer des blocs de données entre la mémoire interne et l'extérieur : mémoire externe, liaisons série, périphériques externes, processeur hôte ...

Il faut maintenant se demander de quels types d'informations le contrôleur de DMA a besoin pour prendre en charge entièrement un tel transfert.

Il y en a au moins 5 que l'on va détailler maintenant :

① **Choix d'un canal de DMA** : en fait c'est le type de transfert qui va être réalisé, en quelque sorte le chemin par où les données seront transférées.

Sur le **DSP 21061**, il existe 6 canaux de DMA indépendants.

Les affectations pour les canaux sont les suivantes :

Canal 0	Liaison série SPORT0 en réception
Canal 1	Liaison série SPORT1 en réception
Canal 2	Liaison série SPORT0 en transmission
Canal 3	Liaison série SPORT1 en transmission
Canal 4	DMA externe : mémoire externe, processeur hôte, multiprocessing
Canal 5	DMA externe : mémoire externe, processeur hôte, multiprocessing

Dans l'exemple que nous traitons, on utilisera le **canal 2** (transmission sur SPORT0).

② **Une adresse de début (de base) du buffer source** :

C'est-à-dire où se trouve en mémoire le buffer à transférer.

Cette adresse sera mise dans le registre **IIi** (Internal Index **i**, pour $i=0$ à 5) du contrôleur de DMA.

③ **Une valeur d'incrément ou de décrétement** :

Cette valeur permettra de calculer à partir de l'adresse précédente du début du buffer, l'adresse de l'élément suivant à transférer sous DMA. Elle correspond en fait à la taille des éléments du bloc à transférer.

Cette valeur d'incrément ou de décrétement sera mise dans le registre **IMi** (Internal Modifier **i**, pour $i=0$ à 5) du contrôleur de DMA.

④ **La taille du bloc à transférer** :

Cette taille sera mise dans le registre **Ci** (Count **i**, pour $i=0$ à 5) du contrôleur de DMA. C'est en fait un compteur qui se décrémente automatiquement de un entre chaque transfert et qui indiquera la fin du transfert DMA quand il atteindra une valeur nulle.

⑤ **Éventuellement la direction du transfert** pour les échanges vers ou venant de l'extérieur du DSP. (implicite pour les canaux 0 à 3, mais nécessaire pour les canaux 4 et 5).

Au final la donnée de ces 5 éléments permettra de paramétrer correctement la DMA.

Cette méthode de transfert sera en pratique très largement utilisée car très rapide et très efficace.

Exemple de structure de programme pour une implantation de transfert de données sur SPORT0 sous DMA

```

/* fichier de définition des registres internes et de certains de leurs champs de bits */
#include <def21060.h>
/* les registres et champs de bits des SPORT0 et 1 */
#include <sport.h>
/* taille du fichier à transmettre : ici on a pris 4 mais le fichier pourrait être plus long... */
#define      N      4

/* le tableau ou buffer des données (entiers sur 32 bits) à transmettre */
/* ce tableau source sera rangé en mémoire de données par dm */
long      dm      source[N]={ 0x11111111, 0x22222222, 0x33333333, 0x44444444};
int      k = 0;

/* Le programme de transfert avec un éventuel traitement en parallèle */
void main ( void )
{
    TDIV0 = 0x00270007 ;    /* idem cas précédent par scrutation*/

    STCTL0 = 0x000465F1 ;    /* ici on autorise la DMA sur transmission, la routine de */
                            /* traitement devenant alors un traitement sous DMA */
                            /* se reporter à son contenu exact pour plus d'infos */

    interrupt ( SIG_SPT0I,it_dma );    /* installation de l'interruption en émission */
                                    /* it_dma est le nom de la routine d'interruption*/
                                    /* cette interruption sera exécutée une seule*/
                                    /* fois, à la fin du transfert du buffer */

    SPT0I = SPT0I | IMASK;    /* mise à 1 du bit 12 IMASK du registre SPT0I de validation */
                            /* des interruptions sur registre de transmission vide */

    IRPTEN=IRPTEN | MODE1; /* démasquage de l'interruption */
/* début du paramétrage de la DMA canal 2 */
    II2 = source;    /* initialisation de II2 avec l'adresse du buffer source */
    IM2 = 1;    /* valeur de l'incrément dans IM2 : 1 */
    C2 = N;    /* initialisation de C2 avec la longueur du buffer source */
/* fin du paramétrage de la DMA canal 2 */

/* à partir de là, le transfert se fait automatiquement sous la responsabilité totale du contrôleur */
/* de DMA : le DSP ne sera dérangé par l'interruption it_dma qu'une fois le transfert terminé */
/* appel de la fonction de traitement du signal */

    while ( 1 )    /* boucle sans fin de traitement */
        traitement() ;    /* on peut faire du traitement pendant que les données*/
                            /* sont transmises */
}

/* routine d'interruption appelée lors le buffer source a été entièrement transmis */
void it_dma ( int sig_num )
{
    .    /* on peut ici par exemple initialiser une autre DMA : il faut pour cela : */
    .    /* - effacer le bit d'autorisation de la DMA dans STCTL0 */
    .    /* - paramétrer une nouvelle DMA ( registres II2, IM2 et C2 */
    .    /* - remettre à 1 le bit d'autorisation de la DMA dans STCTL0 */
}

```

On le voit, cette technique sous DMA représente un moyen efficace, souple et performant pour gérer les liaisons SPORT du DSP.

Mais le raffinement et l'efficacité peuvent encore être poussés plus loin.

En effet, le procédé de gestion de buffer sous DMA décrit ci-dessus nécessite tout de même une interruption entre chaque transfert de buffer. On peut imaginer des applications où un buffer représente par exemple une image et où l'on souhaite transférer des séquences vidéo (suite d'images).

Pour éviter d'avoir une interruption entre chaque image, il existe un moyen encore plus rapide : **le chaînage de DMA**.

Cette dernière technique permet au contrôleur de DMA de « s'auto-initialiser » automatiquement entre chaque transfert DMA, grâce à une chaîne de structures définies en mémoire que l'on appelle une chaîne de **TCB** (ou **T**ransfert **C**ontrol **B**lock), comme indiqué sur la figure IV.7.

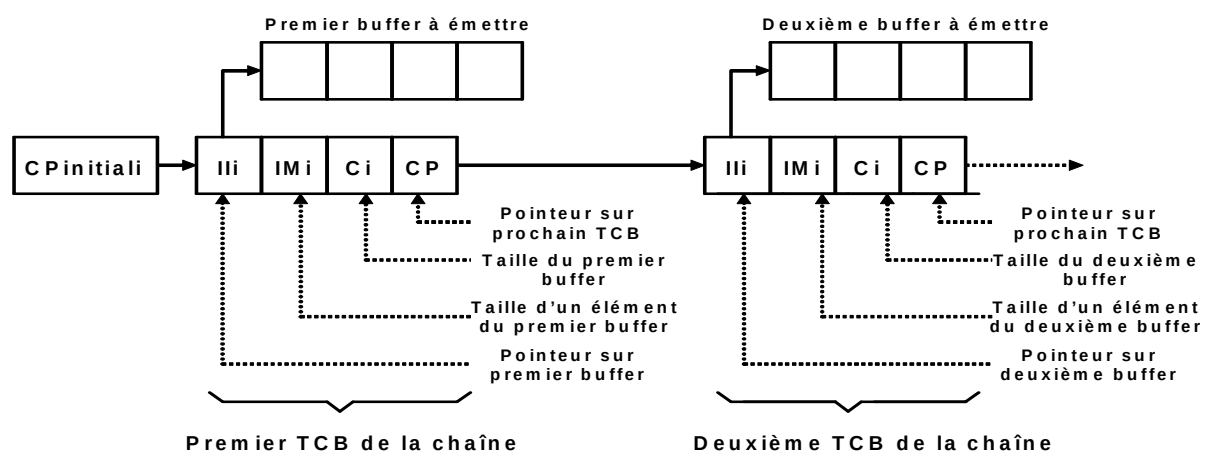


Figure IV.7.

Chaque **TCB** de cette chaîne est en fait une image des paramètres d'initialisation des différents transferts DMA auxquels on a rajouté un **CP** (Chain Pointer) qui « pointe » sur le **TCB** suivant dans la chaîne. Grâce à cette chaîne, les différents transferts DMA vont s'enchaîner, les uns à la suite des autres. Inutile pour le DSP de paramétrer une nouvelle DMA à la suite de la précédente : le contrôleur de DMA, grâce à cette chaîne de **TCB** a toutes les informations pour initialiser la DMA suivante.

Comme on le voit également sur la figure IV.7., un registre spécial pour chaque canal *i* de DMA (en plus des registres **Ili**, **IMi** et **Ci** déjà présentés) appelé le registre **CPinitiali** pointe sur le premier **TCB** de la chaîne pour initialiser le processus.

En résumé : en autorisant le chaînage de DMA et en chargeant le **CPinitiali**, le contrôleur de DMA vient automatiquement paramétrer la première DMA en chargeant les registres du canal *i* avec le contenu du premier TCB, et le transfert peut commencer.

Lorsque ce transfert est fini (quand **Ci**=0), le canal s'auto-initialise en pointant sur le TCB suivant ..., jusqu'au dernier TCB de la chaîne (que l'on peut tester par la dernière valeur du registre **CP** par exemple).

On évite ainsi les interruptions entre chaque transfert DMA, ce qui était bien l'objectif recherché.

Qu'en est-il de la méthode qui sera utilisée pendant les manipulations des séances de TP ?

Nous ferons de l'acquisition de signaux temps réel (grâce à un CAN), échantillons que nous traiterons au fur et à mesure de leur prélèvement avant de les restituer sur le CNA en sortie.

Il est clair que dans ce cas (que ce soit en réception ou en émission sur SPORT0) les échantillons une fois qu'ils ont été traités et restitués n'ont pas à être conservés dans un buffer différent propre à chaque échantillon.

Ce qui veut dire que ce sera le même buffer qui pourra être utilisé à chaque transfert DMA.

Cette situation permet de faire **du chaînage (de transfert sous DMA) « auto-référencé »**, c'est-à-dire que chaque nouveau transfert se fera avec les mêmes paramètres d'initialisation que le transfert précédent, conformément au schéma de la figure IV.8.

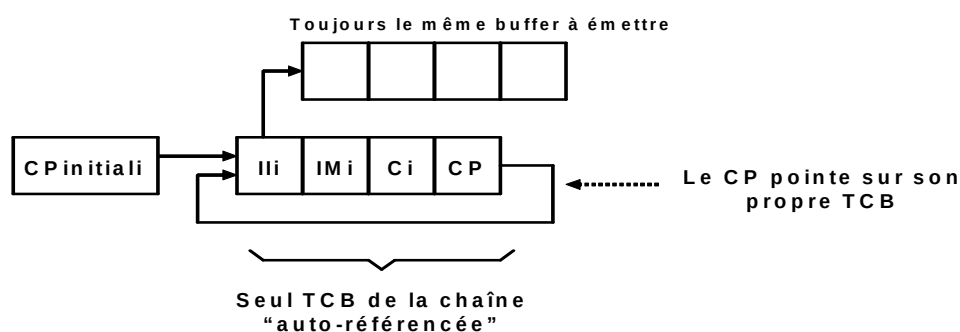


Figure IV.8.

Un exemple de soft écrit en C utilisant cette technique de DMA sera fourni plus loin dans ce polycopié et permet de gérer deux buffers de données :

- un premier en **réception** (**rx_buf**) pour les échantillons fournis par un **CAN**,
(et déclaré comme un tableau de 3 entiers) :

```
int rx_buf[3] ;
```

Ce buffer contiendra la valeur numérique $x(n)$ (à chaque instant d'échantillonnage nT_e) de l'entrée analogique $x(t)$.

- et un deuxième en **transmission** (**tx_buf**) pour les données traitées numériquement et qui seront envoyées sous DMA vers un **CNA**.
(et déclaré également comme un tableau de 3 entiers initialisés avec):

```
int tx_buf[3] = {0xCC40,0,0} ;
```

Ce buffer contiendra la valeur numérique $y(n)$ (à chaque instant d'échantillonnage nT_e) de la sortie analogique $y(t)$.

N'hésitez pas à jeter un œil sur ce programme pour vous en inspirer...

Quelques extraits de ce programme sont d'ailleurs fournis ci-dessous.

On pourra vérifier la cohérence de ces éléments avec les principes théoriques que l'on vient de présenter.

Voici par exemple ci-dessous la partie permettant de définir une structure de type **TCB** :

```
/* définition d'une structure de TCB pour le chaînage des DMA de la liaison DSP-Codec */
/* un TCB complet comprend en fait 9 éléments et non 4 comme dans la présentation */
/* simplifiée vue précédemment */
```

```
typedef struct
{
    unsigned ei; /* non utilisé ici : multiprocessing */
    unsigned em; /* non utilisé ici : multiprocessing */
    unsigned ec; /* non utilisé ici : multiprocessing */
    unsigned gp1; /* registre d'usage général n°1 */
    unsigned gp2; /* registre d'usage général n°2 */
    unsigned** cp; /* pointe sur le TCB suivant dans la chaîne */
                /* pointeur sur un pointeur !!! */
    unsigned c; /* taille du bloc à transférer à chaque DMA */
    int im; /* incrément pour calculer l'adresse de l'élément suivant */
            /* dans le transfert DMA en cours */
    unsigned* ii; /* pointeur sur le buffer : adresse de base du buffer */
} tcb;
```

Déclaration de deux TCB dont les noms sont **rx_tcb** (associé au buffer **rx_buf**) et **tx_tcb** (qui va gérer le buffer **tx_buf**) :

```
tcb rx_tcb,tx_tcb ;
```

Initialisation des éléments de ces deux TCB :

```
rx_tcb.c = 3; /* taille du buffer rx_buf (bloc à transférer du CAN) */
rx_tcb.im = 1; /* pour pointer sur l'élément suivant de rx_buf */
rx_tcb.ii = rx_buf; /* pointe sur le buffer de réception */
rx_tcb.cp = &rx_tcb.ii; /* pointe sur lui-même donc paramétrage identique pour */
                        /* toutes les DMA qui vont s'enchaîner (autoréférentiation) */
```

```
tx_tcb.c = 3; /* taille du buffer tx_buf (bloc à transférer vers le CNA) */
tx_tcb.im = 1; /* pour pointer sur l'élément suivant de tx_buf */
tx_tcb.ii = tx_buf; /* pointe sur le buffer de transmission */
tx_tcb.cp = &tx_tcb.ii; /* pointe sur lui-même donc paramétrage identique pour */
                        /* toutes les DMA qui vont s'enchaîner (autoréférentiation) */
```


Il est parfois difficile de s'y retrouver dans la « jungle » des différents procédés pour réaliser le passage du monde analogique vers le monde numérique, et vice-versa :

- convertisseurs simple ou double rampe (pas très précis),
- convertisseurs à approximations successives (assez lents),
- convertisseurs « flash » (très rapides mais aussi les plus chers, plutôt réservés aux applications vidéo où on n'a pas le choix)...

La technique de conversion dite $\Sigma\Delta$ apparaît ici comme un bon compromis entre les contraintes de **précision**, de **rapidité** mais aussi de **coût**. Son avantage est de n'utiliser que des circuits issus du domaine numérique.

Cette méthode de conversion dérive en fait du procédé de modulation portant le même nom.

Sans prétendre être exhaustif, voici le principe de fonctionnement d'un tel CAN.

Il est basé sur la remarque suivante : dans le cas du son (ou de l'image également), dans la très grande majorité des cas, chaque échantillon prélevé à l'instant t_n ressemble à celui de l'instant t_{n-1} . En d'autre terme, il n'y a pas de différence importante entre les valeurs de deux échantillons **successifs**. L'idée sera donc de ne coder que la **différence** entre ces deux échantillons consécutifs. Cela permet d'éviter les redondances (partie commune des deux échantillons) et de ne prendre en compte que les informations pertinentes (la différence entre 2 échantillons). Voilà pour la partie Δ du nom de ce convertisseur.

Comment s'y prend-t-on ? Regardons le schéma de principe d'un tel convertisseur sur la figure V.2. suivante pour en faire une description **qualitative**.

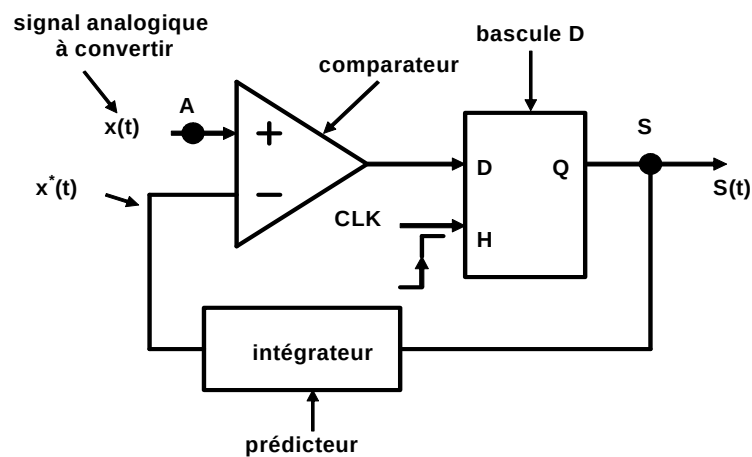


Figure V.2.

On va comparer la valeur du signal analogique $x(t)$ à l'instant t_n avec une image $x^*(t)$ de la valeur de ce signal à l'instant t_{n-1} (on verra un peu plus loin comment est déterminé cette valeur $x^*(t_{n-1})$).

Ce comparateur est prévu pour fournir :

- soit un niveau **haut** (1 logique) si $x(t_n) \geq x^*(t_{n-1})$,
- soit un niveau **bas** (0 logique) si $x(t_n) < x^*(t_{n-1})$.

En sortie de ce comparateur, on aura donc un signal qui vaut soit **1**, soit **0**.

Au rythme d'une horloge **CLK** (en réalité sur les fronts montants ou descendants de cette horloge), ces valeurs de sortie du comparateur sont mémorisées sur la sortie **Q** d'une **bascule D**. On aura donc au point **S** un signal **S(t)** formé d'une séquence de 1 et de 0, **synchrone** avec le signal **CLK**, codant la différence entre deux échantillons successifs : un **1** va coder une **augmentation** du signal $x(t)$ et un **0** codera une **diminution** de ce signal.

Cette horloge **CLK** sert aussi à fixer les différents instants t_n .

Au final, quand on aura sur **S(t)** une séquence de 1 **successifs**, cela voudra dire que le signal $x(t)$ a tendance à augmenter. Et réciproquement, une séquence de 0 **consécutifs** traduira une diminution de $x(t)$. Par contre, un signal carré traduira un intervalle de temps où, en moyenne, le signal $x(t)$ varie très peu.

Ces remarques nous permettent de dire qu'en intégrant cette séquence de 1 et de 0, on obtiendra l'image recherchée $x^*(t_{n-1})$ de l'échantillon précédent, prédiction de la valeur du signal analogique $x^*(t_{n-1})$. D'où la présence de l'intégrateur sur la figure V.2. : cet **intégrateur** est aussi parfois appelé un **prédicteur**, car c'est lui qui nous permet de « prédire » la valeur $x^*(t_{n-1})$.

La figure V.3. ci-dessous fournit « l'allure » des différents signaux concernés, tout en montrant bien que lorsque le signal $x(t)$ augmente, le signal $S(t)$ passe plus de temps à 1 qu'à 0 (ce qui prouve comme on l'a dit une succession de bits à 1). Et réciproquement, lors des diminutions de $x(t)$, $S(t)$ passe plus de temps à 0.

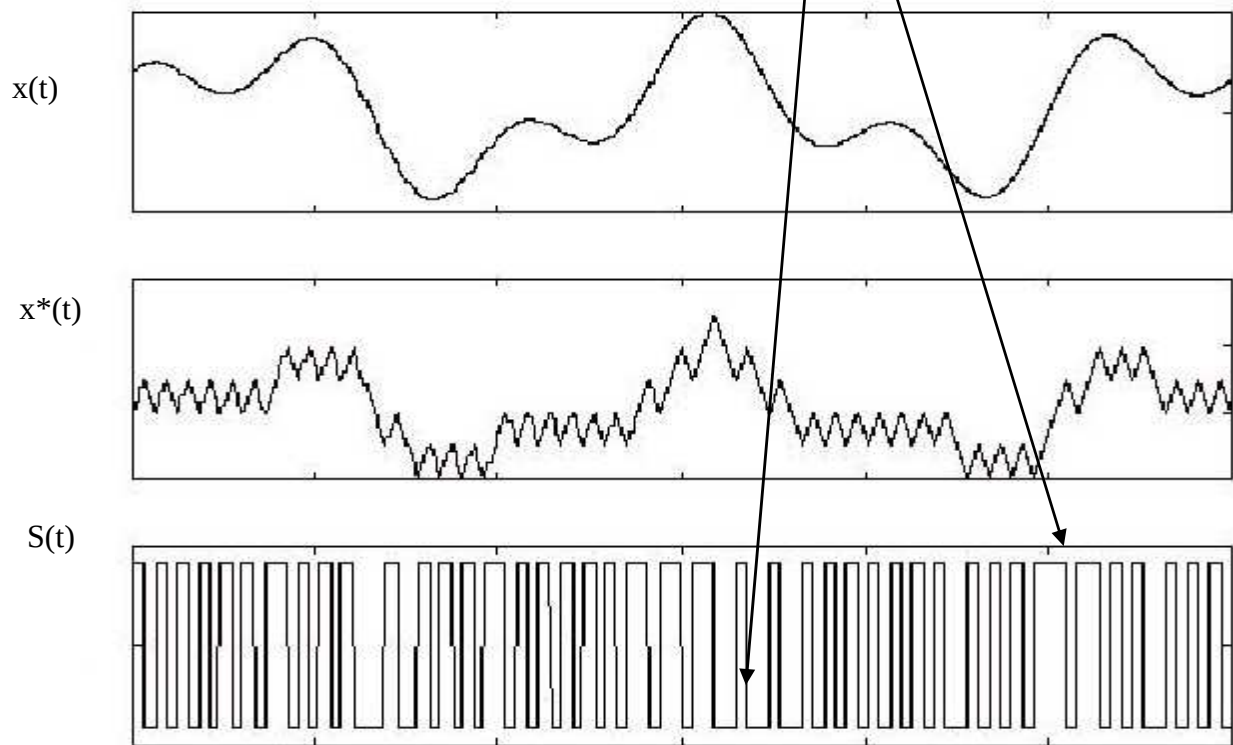


Figure V.3.

Un tel convertisseur fonctionne finalement sur 1 bit permettant de coder soit l'augmentation, soit la diminution du signal analogique $x(t)$.

Il permet de travailler à des fréquences (fréquence du signal **CLK**) très élevées (d'après la documentation technique de l'**AD 1847**, la fréquence de **CLK** vaut 64 fois la fréquence d'échantillonnage programmée), ce qui simplifie le filtre **anti-aliasing** analogique (ou filtre **d'anti-repliement**) nécessaire avant la conversion (un simple RC analogique du premier ordre est ici suffisant), un **filtre passe-bas numérique** interne au convertisseur venant s'ajouter pour limiter la bande de fréquences en entrée à 0,4 fois la fréquence d'échantillonnage programmée et supprimer ainsi tout repliement spectral dans la bande de base du signal analogique.

Tous ces éléments seront repris et détaillés pendant les séances de travaux pratiques.

On pourrait cependant montrer qu'un tel système présente des inconvénients si la tension analogique d'entrée $x(t)$ varie trop vite : la séquence de 1 et de 0 produite en sortie du comparateur n'arrive plus à suivre cette forte variation de $x(t)$ (on a un phénomène un peu analogue au slew-rate d'un A.O.P.).

Pour lisser les variations de la tension analogique et éviter ainsi les trop fortes variations de $x(t)$, on commence donc par l'intégrer (d'où le Σ avant le Δ dans le nom de ce convertisseur) en rajoutant un intégrateur au point **A** du schéma de la figure V.2. Il faudra bien sûr rajouter un dérivateur avant de reconstruire le signal.

Nous ne détaillerons pas ici le principe du CNA $\Sigma\Delta$

V.2. Format des données gérées par le Codec.

Il est maintenant intéressant de donner quelques éléments sur les formats de données gérés par ce Codec. Ils sont au nombre de trois.

V.2.1. 16 bits signé et linéaire.

Ce format est largement utilisé en audio.

La courbe de transfert entre la valeur analogique (en ordonnées) et la valeur numérique (en abscisses) est fournie sur la figure V.4. : elle est bien linéaire ; bien qu'en fait elle soit constituée d'un très grand nombre ($2^{16} = 65536$) de marches d'escaliers comme le montre le zoom sur cette figure V.4.

La différence $P = M - m$ est appelée **la plage de conversion (Full Scale Input Voltage)**.

⚡ ⚡ : les variations du signal analogique doivent rester entre ces 2 limites sous peine de saturer le convertisseur. On a d'ailleurs ici $m = -M$ (valeurs symétriques).

L'extrait suivant de la documentation technique de l'**AD1847** permet de conclure que typiquement :

$$M \cong +1,4 \text{ V } (+\sqrt{2} \text{ V}) \text{ et } m \cong -1,4 \text{ V } (-\sqrt{2} \text{ V}).$$

ANALOG INPUT	Min	Typ	Max	Units
Full-Scale Input Voltage (RMS Values Assume Sine Wave Input) Line1, Line2, AUX1, AUX2		1		V rms
	2.54	2.8	3.10	V p-p

La quantité $q = \frac{M-m}{2^{16}}$ est appelée **le pas de quantification**. Elle donne une information sur la précision de la conversion (c'est la hauteur de chaque marche d'escalier (cf. zoom sur la figure V.4.)).

Le nombre $20.\log(2^{16}) = 320.\log(2) = 96 \text{ dB}$ est **la dynamique** du codage, image de la **précision**.

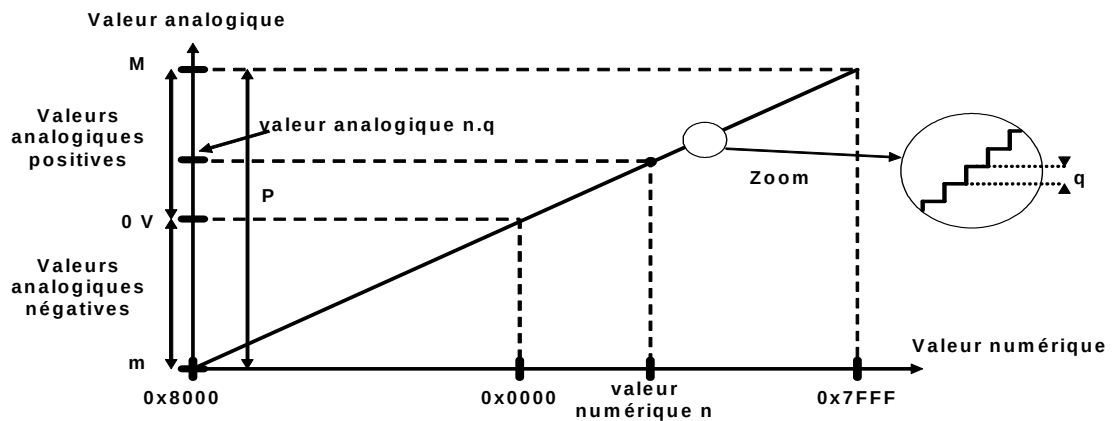


Figure V.4.

On voit bien que les valeurs numériques sont sur 16 bits et que :

- 0x8000** : correspond à la valeur analogique minimum (**m**),
- 0x0000** : correspond à la valeur analogique nulle (**0V**),
- 0x7FFF** : correspond à la valeur analogique maximum (**M**).

Le codage est bien signé (c'est-à-dire en complément à 2) :

- les valeurs **analogiques positives** donneront une valeur numérique avec le **MSB** à **0**, (entre 0x0000 et 0x7FFF),
- les valeurs **analogiques strictement négatives** donneront une valeur numérique avec le **MSB** à **1**, (entre 0x8000 et 0xFFFF),

V.2.2. 8 bits non signé et linéaire.

La courbe de transfert de ce format est fournie sur la figure V.5. : elle est également linéaire avec la même réserve sur sa forme en marche d'escalier de $2^8 = 256$ marches ici.

Les valeurs extrêmes **M** et **m** de la tension analogique sont bien sûr les mêmes que dans le cas précédent.

Le pas de quantification $q = \frac{M-m}{2^8}$ est ici 256 fois plus grand que précédemment : la précision en sera directement affectée (chaque valeur numérique code un intervalle de valeurs analogiques plus grand).

La dynamique du codage est ici 2 fois plus faible ($20 \cdot \log(2^8) = 160 \cdot \log(2) = 48$ dB).

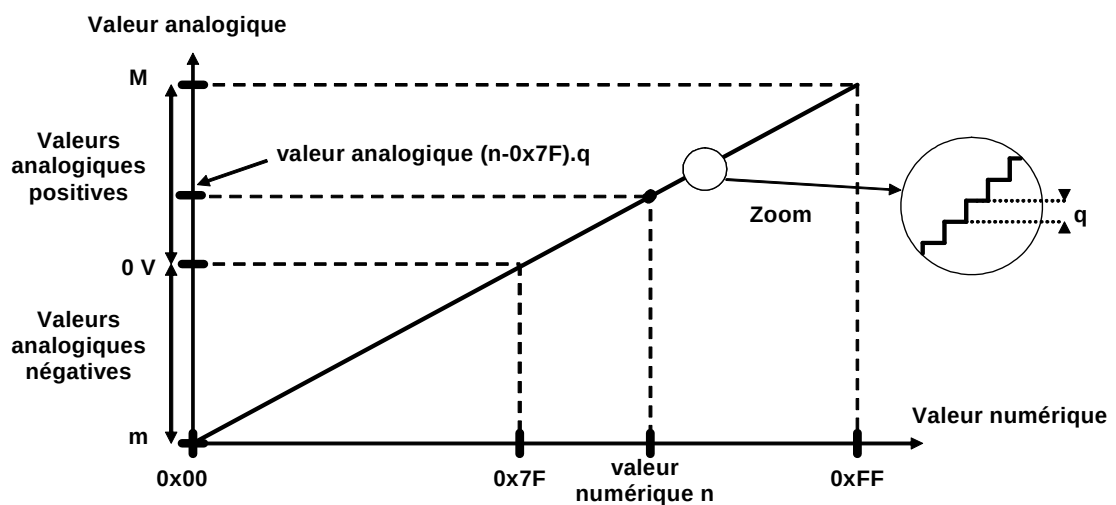


Figure V.5.

On voit bien que les valeurs numériques sont maintenant codées sur 8 bits et que :

- 0x00** : correspond à la valeur analogique minimum (**m**),
- 0x7F** : correspond à la valeur analogique nulle (**0V**),
- 0xFF** : correspond à la valeur analogique maximum (**M**).

Le codage est bien non signé (au sens classique du terme) puisque :

- les valeurs **analogiques strictement positives** donneront une valeur numérique avec le **MSB à 1**, (entre 0x80 et 0xFF),
- les valeurs **analogiques négatives** donneront une valeur numérique avec le **MSB à 0**, (entre 0x00 et 0x7F).

V.2.3. 8 bits comprimés.

Ici, la courbe de transfert entre le domaine analogique et le domaine numérique n'a plus globalement la forme d'une droite, comme on peut le voir sur la figure V.6. ci-dessous.

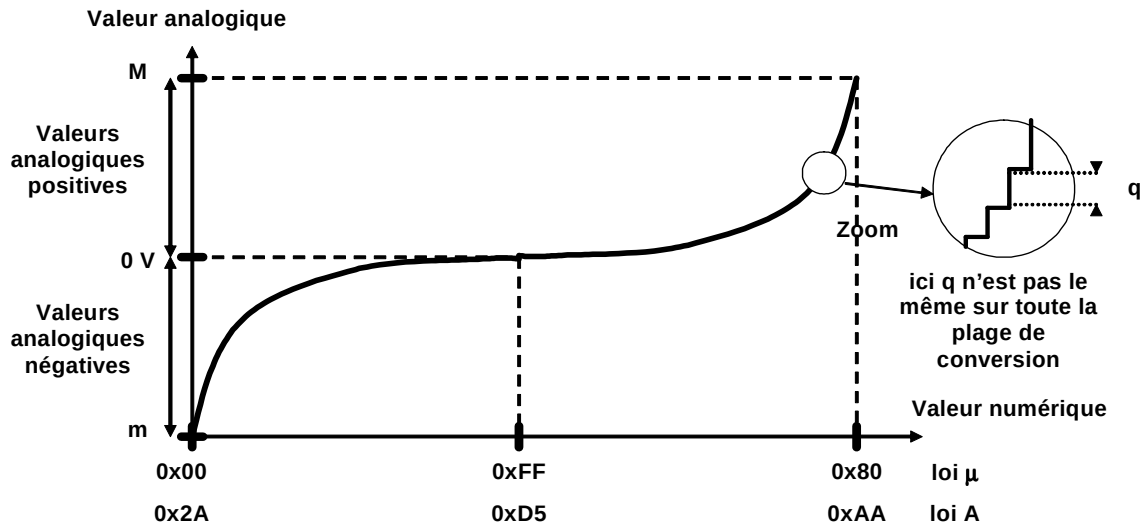


Figure V.6.

Cette courbe correspond à la loi appelée **A** (normes Européennes) ou **μ** (normes Américaines et Japonaises).

D'où vient la forme un peu « bizarre » de cette courbe ?

Dans un codage linéaire, toutes les « marches » de l'escalier ont la même hauteur, qu'on soit au milieu de l'escalier (valeurs analogiques proches du 0V et donc plutôt faibles en valeur absolue) ou à ses extrémités (valeurs analogiques éloignées du 0V et donc de valeurs absolues plutôt importantes).

Ce qui revient à dire que le pas de quantification q est le même sur toute la plage de conversion du convertisseur.

Mais on peut raisonnablement se dire qu'il serait plus intéressant d'avoir une précision plus grande pour les faibles valeurs analogiques que pour les grandes (en effet, faire une erreur de 1 mV sur une valeur analogique de 1 V est moins grave que faire la même erreur de 1 mV sur une grandeur analogique de 1 mV !!!).

Cela revient à assigner plus de valeurs numériques pour les faibles valeurs analogiques (pour avoir un pas de quantification q plus petit pour ces faibles valeurs analogiques) et en conséquence un pas de quantification q plus grand pour les valeurs analogiques de forte valeur absolue.

Et donc d'avoir un escalier plutôt « plat » aux alentours du 0V et de plus en plus « raide » quand on s'éloigne de part et d'autre du 0V. C'est bien ce qui se passe avec la forme de la courbe de la figure V.6.

Au final, on obtient avec 8 bits comprimés une dynamique globale intermédiaire entre le format 8 bits linéaire (48 dB) et le format 16 bits linéaire (96 dB), puisque, des calculs montrent que :

- la loi A donne une **dynamique équivalente** de **72 dB**,
- la loi μ donne une **dynamique équivalente** de **64 dB**.

Cette technique sera surtout utilisée en télécommunications audio car elle permet d'avoir une dynamique tout à fait acceptable uniquement en codant le son sur 8 bits (et non sur 16 bits), ce qui permet de gagner un facteur 2 dans les transmissions, toute chose étant égale par ailleurs (au final, c'est un bon compromis entre la précision et la rapidité de transmission).

A la base, le Codec travaille en 16 bits linéaire, et ce n'est qu'après, en sortie du CAN, ou avant, en entrée du CNA (cf. blocs **μ A LAW ⑦** sur la figure V.1.) qu'on peut utiliser ou pas (via un registre interne du codec) ce format de 8 bits comprimés pour diminuer le nombre de bits échangés entre le Codec et le DSP, tout en gardant, comme on vient de le voir une précision acceptable.

Comment tout cela fonctionne t'il ?

- dans le sens **CAN** vers **DSP** : il y a une modification des valeurs numériques, conformément au schéma de la figure V.7. donnée pour la loi A.

Dans un premier temps, il y a une **truncation** de la valeur numérique de 16 bits pour ne garder que 14 bits (loi A) (ou 13 bits pour la loi μ) en supprimant les 2 (ou 3) bits de poids faibles. Et ensuite vient une **compression** en utilisant une table de conversion pour respecter la forme de la courbe de la figure V.6. et passer sur 8 bits. C'est pendant cette phase de compression que sera adaptée la valeur du pas de quantification q en fonction de la valeur absolue du signal, comme on l'a dit précédemment.

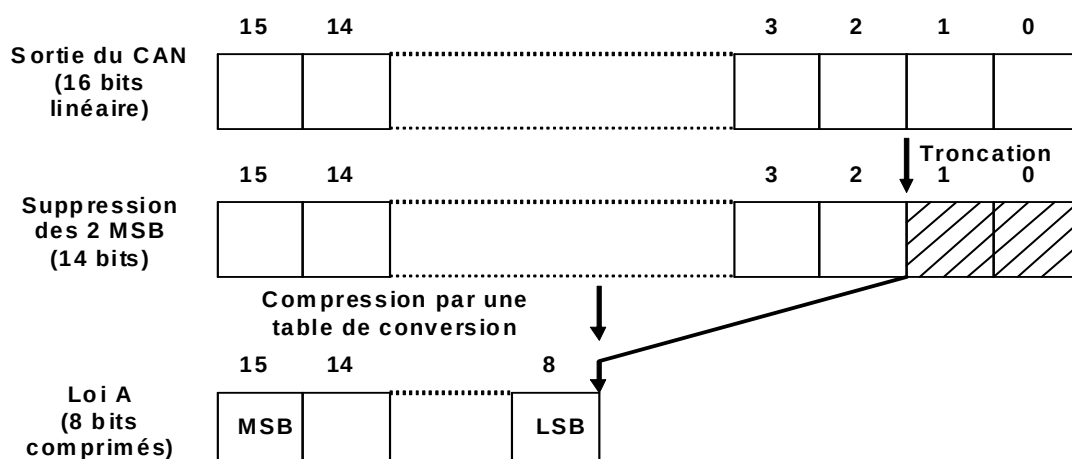


Figure V.7.

- dans le sens **DSP** vers **CNA** : c'est un peu le mécanisme inverse qui se produit conformément au schéma de la figure V.8. donnée toujours pour la loi A. Dans un premier temps, il y a une **expansion** de la valeur numérique reçue de 8 bits vers 14 bits (loi A) (ou vers 13 bits pour la loi μ) en utilisant une table de conversion inverse de la précédente. Puis s'enchaîne une opération de **remplissage par des 0** des 2 (ou 3) bits de poids faibles pour obtenir une valeur numérique sur 16 bits qui sera utilisée comme entrée du **CNA**.

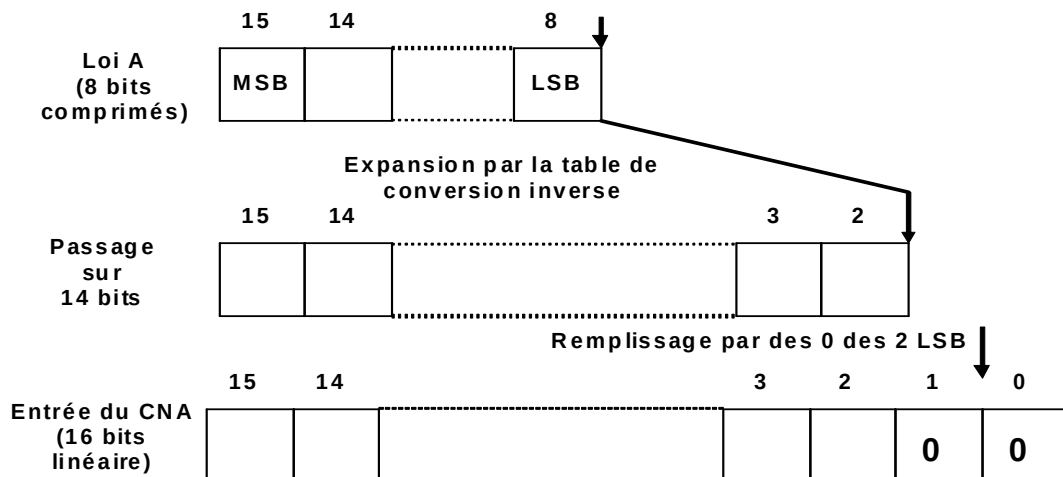


Figure V.8.

Notons que les liaisons séries **SPORT0** et **SPORT1** côté DSP peuvent aussi gérer automatiquement cette compression / expansion, ce qui évite d'avoir à écrire la partie de code qui l'effectuerait sur le DSP si ce n'était pas le cas.

V.3. Protocole de communication entre le DSP et le Codec.

Il est temps maintenant de coupler le DSP et le Codec via la liaison série synchrone SPORT0.

La communication des valeurs numériques des échantillons entre le Codec et le DSP utilise le protocole **TDM (Time Division Multiplexed)** implanté sur la liaison série **SPORT0**.

Des éléments sur ce protocole sont indispensables pour comprendre (et éventuellement modifier ou concevoir) les parties de softs qui viennent soit écrire (initialisation du Codec, lancement d'une conversion...), soit lire (état du Codec, acquisition d'une donnée) dans le Codec.

A première vue, ce protocole semble compliqué et on peut se demander ce qui a motivé ses concepteurs.

En fait, l'idée de base est de pouvoir gérer des liaisons multipoints, permettant d'avoir plusieurs émetteurs/récepteurs de données sur la même liaison.

Pour cela, des tranches de temps (des **Slots**) sont allouées à chacun des éléments pouvant dialoguer sur la liaison (d'où le nom Time Division Multiplexed : multiplexage temporel par division du temps en slots).

V.3.1. Bilan des échanges.

Essayons de faire un bilan, en les numérotant, des informations à échanger entre le DSP et le Codec.

Si on prend, pour simplifier, le cas **d'un seul** DSP et **d'un seul** Codec :

- du DSP vers le Codec :

① initialisation des registres internes du Codec, pour son fonctionnement (Control Word)

- choix des différents gains,
- cheminement interne par mixage des différents signaux,
- choix de la fréquence d'échantillonnage F_e ,
- mise en route de l'auto-calibrage ...

① envoi d'une valeur numérique vers le CNA de la voie gauche (Left Line Output),

② envoi d'une valeur numérique vers le CNA de la voie droite (Right Line Output).

- du Codec vers le DSP :

③ **envoi de l'état interne du Codec** pour sa surveillance :

- informations sur des saturations éventuelles sur les voies d'entrées (**Status Word**),
- contenu des registres internes de configuration (**Index Readback**) ...

④ **envoi de la sortie numérique du CAN de la voie gauche :**

- venant de **Line 1 Left Input**,
 - ou de **Line 2 Left Input**,
 - ou de **Aux 1 Left Input**,
 - ou même de **Left Line Output** par rebouclage interne,
- tout ceci suivant la configuration effectuée pendant la phase ⑩.

⑤ **envoi de la sortie numérique du CAN de la voie droite :**

- venant de **Line 1 Right Input**,
 - ou de **Line 2 Right Input**,
 - ou de **Aux 1 Right Input**,
 - ou même de **Right Line Output** par rebouclage interne,
- tout ceci suivant la configuration effectuée pendant la phase ⑩.

V.3.2. Solutions hardware.

Pour réaliser cet échange, deux solutions hardware sont possibles, que nous allons détailler.

Mais avant, précisons que, tant au niveau du Codec (cf. ⑧ de la figure V.1.) que du DSP, il existe 2 broches notées :

- **SDO** (comme **S**erial **D**ata **O**utput) : broche **de sortie** par laquelle les valeurs numériques sortiront bit à bit du composant. Côté Codec, c'est la sortie du CAN et côté DSP c'est la sortie correspondant par exemple à $y(n)$ (sortie du filtre numérique),
- **SDI** (comme **S**erial **D**ata **I**nput) : broche **d'entrée** par laquelle les valeurs numériques rentreront bit à bit dans le composant. Côté Codec, c'est l'entrée du CNA et côté DSP c'est l'entrée correspondant par exemple à $x(n)$ (entrée du filtre numérique).

V.3.2.1. Liaison à un seul fil.

On réalise la liaison comme indiqué sur la figure V.9. ci-dessous. La sortie série **SDO** (côté DSP et Codec) et l'entrée série **SDI** (côté DSP et Codec) sont reliées entre elles.

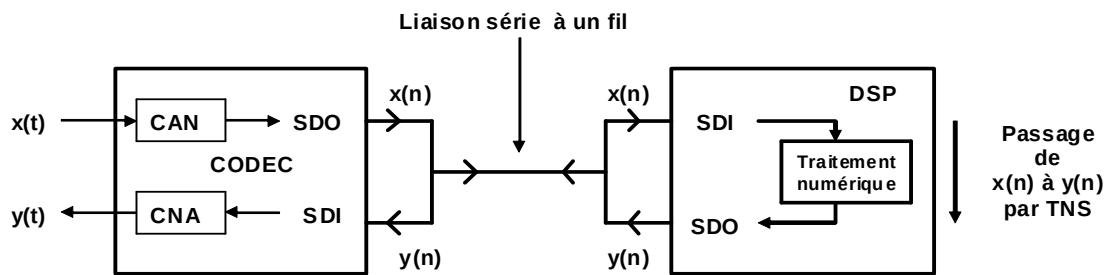


Figure V.9.

L'intérêt est de n'avoir qu'un seul fil de dialogue **bidirectionnel** entre le DSP et le Codec.

Par contre, on ne peut faire du **vrai full duplex** sur un seul fil (essayez de faire circuler 2 TGV en sens inverse sur une seule voie : si vous y arrivez sans dégât, vous serez embauché à la SNCF car vous diviserez par deux le coût de construction de la ligne. Mais plus probablement, ce n'est même pas la peine de leur envoyer votre CV !!!). On ne peut faire que du **half duplex** (ou semi-duplex).

En effet si le Codec et le DSP envoient en même temps les valeurs numériques $x(n)$ et $y(n)$, il y aura **collision** entre ces valeurs.

On peut quand même réaliser du **full duplex** mais à **l'alternat** en utilisant le **multiplexage temporel** : 3 tranches de temps consécutives (**SLOT0**, **SLOT1** et **SLOT2**) et dans cet ordre sont allouées pour les blocs ①, ② et ③ précédentes suivies immédiatement de 3 autres tranches de temps consécutives (**SLOT3**, **SLOT4** et **SLOT5**) et dans cet ordre pour les blocs ④, ⑤ et ⑥. (cf. figure V.10.).

Ce qui assure bien les échanges **dans les deux sens** et **sans collision**. Mais avec une perte de temps, car il faut consommer 6 tranches de temps au total pour un échange complet dans les deux sens.

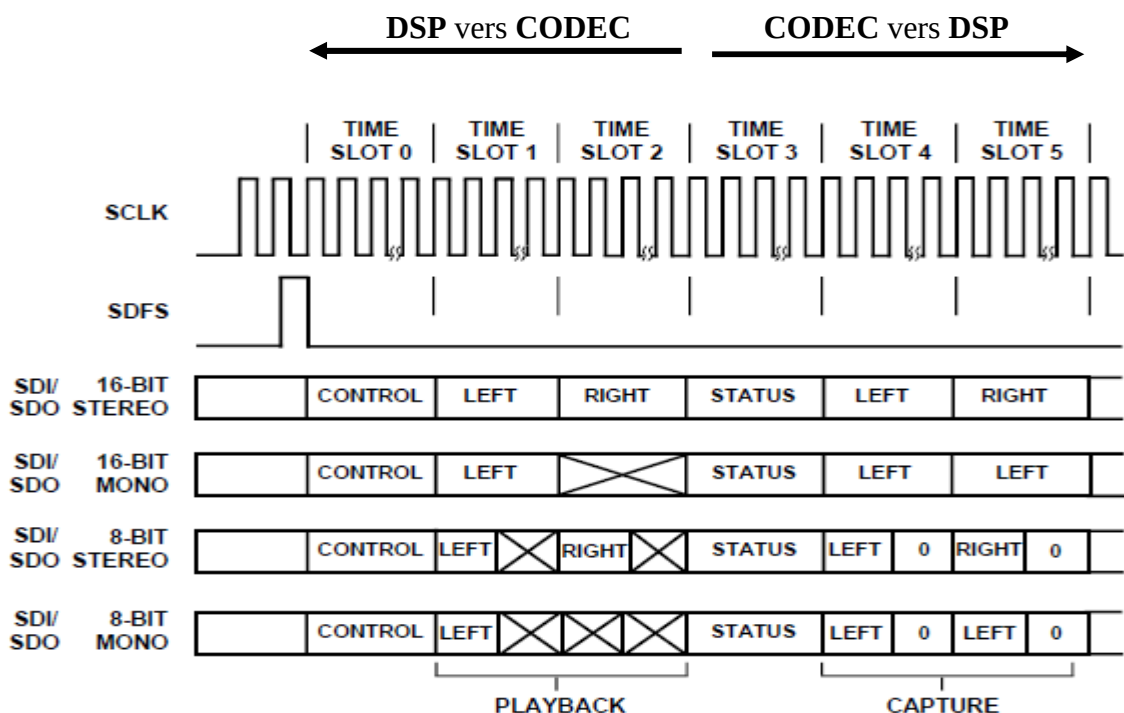


Figure V.10.

V.3.2.2. Liaison à deux fils.

C'est cette solution si est choisie sur la carte de développement utilisée en TP.

Il y a maintenant deux fils **monodirectionnels** et **différents** pour **SDI** et **SDO**, donc plus aucun risque de collisions (la voie TGV a été doublée, ce qui est plus rassurant pour l'utilisateur), comme indiqué sur la figure V.11.

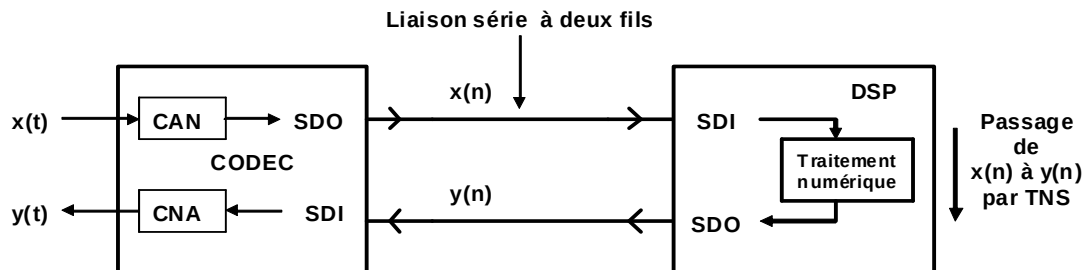


Figure V.11.

On peut donc envisager d'affecter 3 tranches de temps **identiques** et **synchrones** (**SLOT0**, **SLOT1** et **SLOT2**) pour les blocs ①, ② et ③ et pour les blocs ④, ⑤ et ⑥ (c'est-à-dire bloc ① en même temps que bloc ③ etc...).

D'où un gain en temps d'un facteur 2 par rapport au cas précédent, puisque seulement 3 tranches de temps sont consommées pour un échange complet dans les deux sens, résumé sur la figure V.12.

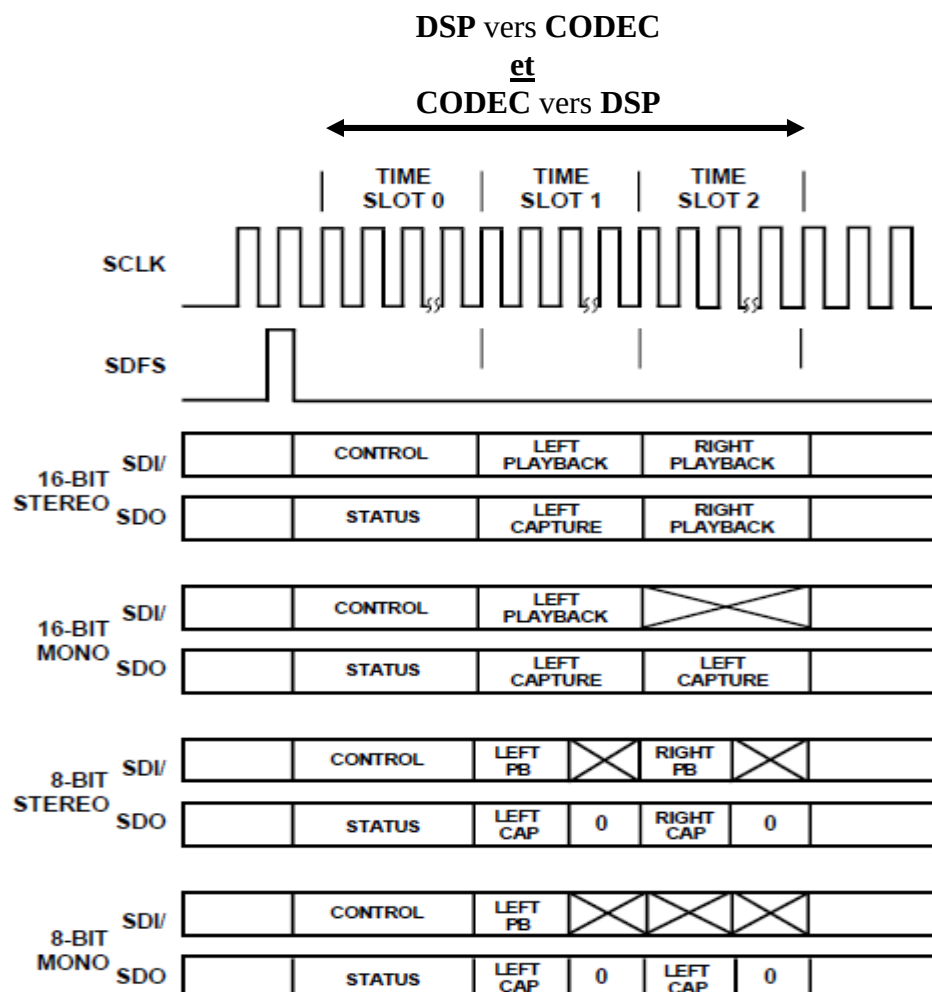


Figure V.12.

C'est le bit **TSSEL** du registre d'index 12 du Codec qui permet de signaler à ce dernier l'une ou l'autre de ces configurations hardware.

Côté DSP, les liaisons SPORT0 et SPORT1 peuvent également être configurées pour gérer ce type de protocole.

Un exemple de soft en C configurant le Codec et la liaison SPORT0 du DSP pour gérer ce protocole TDM sera donné plus loin dans ce polycopié. Jetez-y également un coup d'œil.

Avant d'en terminer avec le Codec, notons qu'il possède :

- 7 registres internes de 16 bits et,
- 13 registres internes de 8 bits,

tous ces registres permettent sa configuration, son fonctionnement et son contrôle par logiciel.

Pour la signification exacte du rôle et du contenu de chacun de ces registres internes, se reporter à la documentation complète du Codec AD1847 fournie sur les PC des salles de TP. On aura d'ailleurs l'occasion en TP d'en modifier quelques uns.

VI. LE SYSTEME D'EVALUATION EZ-KIT UTILISE DANS LE TP.

Nous allons maintenant donner quelques éléments sur la carte d'évaluation que vous utiliserez pendant les séances de TP.

De façon très générale, la **méthodologie de travail**, très standard, sera la même que celle utilisée lors des développements faits sur microcontrôleurs :

- carte d'évaluation à l'extérieur du PC,
- écriture, compilation, linkage des programmes sur le PC,
- téléchargement des exécutables sur la carte d'évaluation,
- test, mise au point et validation de ces programmes en fonction du cahier des charges.

VI.1. Architecture matérielle.

Le synoptique de la figure VI.1. représente la carte d'évaluation.

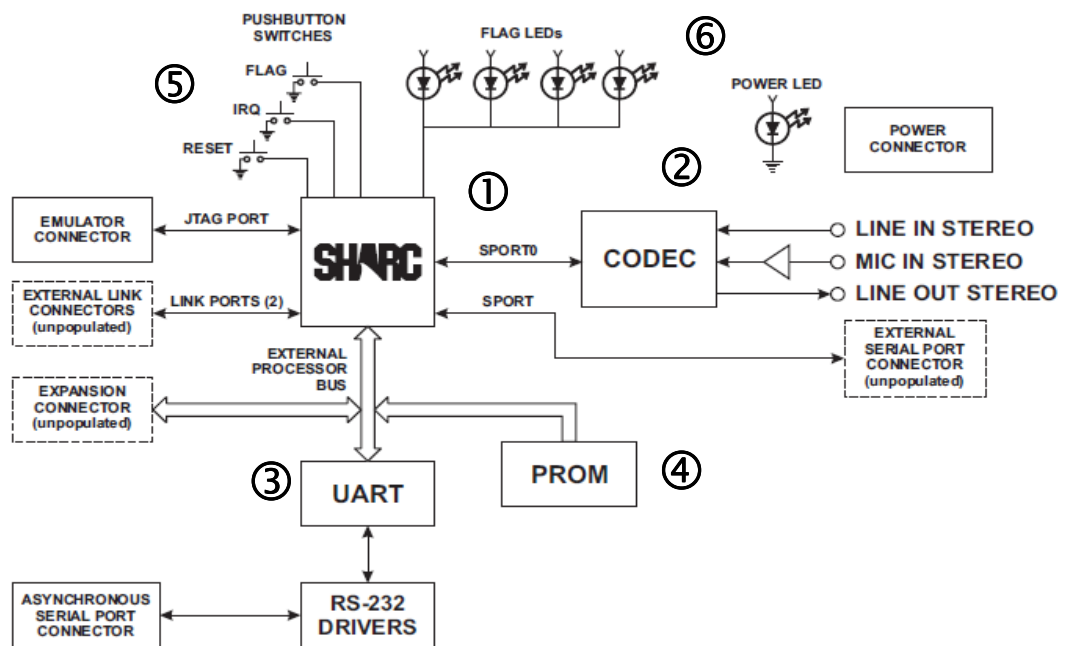


Figure VI.1.

Les principaux éléments vous sont maintenant familiers :

- **le SHARC (1)** : cœur du système : l'ADSP 21061 à 40 Mhz,
 - **le CODEC (2)** : l'AD 1847 avec ses entrées et sorties analogiques et une liaison 2 fils via SPORT0 du DSP,
 - **l'UART (3)** : ici un PC16550 avec l'interface RS232 pour la liaison avec le PC hôte de développement. C'est en fait le même UART que celui qui équipe la majorité des PC sur leurs COM (descendant du maintenant déjà ancien 8250). Cet UART, fonctionnant sur l'IRQ2 du DSP, est situé par défaut à l'adresse de base \$0040 2000.
 - **une PROM (4)** : pour l'initialisation du KIT à la mise sous tension. Cette PROM, sur support, peut être avantageusement remplacée par une autre PROM contenant un soft applicatif réalisé et débuggé par le développeur sur le KIT: il existe un utilitaire (le BOOT LOADER) permettant de convertir ce soft en un fichier compatible avec la plupart des claqueurs de PROM.
- Notez l'absence de mémoire externe (autre que cette PROM) sur le KIT, les 1 Mégabits internes au DSP 21061 devant suffire pour la plupart des applications qui seront demandées dans le TP.

- un ensemble de 3 **switches** ⑤ (**PUSHBUTTON SWITCHES** : boutons poussoirs) : reset, simulation d'IRQ1, Flag sur l'input Flag du DSP et de 5 **leds** ⑥ (**FLAG LEDs**) sur les outputs Flags du DSP (permettant par exemple de visualiser si le programme est bien passé à certains endroits, ce qui peut offrir une aide pendant la mise au point) ou sur l'alimentation.

L'implantation physique sur la carte de ces différents éléments est donnée en figure VI.2.

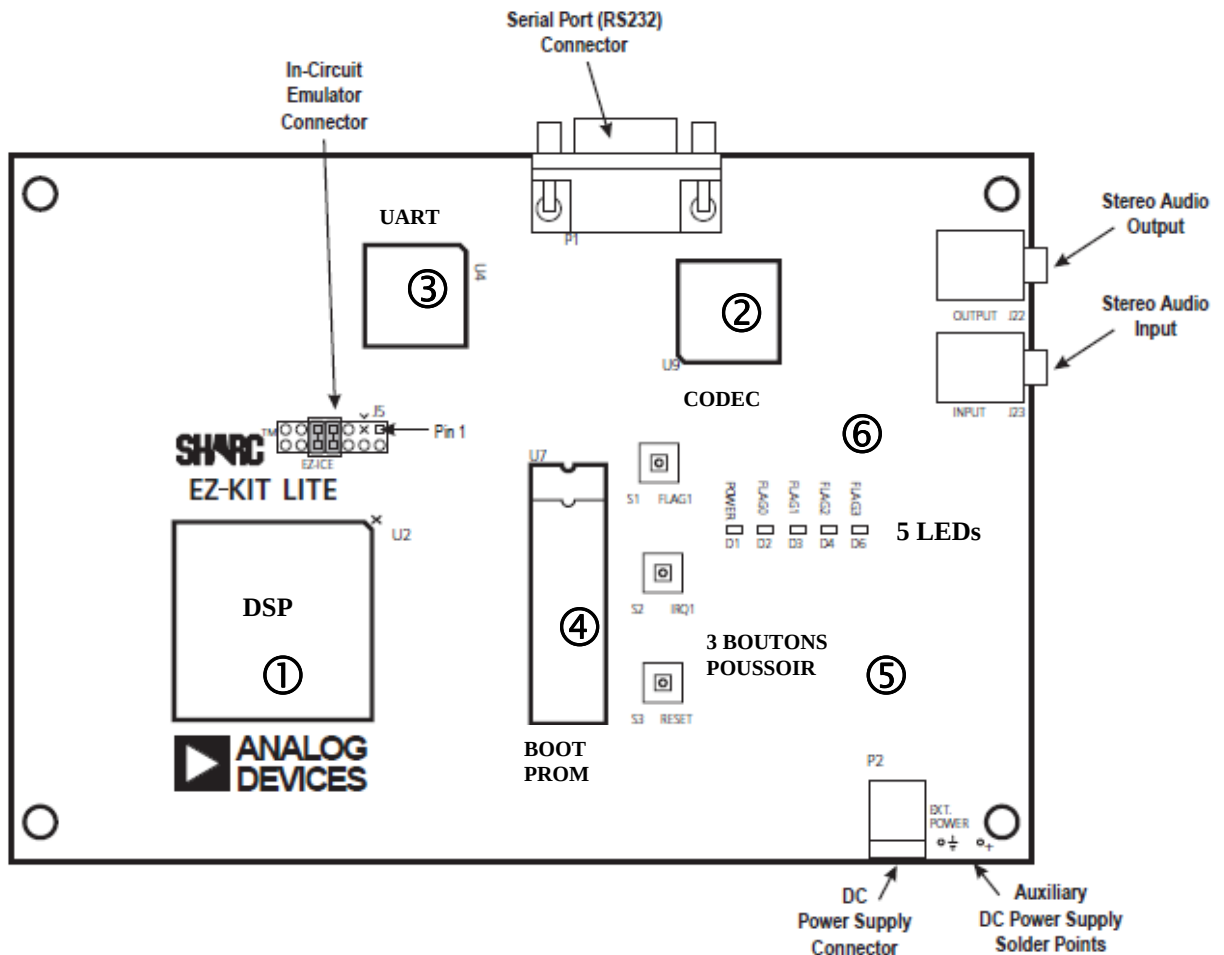


Figure VI.2.

VI.2. Les fichiers et les logiciels utilisés.

VI.2.1. Le fichier d'architecture matérielle EZKIT.ACH.

Il permet de fournir aux différents outils de développement (principalement au linker pour placer correctement le code et/ou les données en mémoire, mais aussi au simulateur et éventuellement à l'émulateur) des informations sur l'architecture matérielle du système :

- type de DSP dans la famille,
- emplacement et taille des différents segments de code et de données système et utilisateur...

Si on programme en C, le linker connaîtra grâce à ce fichier d'architecture les affectations de ces zones, ce qui évite les problèmes en permettant un linkage optimal des fichiers .OBJ. En assembleur, il faudrait être plus vigilant ...

Le listing de ce fichier **EZKIT.ACH** est fourni à titre indicatif sur la figure VI.3. ci-dessous.

```

|-----|
| .SYSTEM          SHARC_EZKIT_Lite;
|
| This architecture file is required for use with the SHARC EZ-KIT
| Lite development software. It is structured for use with the C
| compiler but also can be used with assembly code.
|
| This architecture file allocates:
|   Internal 133 words of 48-bit run-time header in memory block 0
|           16 words of 48-bit initialization code in memory block 0
|           619 words of 48-bit kernel code in memory block 0
|          7424 words of 48-bit C code space in memory block 0
|           4K words of 32-bit PM C data space in memory block 0
|
|           8K words of 32-bit C DM data space in memory block 1
|           4K words of 32-bit C heap space in memory block 1
|          3712 words of 32-bit C stack space in memory block 1
|           384 words of 32-bit kernel data in memory block 1
|
| .PROCESSOR = ADSP21061;
|
|-----|
| Internal memory Block 0
|-----|
| .SEGMENT/RAM/BEGIN=0x00020000 /END=0x00020084 /PM/WIDTH=48      seg_rth;
| .SEGMENT/RAM/BEGIN=0x00020085 /END=0x00020094 /PM/WIDTH=48      seg_init;
| .SEGMENT/RAM/BEGIN=0x00020095 /END=0x000202ff /PM/WIDTH=48      seg_knlc;
| .SEGMENT/RAM/BEGIN=0x00020300 /END=0x00021fff /PM/WIDTH=48      seg_pmco;
| .SEGMENT/RAM/BEGIN=0x00023000 /END=0x00023fff /PM/WIDTH=32      seg_pmda;
|
|-----|
| Internal memory Block 1
|-----|
| .SEGMENT/RAM/BEGIN=0x00024000 /END=0x00025fff /DM/WIDTH=32      seg_dmda;
| .SEGMENT/RAM/BEGIN=0x00026000 /END=0x00026fff /DM/WIDTH=32 /cheap seg_heap;
| .SEGMENT/RAM/BEGIN=0x00027000 /END=0x00027e7f /DM/WIDTH=32      seg_stak;
| .SEGMENT/RAM/BEGIN=0x00027e80 /END=0x00027fff /DM/WIDTH=32      seg_knld;
|
|-----|
| External Memory Select 1 is reserved for the UART.
|-----|
|
| .ENDSYS;

```

Figure VI.3.

On peut rappeler que le **Block 0** de la mémoire correspond à la mémoire de programme **PM** et que le **Block 1** correspond à la mémoire de donnée **DM**.

Au reset de la carte, les 256 premières instructions de l'EPROM située sur le KIT sont automatiquement chargées dans le DSP (le SHARC est en mode BOOT sur PROM) puis exécutées aussitôt. Ces instructions correspondent à un programme appelé le LOADER qui vient charger en RAM le reste de l'EPROM conformément à cette architecture matérielle.

Après cette phase, le découpage mémoire et les affectations sont données en figure VI.4.

Segment Name	Description	Memory Block	Length	Width	Start Address	End Address
seg_rth	Run-time header	0	133	48	0x00020000	0x00020084
seg_init	Initialization data	0	16	48	0x00020085	0x00020094
seg_knlc	Kernel Code	0	619	48	0x00020095	0x000202FF
seg_pmco	User Code	0	7424	48	0x00020300	0x00021FFF
seg_pmda	User PM Data	0	4096	32	0x00023000	0x00023FFF
seg_dmda	User DM Data	1	8192	32	0x00024000	0x00025FFF
seg_heap	User C Heap	1	4096	32	0x00026000	0x00026FFF
seg_stak	User C Stack	1	3712	32	0x00027000	0x00027E7F
seg_knld	Kernel Data	1	384	32	0x00027E80	0x00027FFF

Figure VI.4.

Ces affectations appellent les précisions suivantes :

- **le Run-time-header** : pour la gestion des vecteurs d'interruptions et du reset et pour le code d'initialisation du système implanté dans le segment nommé **seg_rth** (cf. fichier 060_hdr.obj ou 060_hdr.asm). Il est utilisé par exemple par les fonctions du C comme **interrupt**, **signal** ou **raise**,
- le segment nommé **seg_init** : contient les données et les paramètres d'initialisation du système,
- le segment nommé **seg_knlc** contient le code du noyau ou **KERNEL**. C'est un petit logiciel de communication fonctionnant sous l'IRQ2 de l'UART permettant d'échanger des paquets de données entre le PC et l'EZ-KIT sur le principe de question / réponse.

Les questions sont du type :

- *lecture de mémoire de code ou de données (Read PM48 ou Read DM32),*
- *écriture en mémoire de code ou de données (Write PM48 ou Write DM32),*
- *lecture de registres (Read Core),*
- *lancement de programme (Start Processor),*
- *reset du DSP (Reset Processor),*
- *configuration et test de la liaison avec le PC (Set Serial Port Speed, Verify Communications, Resynchronize Communications).*

Pour plus de précisions sur l'utilisation de ce noyau implanté sur le DSP, voir plus loin dans le chapitre sur la phase de téléchargement, de mise au point et de test sur le DSP.

- le segment nommé **seg_knld** : contient les données de ce noyau,
 - le segment nommé **seg_stak** : la pile du C pour le stockage de l'adresse de retour par la fonction appelante et le pointeur du cadre de pile. Cette pile dont chaque position contient 32 bits, se remplit vers les adresses décroissantes (PUSH) et se vide vers les adresses croissantes (POP),
 - le segment nommé **seg_heap** : contient le tas du C pour les allocations/désallocations dynamiques de mémoire (fonctions **malloc**, **calloc**, **realloc** ou **free** du C).
- Normalement, les zones allouées au KERNEL (Code, Data, Pile ou Tas du KERNEL) ne doivent pas être utilisées pour implanter vos programmes, ou vous risquez de perdre le lien avec l'EZ-KIT.
- les programmes de l'utilisateur doivent se trouver en User Code (**seg_pmco**) et les données correspondantes (globales ou statiques) en User PM Data (segment nommé **seg_pmda**) ou en User DM Data (segment nommé **seg_dmda**).

Mais rassurez-vous : quand vous programmez en C, toutes ces affectations sont gérées automatiquement par le linker et seront donc transparentes pour vous.

VI.2.2. Les différentes phases de développement d'un programme.

Le synoptique représenté sur la figure VI.5. rappelle et résume les différentes phases que l'on rencontre dans un projet de développement informatique utilisant un processeur entre le cahier des charges initial (**START**) et la remise du produit final au professeur ou au client (**END**).

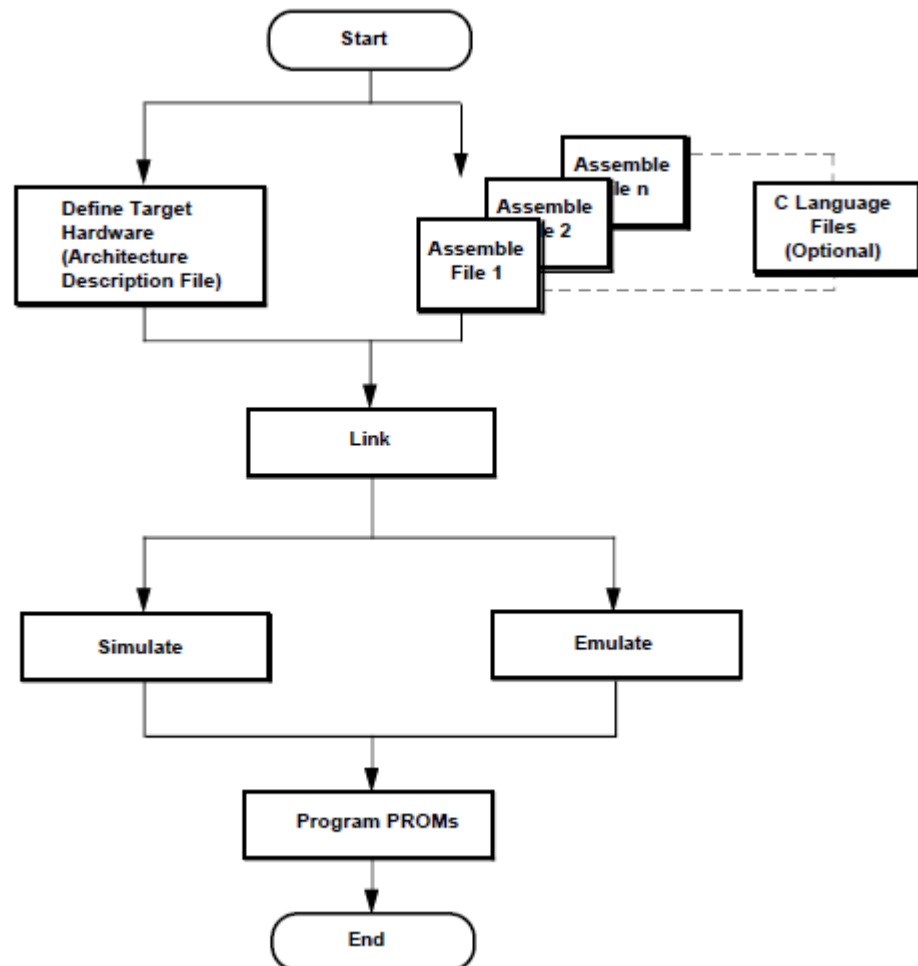


Figure VI.5.

Ces différentes phases restent assez classiques et Analog Devices fournit des outils logiciels pour chacune d'entre elle.

Nous les détaillerons dans les chapitres suivants.

VI.2.2.1. La phase d'écriture du programme.

Votre éditeur de textes préféré conviendra très bien ... ou mieux un **éditeur syntaxique**.

Ecrire le programme source en C ou en assembleur ...

VI.2.2.2. La phase de compilation du programme.

Les outils correspondant se trouvent dans le répertoire C:\EZ-KIT\BIN.

Pour un programme écrit en C :

- soit utiliser le compilateur **G21K** en tapant : G21K MY_PROG.C.

Cet utilitaire enchaîne plusieurs traitements, chacun d'entre eux appelant un programme différent.

- préprocesseur **cpp**,
- compilateur C **cc1**,
- assembleur **asm21K**,
- linker **ld21K**,

la sortie de chaque étage devenant l'entrée de l'étage suivant (transformation du C en assembleur du DSP natif puis un assemblage en code intermédiaire objet .OBJ relogeable mais non encore exécutable, et enfin linkage du .OBJ pour résoudre l'adressabilité sur l'EZ-KIT et produire MY_PROG.EXE.

Les .OBJ et les .EXE sont au format **COFF** (Commun Object File Format, bien adapté pour contenir des informations sur la résolution des adresses, sur le débogage symbolique et sur le modèle de noyau implanté). Cette méthode permet d'éviter l'étape suivante du linkage.

Remarque : taper G21K -? pour obtenir la liste des options possibles de G21K.

- soit taper **G21K -C MY_PROG.C** qui ne fait que fabriquer le code objet, puis qui rend la main.

Pour un programme écrit en l'assembleur DSP :

- soit utiliser l'assembleur **ASM21K** (cf. figure VI.6.) en tapant :
ASM21K -ADSP21060 MY_PROG.ASM pour fabriquer le .OBJ,
De la même façon, taper **ASM21K -?** pour obtenir la liste des options possibles de ASM21K.
Par exemple l'option **-l** permet la création d'un fichier .LST qui peut être utile pendant le débogage.

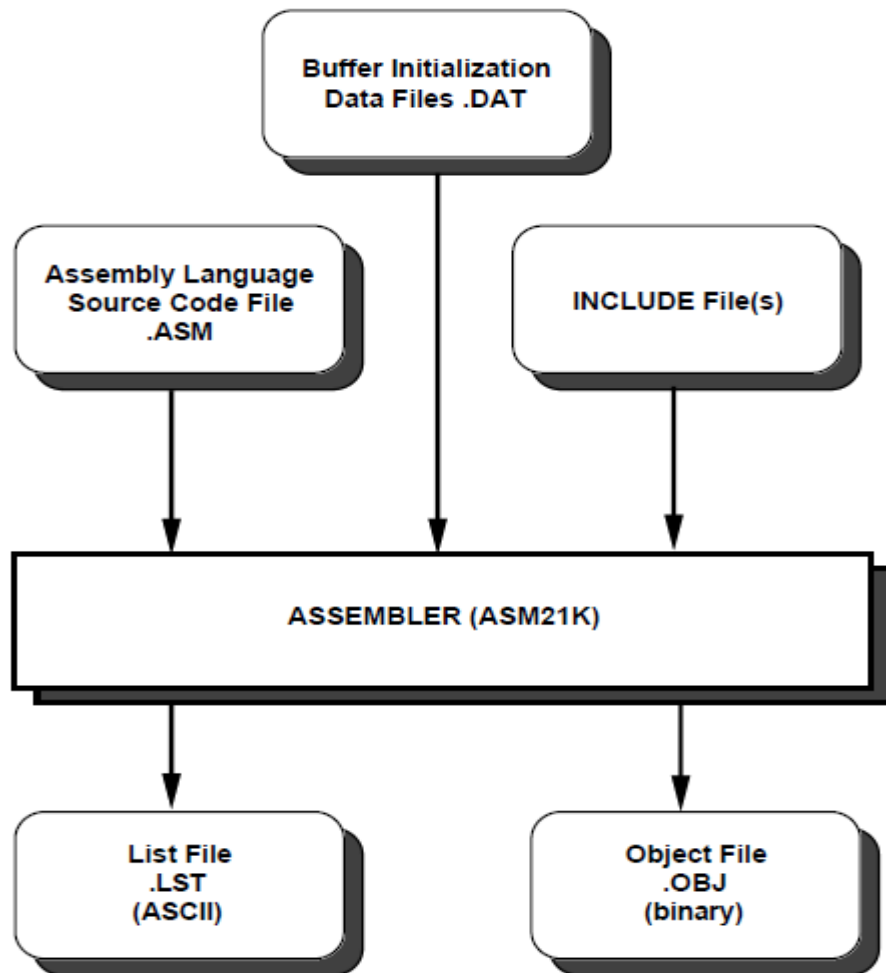


Figure VI.6.

- soit le compilateur **G21K** en tapant :

G21K -C MY_PROG.ASM, l'extension .ASM permettant d'identifier un programme écrit en assembleur et l'option **-C** stoppant comme on l'a déjà vu après l'étape d'assemblage.

VI.2.2.3. La phase d'édition de liens ou de linkage du programme.

Le linker est **LD21K**. Son action est résumée sur la figure VI.7.

Il utilise le fichier d'architecture matérielle **.ACH** que l'on a présenté plus haut pour placer le code et les données au bon endroit de la mémoire, sans empiéter sur et sans rentrer en conflit avec les parties résidentes du système.

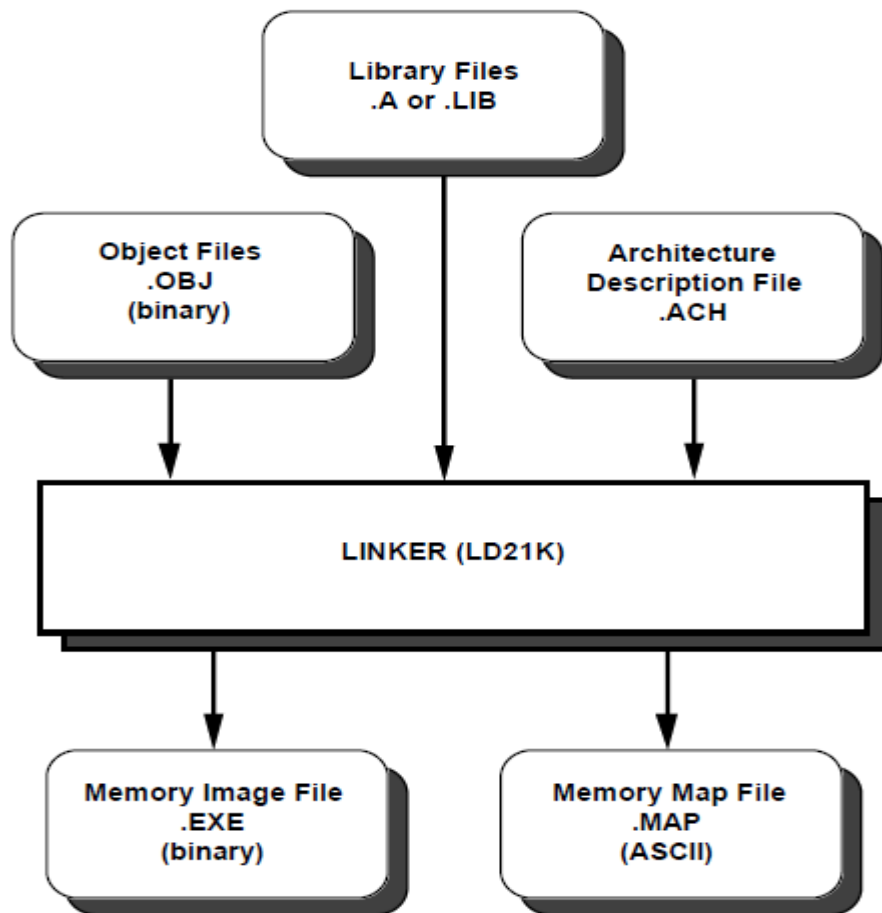


Figure VI.7.

Taper **LD21K MY_PROG** (ou liste des programmes et bibliothèques à linker).

De la même façon, taper **LD21K -?** pour obtenir la liste des options possibles de LD21K comme la création d'une table des symboles (du type **SYMBOLE : VALEUR**) ou l'obtention d'un fichier **.MAP** (localisation en mémoire du code et des données), ces deux fichiers pouvant être utiles pendant les phases de mise au point du programme.

Notez dès à présent l'option **-O** du linker permettant de changer l'extension **.EXE** par défaut de l'exécutable généré, en extension **.21K** pour éviter la confusion entre les exécutables pour PC et ceux pour le DSP.

REMARQUE IMPORTANTE : pour simplifier votre tâche en TP et pour ne pas trop se perdre dans la localisation des différents fichiers ou dans leurs nombreuses options, je vous ai fabriqué des fichiers batch qui permettent d'enchaîner automatiquement toutes ces opérations avec une configuration suffisante pour les TP, comme par exemple le batch **COMPIL**. Ce batch permet d'obtenir directement un exécutable DSP (**.21K**) et téléchargeable sur la carte de d'évaluation.

VI.2.2.4. La phase de simulation du programme.

Cette phase n'étant pas obligatoire, elle ne sera donc pas systématiquement effectuée en TP.

Le simulateur fourni, **WSIM60** tourne sous Windows. Il permet de vérifier et de tester votre programme avant de le télécharger sur l'EZ-KIT.

Il travaille en fait sur une représentation virtuelle du DSP et de son environnement fourni par le fichier d'architecture .ACH.

On peut notamment :

- visualiser les registres internes du DSP ou examiner la mémoire,
- visualiser les données sous leur nom symbolique,
- visualiser le code du programme et son implantation mémoire,
- lancer un programme,
- faire du pas à pas de programme (stepping ou mode trace) en C ou en assembleur,
- mettre des points d'arrêt (breakpoints)...

C'est un outil puissant mais malgré tout relativement convivial d'utilisation.

VI.2.2.5. La phase de téléchargement, de mise au point et de test sur le DSP.

On a vu précédemment l'existence d'un KERNEL sur le DSP pour assurer la liaison avec le PC.

Si vous le souhaitez, en gérant le protocole question/réponse de ce noyau, vous pouvez vous-même (et si vous êtes courageux) écrire un programme de communication avec l'EZ-KIT.

En pratique, Analog Devices l'a fait pour vous et vous propose deux utilitaires pour cela :

- sous Windows avec le **SHARC EZ-KIT HOST PROGRAM** :

Les principaux icônes et leur signification sont donnés sur la figure VI.8.

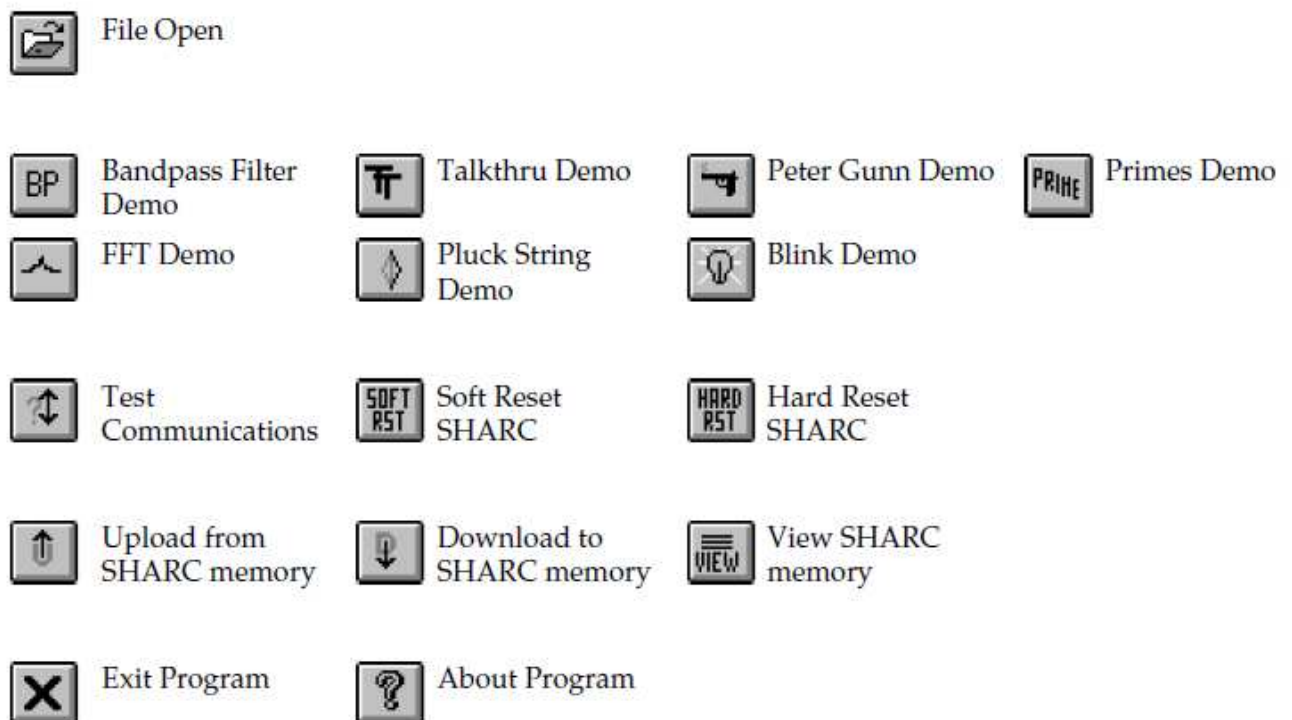


Figure VI.8.

Ainsi, on peut :

- lancer des programmes de démonstration,
- télécharger puis exécuter des programmes écrits en C ou en assembleur,
- lire ou modifier le code ou les données sur l'EZ-KIT,
- arrêter un programme en cours d'exécution ou « resetter » l'EZ-KIT,
- tester ou paramétrer la liaison série avec le PC : après un reset de l'EZ-KIT, il communique avec le PC à 9600 Bauds. Notons qu'on peut changer cette vitesse (des vitesses plus importantes comme 19200, 38400, 57600 ou 115200 Bauds sont disponibles) mais si l'on "re-resette" l'EZ-KIT ou si on coupe son alimentation, la liaison ne marchera plus car il faudra tout d'abord la remettre à 9600 Bauds.

- sous DOS avec le programme utilitaire **DIAG21K** :

On le lance en tapant simplement **DIAG21K** sous DOS ou **DIAG21K -h** (ou **-?**) si on veut connaître les options de sa ligne de commande. Celles-ci sont résumées sur le tableau de la figure VI.9.

Diag21k Command-line Switches

Switch	Description
-h or -?	Displays Diag21k command-line syntax and options.
-bn or -Bn	Instructs Diag21k to open board number <i>n</i> at start-up. Diag21k will look for the environment variable ADSP <i>n</i> to determine the board type. You should specify multiple -b switches to use Diag21k with more than one board. If you do not specify a board number, Diag21k defaults to board number 0.
-pfilename	Diag21k will automatically load the DSP program <i>filename</i> after the board is opened. If you specify multiple -b switches, Diag21k will download the program to the board described by the first -b switch.
-Pprompt	Diag21k will use the string <i>prompt</i> at the command line instead of the standard "diag21k" prompt. The active board number and ">" will be appended to the end of <i>prompt</i> .
-xfilename	Diag21k will execute the command file <i>filename</i> after all the boards are opened.

Figure VI.9.

Lorsque l'on est sous DIAG21K, les tableaux de la figure VI.10. donnent les principales commandes ainsi que les conventions utilisées pour leurs arguments.

On obtient de l'aide sur la syntaxe d'une commande en tapant sous DIAG21K :

? NOM_DE_LA_COMMANDE.

On voit que cet utilitaire permet entre autre :

- d'identifier la cible (type de DSP, fichier .ACH utilisé pour la configuration),
- de « resetter » l' EZ-KIT,
- de télécharger un exécutable,
- d'obtenir des informations sur les registres internes du DSP,
- de lire ou de modifier ces registres internes,
- de lire ou d'écrire en mémoire,
- de lire ou d'écrire sur les Flags,
- de lancer le programme téléchargé ...

Mais on ne peut pas faire du pas-à-pas ou mettre des break points ...

Summary of Diag21k Commands

Command	Description
--	Comment (ignore remainder of line)
?	Display command list (help)
bi	Board Info: display information about active board
br	Board Reset: hard-reset entire DSP board
bs	Board Select: specify active board
echo	Print a message to the screen
fl	File Load: download DSP executable (COFF)
fx	File Execute: Run a command file
ga	Get Address: Display address of global variable
ih	IOP Help: display IOP register descriptions
in	Input word from I/O port
ir	IOP Read: display IOP register value
iw	IOP Write: set IOP register value
md	Memory Dump: read DSP memory and write output to file
mh	Memory Help: display memory command help screen
ml	Memory Load: Read data from file and write to DSP memory
mr	Memory Read
mt	Memory Test
mv	Memory View: display memory range graphically
mw	Memory Write
os	Operating System: Open a command shell
out	Output word to I/O port
pc	Processor Configure
pf	Processor Flag read
pfw	Processor Flag write
pr	Processor reset
ps	Processor start
q	Quit Dsp21k, reset active board
x	Exit: quit DSP21k, leave processor running

Diag21k Command Argument Conventions

Notation	Description
[]	The argument inside the brackets is optional
[c]	Continuous Mode: Appending 'c' to the command will cause it to repeat continuously—until a key is hit. An example is memory read (mrc), which will poll a memory location until a key is hit.
[p]	Paged Mode: Appending 'p' to the command will limit the command's output to a single screen. Pressing the <esc> will terminate the command; pressing any other key will allow the next screen to be output.
<bank>	A single letter indicating which memory segment should be affected by the command: p=program, s=data-sram, d=dram, e=external, l=label*, a=all (test command only).
<fmt>	A single letter indicating the data size and format: c=char, f=float (single-precision), h=hex, i=integer, l=long, s=string
<test>	A single letter indicating a memory test that should be performed: s=sequential, r=random, c=checkerboard, a=all.

**When label 'l' is given as a bank argument, the supplied address is assumed to be the name of a global variable.*

Figure VI.10.

VI.2.2.6. La phase de claquage d'une EPROM.

(Non fait dans le cadre de ce TP).

C'est en général une des dernières étapes de la conception d'un programme.

Une fois votre logiciel validé (débugé et testé de « A à Z » à **150%**), l'idée est de couper le lien avec le PC pour rendre votre application autonome.

A chaque mise sous tension de la carte applicative, ce sera votre programme qui fonctionnera, plus besoin de le télécharger à partir d'un PC...

Le **BOOT LOADER**, c'est-à-dire le programme **LDR21K.EXE** est en fait un utilitaire fourni par Analog Devices qui transforme un exécutable DSP fourni par le linker pour qu'il puisse être implanté en EPROM.

En fait, il rajoute un bout de code appelé **LOADER** en tête de cet exécutable pour que ce dernier soit chargé dans la mémoire interne du DSP à partir de l'EPROM, ce qui permet d'avoir son application qui tourne de façon autonome sur l'EZ-KIT.

Le fichier fourni par LDR21K est au format **INTEL HEX-32** ou **MOTOROLA S-RECORD** (format que vous connaissez puisqu'il avait été utilisé au cours des TP microcontrôleurs), tous les deux compatibles avec la plupart des programmeurs d'EPROM dignes de ce nom.

Une fois l'EPROM claquée, la mettre en lieu et place de celle fournie sur l'EZ-KIT et c'est votre programme qui s'exécutera sur la carte à chaque mise sous tension ou après un reset (attention à ce sujet de ne pas oublier de gérer le vecteur d'interruption du reset : prévoir un saut à l'adresse de début du programme...).

Remarque :

L'utilitaire LDR21K.EXE peut aussi gérer d'autres types de BOOT du DSP :

- par le **HOST BOOT** : c'est-à-dire par un autre processeur via les bus d'adresses et de données externes du DSP,
- par le **LINK BOOT** : c'est-à-dire via le LINK PORT par un autre processeur en utilisant le canal 6 de DMA.

VI.2.3. Autres utilitaires et bibliothèques.

VI.2.3.1. Un serveur de fonctions C.

DSPHOST est un outil qui permet d'utiliser au sein d'un programme C **écrit pour le DSP** les ressources du PC hôte (gestion de l'écran, accès au clavier ou aux disques ...).

Cela est très intéressant car à la base le DSP « ne voit pas le PC » qui ne fait pas partie de son environnement normal.

Rappelons que le DSP de la carte d'évaluation est conçu pour traiter numériquement les signaux fournis par le Codec. De plus son langage naturel (son code machine) n'a rien à voir avec le langage machine du microprocesseur Intel d'un PC.

Mais grâce à cet utilitaire, on pourra par exemple piloter à distance une application tournant sur l'EZKIT en utilisant par exemple le clavier et l'écran du PC grâce à un programme écrit en C.

On pourra donc « mélanger » **dans un même programme** des fonctions devant s'exécuter sur le DSP et des fonctions devant s'exécuter sur le PC.

Le principe, en gros, est de faire exécuter par le DSP toutes les fonctions et instructions qui le concernent directement et par le PC toutes les autres fonctions et instructions comme par exemple un printf ou un scanf et de coupler ces deux exécutions.

On utilise pour cela le batch **MK** qui appelle G21K avec la bibliothèque **LIBDH.A** du DSPHOST.

On télécharge ensuite sous DOS l'exécutable sur le DSP de l'EZ-KIT par **DH21K.EXE**

C'est en fait un genre de serveur qui attend les demandes d'I/O émanant du DSP vers le PC et qui les sert.

Une fois le programme DSP terminé, la main est rendue au PC.

La liste des fonctions C servies par DSPHOST est donnée sur les tableaux de la figure VI.11.

On y retrouve bien les fonctions bien connues et standards du C.

Pour le TP, je vous ai écrit un fichier batch, **DSPH.bat**, qui permet d'enchaîner ces différentes étapes de façon automatique.

Function Listing for conio.h

Console and port I/O functions.

cgets	cscanf	kbhit
cprintf	getch	putch
cputs	getche	ungetch

Table 7-2 Function Listing for direct.h

Directory handling and creation functions.

chdir	mkdir
getcwd	rmdir

Table 7-3 Function Listing for io.h

Low-level file handling functions.

access	isatty	setmode
chmod	lseek	tell
chsize	mktemp	umask
close	open	unlink
creat	read	write
dup	read8	write8
dup2	read16	write16
eof	remove	
filelength	rename	

Function Listing for process.h

Process control functions

abort	exit
-------	------

Function Listing for stdio.h

Standard I/O functions

clearerr	fopen	fwrite	rename
fclose	fprintf	fwrite8	rewind
fcloseall	fputc	fwrite16	scanf
fdopen	fputchar	getc	sprintf
feof	fputs	getchar	sscanf
ferror	fread	gets	tmpfile
fflush	fread8	getw	tmpnam
fgetc	fread16	printf	ungetc
fgetchar	freopen	putc	unlink
fgetpos	fscanf	putchar	vfprintf
fgets	fseek	puts	vprintf
fileno	fsetpos	putw	vsprintf
flushall	ftell	remove	

Figure VI.11.

VI.2.3.2. Gestion d'une librairie.

L'utilitaire **LIB21K** permet de maintenir des librairies de fonctions (liste des fonctions présentes en librairie, création de nouvelles fonctions avec rajout en librairie ou destruction de fonctions existantes en librairie).

Il est surtout utile en programmation en assembleur car en C, un grand nombre de fonctions sont déjà disponibles.

De la même façon, taper **LIB21K -?** pour obtenir la liste des options possibles de **LIB21K**.

La figure VI.12. montre l'action de **LIB21K**.

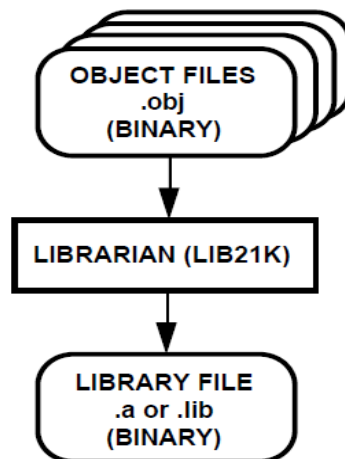


Figure VI.12.

La figure VI.13. montre quant à elle ce qui se passe si on tape successivement :

LIB21K -C FILTER.LIB TAPS COEFFS GO_INPUT ;création de la librairie **FILTER.LIB**,

LD21K MAIN SUM GRAPH FILTER.LIB ; linkage en utilisant cette librairie.

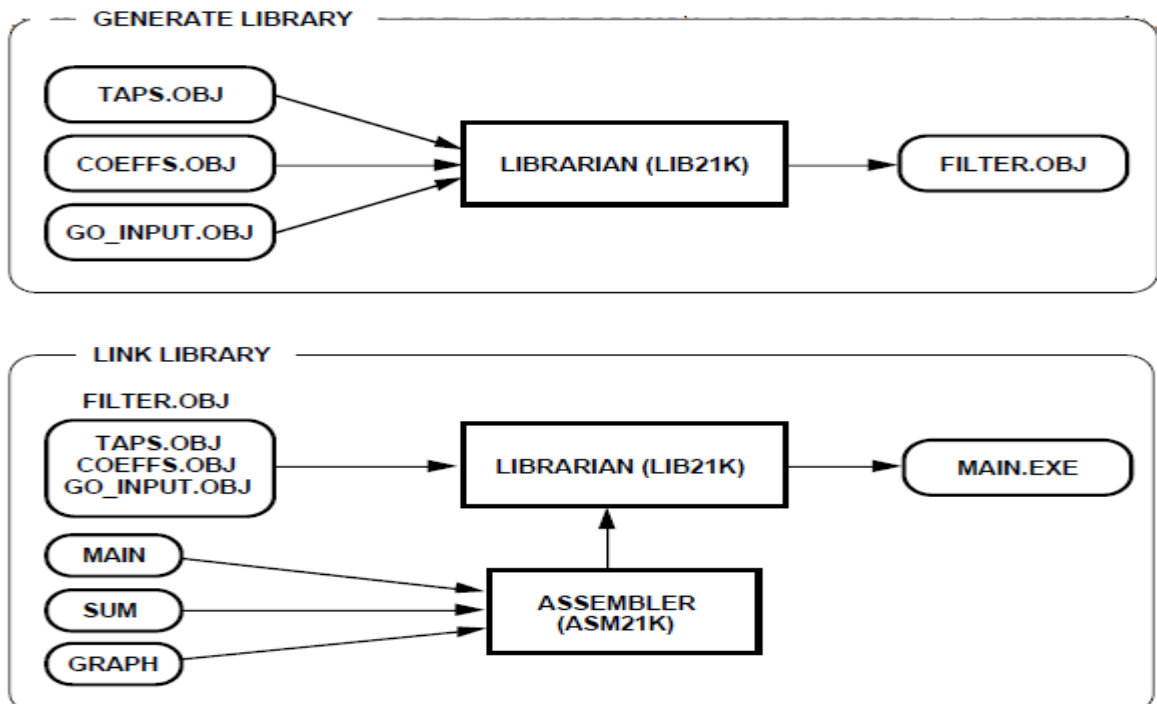


Figure VI.13.

VI.2.3.3. Quelques sources assembleur d'Analog Devices.

Analog Devices fournit également le **code source** de quelques fonctions écrites en assembleur.

Dans certains cas, cela peut être intéressant car on peut s'en servir comme base de travail pour les adapter/modifier et ainsi les utiliser ou pour créer sa propre librairie de travail personnel.

Ce sont les fonctions listées en figure VI.14.

<i>Function</i>	<i>Routine Name</i>
division	divide
reciprocal	recip
square root	sqrt
reciprocal square root	recip_sqrt
sine	sine
cosine	cosine
tangent	tan
e^x	exponential
$\ln x$	log
$\log_2 x$	logb2
$\log_{10} x$	log10
x^y	pow
μ -law compression	ucompress
μ -law expansion	uexpand
A-law compression	acompress
A-law expansion	aexpand

Figure VI.14.

Il faut, pour les utiliser correctement, initialiser certains registres (passage de paramètres) avant de les appeler.

Au retour, la fonction retourne sa (ou ses) valeur(s) dans certains registres également.

Dans tous les cas, il faudra se référer à une documentation plus complète pour l'usage précis de chacune d'entre elles.

VII. MISE EN ŒUVRE EN LANGAGE C.

Les présentations matérielles et les éléments essentiels de l'environnement de développement ayant été fournis, nous pouvons maintenant les mettre en pratique pour écrire un premier programme en C exploitant **au mieux** les principales caractéristiques de la carte d'évaluation.

VII.1. Un exemple commenté et détaillé en C.

Je vous propose donc le programme **CANEVAS.C** dont le source est fourni dans les pages suivantes.

Notez bien que ce programme ne fait aucun traitement numérique du signal (ou si peu), mais il représente la structure minimale nécessaire pour exploiter au mieux les caractéristiques de la carte d'évaluation : c'est une espèce de « socle » minimal que vous utiliserez pour faire les différents traitements numériques demandés en TP. D'où son nom...

Vous y rajouterez donc les lignes de code nécessaire pour réaliser les applications demandées en TP.

☛☛☛ compte-tenu de la relative complexité de ce programme, il est conseillé de toujours garder une version initiale intègre de CANEVAS.C en changeant son nom à chaque modification que vous effectuerez.

Revenons à ce programme.

Différentes structures de données et fonctions ont été écrites ou appelées depuis la librairie du C comme par exemple :

- **void init_dsp(void)** : pour initialiser le DSP et son timer interne pour qu'il génère des interruptions périodiques,
- **void init_sport0(void)** : pour initialiser et configurer la liaison SPORT0 entre le DSP et le Codec sous DMA chaînée en réception (via la structure tcb **rx_tcb** et le buffer **rx_buf[3]**) et en transmission (via la structure tcb **tx_tcb** et le buffer **tx_buf[3]**) avec traitement des interruptions ainsi que pour gérer le protocole TDM,
- **void init_codec(void)** : pour « resetter » correctement le Codec (par sa broche **RESET**) en lançant une auto-calibration avant de l'initialiser,
- **void it_timer()** : routine d'interruption pour gérer les interruptions périodiques du timer interne du DSP,
- **void it_dma_receive()** : routine d'interruption pour traiter les interruptions en réception sous DMA des échantillons du signal en entrée,
- **void it_dma_transmit()** : routine d'interruption pour traiter les interruptions en émission sous DMA des échantillons du signal en sortie,
- **set_flag()**: fonction de Analog Devices pour piloter les FLAGS LEDs,
- **asm(« »)** : syntaxe permettant d'inclure en ligne du code assembleur dans un source écrit en C ...

Conseil : essayer de relire ce source et de le comprendre est un bon moyen pour vous de faire les liens avec et entre tous les aspects théoriques et pratiques vus jusqu'à maintenant.

```

/*****
/*      PROGRAMME CANEVAS.C
*****/

/* fichier de définition des registres internes et de certains de leurs champs de bits */
#include <def21060.h>
#include <21060.h>
#include <sport.h>          /* les registres et champs de bits des SPORT0 et 1 */
#include <macros.h>

/* d'autres include pour le traitement du signal ....*/
#include <signal.h>
#include <filters.h>
#include <math.h>

/* définition d'un tableau de 16 Words, chacun correspondant à : */
/* -pour l'octet de poids fort :      CLOR  MCE   RREQ  Res   numéro d'index du registre */
/*                                1      1      0      0   de $00 à $FF */
/* CLOR = 1 : remise à jour des bits de surcharge des CAN d'entrée dans le Status Word entre */
/* chaque acquisition : permet éventuellement si on le gère d'augmenter la précision des CAN */
/* MCE est mis à 1 pour les 15 premiers Words et à 0 au dernier pour faire une autocalibration du */
/* Codec après son initialisation */
/* - pour l'octet de poids faible : valeur à écrire dans ce registre */
/* Ce tableau regs_1847 est en fait la liste des 16 mots de contrôle qu'il faut envoyer pour initialiser le Codec */

int      regs_1847[16]= {

    0xC000,      /* registre d'index 0 : CAN sur entrée gauche de Line1 , 0 dB d'atténuation */
    0xC100,      /* registre d'index 1 : CAN sur entrée droite de Line1 , 0 dB d'atténuation */
    0xC280,      /* registre d'index 2 : coupe voie gauche de l'entrée Aux1, +12 dB si présente */
    0xC380,      /* registre d'index 3 : coupe voie droite de l'entrée Aux1, +12 dB si présente */
    0xC480,      /* registre d'index 4 : coupe voie gauche de l'entrée Aux2, +12 dB si présente */
    0xC580,      /* registre d'index 5: coupe voie droite de l'entrée Aux2, +12 dB si présente */
    0xC600,      /* registre d'index 6 : CNA validé sur sortie gauche de Line , 0 dB d'atténuation */
    0xC700,      /* registre d'index 7 : CNA validé sur sortie droite de Line , 0 dB d'atténuation */
    0xC850,      /* registre d'index 8 : format des données échangées avec le DSP */
                    /* C/L = 0 : format linéaire ( pas de compression ) */
                    /* PCM=1 : 16 bits  signés */
                    /* S/M=1 : en stéréo */
                    /* CSF2:0 = 000 : échantillonnage à 8 kHz ( voir doc AD1847 pour autres fréquences ) */
                    /* CSL = 0 :choix de XTAL1 à 24,576 MHz */
    0xC909, /* registre d'index 9 : Interface Configuration */
                    /* ACAL = 1 : autorise le lancement d'une autocalibration du Codec après son */
                    /* initialisation lorsque MCE repassera à 0 ( à faire après chaque mise sous tension ) */
                    /* PEN = 1 : sortie des données sur le CNA */
    0xCA00,      /* registre d'index 10 : pour la gestion de l'horloge commune de la liaison */
                    /* série en cas de plusieurs Codecs : ici un seul Codec en Bus Master, donc : */
                    /* CLKTS = 0 et XCTL1:0 = 00 toujours */
    0xCB00,      /* registre d'index 11 : pas de registre : usage futur : on envoie 00 dans rien !! */
    0xCC40,      /* registre d'index 12 : pour la gestion du protocole TDM */
                    /* FRS = 0 : 32 slots par trame */
                    /* TSSEL = 1 : système à 2 fils (voir pages 62 et 63 de ce document) */
                    /* rappelons que la clock de la liaison DSP-Codec est à 12,288 MHz */
    0xCD00,      /* registre d'index 13 : pour le mixage numérique entre sortie CAN et entrée CNA */
                    /* DME = 0 : pas de mixage ici */
                    /* atténuation de -1,5 dB si ce mixage était autorisé */
    0xCE00,      /* registre d'index 14 : pas de registre : usage futur : on envoie 00 dans rien !! */
    0x8F00      /* registre d'index 15 : pas de registre : usage futur : on envoie 00 dans rien !! */
                    /* mais force le retour de MCE à 0, donc lance l'autocalibration !! */

};

```

```

/* buffer de réception des données venant du Codec : taille de 3 Words : status, voie gauche, voie droite */
int    rx_buf[3];
/* buffer de transmission des données vers le Codec : taille de 3 Words : contrôle, voie gauche et voie droite */
/* ici initialisés pour configurer le TDM */
int    tx_buf[3]={0xCC40,    0,    0};

/* définition d'une structure de TCB pour le chaînage des DMA de la liaison DSP-Codec */
/* voir pages 46 à 50 du polycopié de cours complet */
/* un TCB complet comprend en fait 9 éléments et non 4 comme dans la présentation simplifiée page 26 */
typedef struct {
    unsigned    ei;    /* non utilisé ici : multiprocessing */
    unsigned    em;    /* non utilisé ici : multiprocessing */
    unsigned    ec;    /* non utilisé ici : multiprocessing */
    unsigned    gp1;    /* registre d'usage général n°1 */
    unsigned    gp2;    /* registre d'usage général n°2 */
    unsigned**   cp;    /* pointe sur le TCB suivant dans la chaîne */
    unsigned    c;    /* taille du bloc à transférer à chaque DMA */
    int          im;    /* incrément pour calculer l'adresse du transfert suivant dans le DMA en cours */
    unsigned*    ii;    /* pointeur sur le buffer : adresse de base du buffer */
}    tcb;

/* le premier TCB de la chaîne de réception initialisé avec 3 mots par DMA à des adresses successives (c = 3 et im = 1) */
tcb    rx_tcb = {0,    0,    0,    0,    0,    0,    3,    1,    0};
/* le premier TCB de la chaîne d'émission initialisé avec 3 mots par DMA à des adresses successives (c = 3 et im = 1) */
tcb    tx_tcb = {0,    0,    0,    0,    0,    0,    3,    1,    0};
int*    tx_ptr;
int    tx_compteur;

/*****/
/*    interruption timer exécutée ici 4 fois par seconde
*****/
void    it_timer ( int sig_num )
{
    /* inverse l'état de la Led câblée sur la sortie FLAG2 */
    set_flag ( SET_FLAG2, TGL_FLAG );    /* extension du C : */
                                         /* mettre SET_FLAG pour l'éteindre et CLR_FLAG pour l'allumer */
}

/*****/
/*    interruption DMA en transmission sur tx_buf vide
*****/
void    it_dma_transmit ( int sig_num )
{
    /* vérifie si les 16 données d'initialisation ont été envoyées au Codec */
    if ( tx_compteur )
    {
        /* mettre le word dans tx_buf[0], pointer sur le suivant et décrémenter le comptage */
        tx_buf[0] = *tx_ptr++;
        tx_compteur--;

        /* n'oubliez pas que (ici en stéréo) tx_buf[1] (voie gauche) et tx_buf[2] (voie droite) peuvent contenir */
        /* des valeurs numériques que l'on peut envoyer */
        /* vers les CNA du Codec, tx_buf[0] contenant quant à lui le mot de contrôle (revoir le protocole TDM) */
    }
}

/*****/
/*    interruption DMA en réception sur rx_buf plein
*****/
void    it_dma_receive ( int sig_num )
{
    /* on mettra ici le soft de traitement du signal qui reçoit les échantillons (en stéréo) dans rx_buf[1] (voie gauche) et */
    /* rx_buf[2] (voie droite) et qui peut générer une sortie numérique à envoyer dans tx_buf[1] ou tx_buf[2] */
    /* ici on peut par exemple simplement renvoyer la valeur lue sur la voie Line1 gauche vers la sortie Line gauche */
    tx_buf[1] = rx_buf[1];
}

```

```

/*****
/*      initialisation du DSP
/*      et exemples de routine de commande du timer
/*      et d'insertion de code assembleur dans du C : asm ( " mnémonique assembleur " )
*****/
void    init_dsp ( void )
{
    timer_off ( );                /* extension du C ANSI inhibant le timer */
    timer_set( 10000000,    10000000 ); /* initialise TPERIOD et TCOUNT du timer */
                                        /* génération d'une IT lorsque TCOUNT=0 */
                                        /* ici à 40 MHz : entre chaque IT :  $10^7/40.10^6 = 0,25$  s */
                                        /* donc fréquence des IT du timer = 4 Hz */

    asm( " #include <def21060.h> " );
    asm ( " bit set mode1 NESTM; " ); /* imbrication d'interruption autorisée : ici timer, DMA */
                                        /* par mise à 1 du bit 11 NESTM du registre interne mode1 */
                                        /* exemple d'insertion de code assembleur dans du C */

    tx_ptr      = regs_1847;        /* pointeur vers données d'initialisation du Codec */
    tx_compteur  = 0;                /* init du comptage de mots à envoyer */

    interrupt ( SIG_TMZ,    it_timer ); /* installation de l'interruption timer sur TCOUNT = 0 */
                                        /* placée en bas niveau de priorité */
                                        /* pour le haut niveau utiliser SIG_TMZ0 au lieu de SIG_TMZ */
                                        /* toutes les 0,25 s, la routine it_timer sera exécutée */
}

/*****
/*      initialisation et configuration de SPORT0 du DSP
/*      autorisation et installation des vecteurs de DMA
/*      initialisation des TCB et lancement des DMA en émission et en réception
*****/
void    init_sport0 ( void )
{
    /* affectation et configuration des slots */
    sport0_iop.mtcs =    0x00070007; /* transmission sur slots 0,1,2,16,17,18 */
    sport0_iop.mrcs =    0x00070007; /* réception sur slots 0,1,2,16,17,18 */
    sport0_iop.mtccs =    0x00000000; /* pas de compression en émission sur tous les slots */
    sport0_iop.mrccs =    0x00000000; /* pas de compression en réception sur tous les slots */

    /* écriture dans le STCTL0 : le Transmit Control Register */
    /* les champs de bits sont définis dans sport.h */
    sport0_iop.txc.mdf      = 1; /* délai entre chaque trame du TDM en multi-channel */
    sport0_iop.txc.schen    = 1; /* autorisation du chaînage de DMA sur émission */
    sport0_iop.txc.sden     = 1; /* autorisation de la DMA sur émission */
    sport0_iop.txc.lafs     = 0; /* à 0 en multi-channel */
    sport0_iop.txc.ltfs     = 0; /* synchro trame active au niveau haut */
    sport0_iop.txc.ditfs    = 0; /* synchro trame à chaque chargement du transmit register */
    sport0_iop.txc.itfs     = 0; /* à 0 en multi-channel */
    sport0_iop.txc.tfsr     = 0; /* à 0 en multi-channel */
    sport0_iop.txc.ckre     = 0; /* échantillonnage des bits sur front descendant de la clock */
    sport0_iop.txc.gclk     = 0; /* clock générée en transmission seulement */
    sport0_iop.txc.iclk     = 0; /* clock générée en externe utilisée */
    sport0_iop.txc.pack     = 0; /* transmission de 32 bits en 2 fois 16 bits */
    sport0_iop.txc.slen     = 15; /* longueur d'un mot ( ici 16 bits ) -1 */
    sport0_iop.txc.sendn    = 0; /* format Endian ( = 0 : MSB en premier, = 1 : LSB en premier */
    sport0_iop.txc.dtype    = 1; /* justification à droite avec extension de signe aux MSB */
    sport0_iop.txc.spen     = 0; /* à 0 en multi-channel */
}

```

```

/* écriture dans le SRCTL0 : le Receive Control Register */
/* les champs de bits sont définis dans sport.h */
    sport0_iop.rxc.nch      = 31; /* en TDM, nombre de slots -1 */
    sport0_iop.rxc.mce      = 1; /* autorisation du TDM ou multi-channel */
    sport0_iop.rxc.spl      = 0; /* à 0 en multi-channel */
    sport0_iop.rxc.d2dma    = 0; /* non validation de la DMA en D2 : à 0 en multi-channel */
    sport0_iop.rxc.schen    = 1; /* autorisation du chaînage de DMA en réception */
    sport0_iop.rxc.sden     = 1; /* autorisation de la DMA en réception */
    sport0_iop.rxc.lafs     = 0; /* à 0 en multi-channel */
    sport0_iop.rxc.ltfs     = 0; /* synchro trame active au niveau haut */
    sport0_iop.rxc.irfs     = 0; /* synchro trame générée en externe */
    sport0_iop.rxc.rfsr     = 0; /* à 0 en multi-channel */
    sport0_iop.rxc.ckre     = 0; /* échantillonnage des bits sur front descendant de la clock */
    sport0_iop.rxc.gclk     = 0; /* clock générée en transmission seulement */
    sport0_iop.rxc.iclk     = 0; /* clock générée en externe utilisée */
    sport0_iop.rxc.pack     = 0; /* transmission de 32 bits en 2 fois 16 bits */
    sport0_iop.rxc.slen     = 15; /* longueur d'un mot ( ici 16 bits ) -1 */
    sport0_iop.rxc.sendn    = 0; /* format Endian ( = 0 : MSB en premier, = 1 : LSB en premier */
    sport0_iop.rxc.dtype    = 1; /* justification à droite avec extension de signe aux MSB */
    sport0_iop.rxc.spen     = 0; /* à 0 en multi-channel */

/* autorisation des interruptions DMA sur SPORT0 en réception et en émission */
    interrupt ( SIG_SPR0I,   it_dma_receive ); /* installation de l'interruption DMA en réception */
                                              /* cette it sera exécutée chaque fois que le buffer de */
                                              /* réception de 3 mots rx_buf sera plein */
    interrupt ( SIG_SPT0I,   it_dma_transmit ); /* installation de l'interruption DMA en émission */
                                              /* cette it sera exécutée chaque fois que le buffer de */
                                              /* transmission de 3 mots tx_buf sera vide */

/* initialisation du TCB de transmission pour le chaînage des DMA */

    tx_tcb.ii = tx_buf; /* pointe sur le buffer de transmission */
    tx_tcb.cp = &tx_tcb.ii; /* pointe sur lui-même donc paramétrage identique pour toutes les DMA */
                          /* qui vont s'enchaîner */
    *(int*)CP2 = ( (int)&tx_tcb.ii | 0x30000 ); /* initialise le registre CP2 du DSP */
                                              /* ( canal 2 de DMA en transmission ) */
                                              /* force le bit 17 ( bit PCI autorisant l'IT DMA ) */
                                              /* et le bit 16 ( MSB de l'adresse du premier TCB ) à 1 */
                                              /* initialise le premier paramétrage et lance la DMA en transmission */
                                              /* les 3 words de tx_buf vont être émis puis une IT DMA émission aura lieu */
                                              /* entraînant l'exécution de la routine it_dma_transmit et un paramétrage */
                                              /* automatique de la prochaine DMA avec les mêmes tx_tcb et tx_buf */
                                              /* ( ce n'est pas à l'utilisateur de lancer la prochaine DMA ) */

/* initialisation du TCB de réception pour le chaînage des DMA */

    rx_tcb.ii = rx_buf; /* pointe sur le buffer de réception */
    rx_tcb.cp = &rx_tcb.ii; /* pointe sur lui-même donc paramétrage identique pour toutes les DMA */
                          /* qui vont s'enchaîner */
    *(int*)CP0 = ( (int)&rx_tcb.ii | 0x30000 ); /* initialise le registre CP0 du DSP */
                                              /* ( canal 0 de DMA en réception ) */
                                              /* force le bit 17 ( bit PCI autorisant l'IT DMA ) */
                                              /* et le bit 16 ( MSB de l'adresse du premier TCB ) à 1 */
                                              /* initialise le premier paramétrage et lance la DMA en réception */
                                              /* les 3 premier words reçus seront stockés dans rx_buf */
                                              /* puis une IT DMA réception aura lieu */
                                              /* entraînant l'exécution de la routine it_dma_receive et un paramétrage */
                                              /* automatique de la prochaine DMA avec les mêmes rx_tcb et rx_buf */
                                              /* ( ce n'est pas à l'utilisateur de lancer la prochaine DMA ) */
}

```

```

/*****
/*      envoi des initialisations vers le Codec
*****/
void    init_codec ( void )
{
    tx_ptr      = regs_1847;          /* pointeur vers données d'initialisation du Codec */
    tx_compteur  = 16;                /* init du comptage de mots à envoyer */

    /* puis on attend l'émission complète des 16 mots sous IT DMA par la routine it_dma_transmit */
    while ( tx_compteur )
        idle ();                    /* attente jusqu'à tx_compteur = 0 */

    /* on a vu qu'après le dernier mot de contrôle envoyé par le DSP, le Codec se met en autocalibration */
    /* on vient lire le Status Word émis par le Codec et reçu sous IT DMA réception dans rx_buf[ 0 ] */
    /* le bit 1 (bit ACI) du Status Word est à 1 pendant l'autocalibration ( 384 périodes d'échantillonnage ) et à 0 sinon */
    /* il faut d'abord attendre que le bit ACI soit à 1 : démarrage de l'auto-calibration */
    /* sinon on risque de trouver 0 tout de suite et de penser que l'auto-calibration est finie alors qu'elle n'a pas commencé */
    /* on fait pour cela une scrutation de l'état haut de ACI */
    while ( !( rx_buf[ 0 ] & 0x0002 ) )
        idle ();                    /* attente jusqu'à bit ACI = 1 : début auto-calibration */
    /* puis on attend la fin de l'autocalibration en scrutant le passage à 0 du bit ACI */
    while ( rx_buf[ 0 ] & 0x0002 )
        idle ();                    /* attente jusqu'à bit ACI = 0 : fin d' auto-calibration */
}

/*****
/*      Le main
*****/
void    main    ( void )
{
    int i ;
    /* initialisation du DSP */
    init_dsp ( );

    /* reset du Codec : mise à 0 de la sortie FLAG0 relié à la broche /Reset du Codec */
    set_flag ( SET_FLAG0,CLR_FLAG);
    for ( i=0 ; i<0xFFFF ; i++ );    /* et attendre suffisamment longtemps */
    set_flag ( SET_FLAG0,SET_FLAG);   /* avant de remonter le reset */

    /* configuration du SPORT0 du DSP */
    init_sport0 ( );

    /* envoi des commandes d'initialisation vers le Codec : ces commandes sont envoyées sous DMA */

    init_codec ( );

    /* valide le timer et ses interruptions périodiques */
    timer_on ( );

    /* attente infinie, dérangée par des IT du timer et des DMA */
    while ( 1 )
        idle;

}

*****/

```

VII.2. Quelques précisions sur le C fourni par Analog Devices.

VII.2.1. Couplage C/Assembleur.

Cette partie vous permettra de comprendre les conventions utilisées par le compilateur C pour l'appel de fonctions depuis le C. Ces éléments devraient vous aider à écrire des fonctions en assembleur et appelables depuis le C. Ou alors à comprendre comment le compilateur C utilise les ressources hardware du DSP pour gérer les paramètres d'une fonction, pour appeler cette fonction, pour créer des variables locales à une fonction, pour retourner une valeur et pour rendre la main à l'instruction C qui suit l'appel de la fonction. La structure dynamique qui permet tout ce mécanisme est la pile.

Il faut donc tout d'abord apporter des informations sur la gestion de la pile par le C au moment des appels de fonctions.

Certains registres (de **DAG1** permettant d'accéder dans **DM** à la pile du C (**seg_stak**) définie dans le fichier d'architecture matérielle) sont réservés :

- **registre I7** : il sera utilisé comme pointeur de pile (**STACK POINTER** ou **SP**) et contient l'adresse de la première position vide de la pile. On dit qu'il pointe sur **le sommet de la pile**.

- **registre I6** : il sera utilisé comme pointeur du cadre de pile (**FRAME POINTER** ou **FP**).

Le cadre de pile est une zone particulière de la pile qui sert à stocker des informations sur le contexte du programme comme:

- * les paramètres passés à la fonction appelée.
- * les variables locales (automatiques) ou temporaires de cette fonction,

La fonction appelée accèdera à ces informations par leur décalage (**OFFSET**) par rapport à ce frame pointer.

Nous décrirons l'évolution de la pile au fur et à mesure des différentes étapes concernées en partant de l'état initial fourni sur la figure VII.1. ci-dessous :

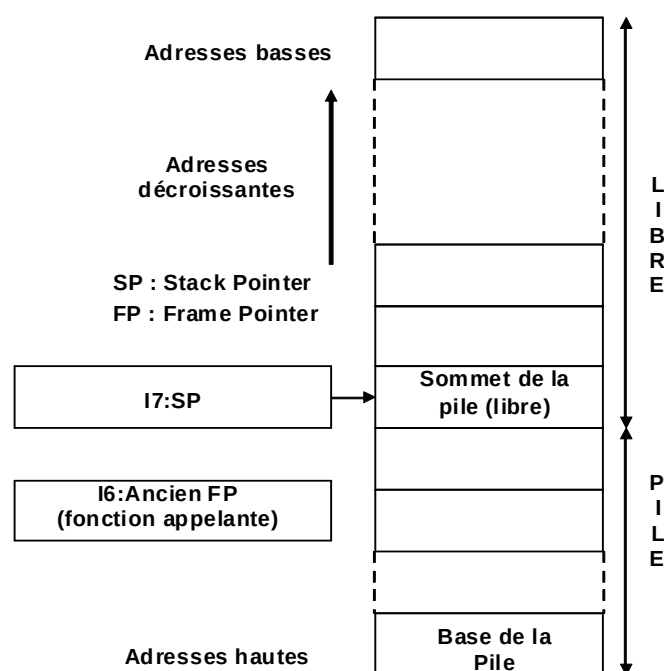


Figure VII.1.

Chaque fois qu'une fonction C est appelée la procédure respectera les étapes suivantes :

① gestion du passage des paramètres de la fonction :

Les conventions suivantes sont appliquées :

- les trois premiers paramètres (si leur taille est ≤ 32 bits) sont passés par registre, ce qui est, on le sait la façon la plus efficace en temps : par convention, le premier sera mis dans le registre **R4**, le deuxième dans le registre **R8** et le troisième dans le registre **R12** (ces 3 registres ont chacun une taille de 32 bits),

- s'il y a plus de trois paramètres ou si un parmi les trois premiers a une taille > 32 bits (par exemple comme un réel double précision sur 40 bits), la fonction appelante empile les paramètres restants sur la pile. Le pointeur de pile **I7** sera à chaque fois décrémenté pour toujours pointer sur le sommet de la pile, qui grandira donc vers les adresses décroissantes. Par convention, les paramètres sont gérés de la droite vers la gauche (le plus à droite dans la liste des arguments de la fonction se trouvera donc à l'adresse la plus haute de la pile et ainsi de suite).

La figure VII.2. suivante donne l'état de la pile après cette opération, en supposant un passage de deux paramètres (paramètre 1 et paramètre 2 par la pile).

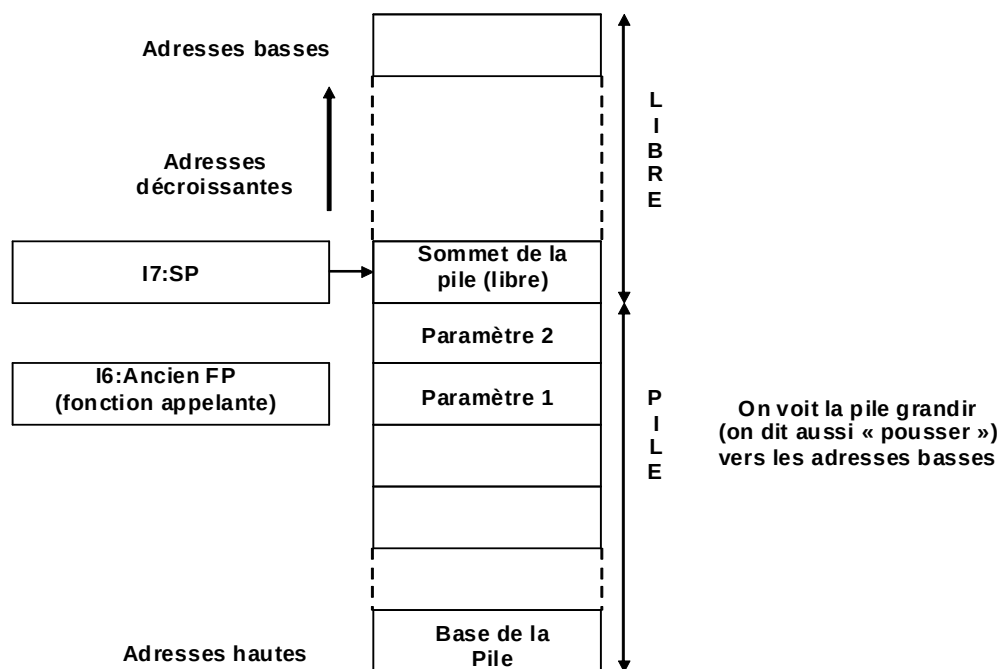


Figure VII.2.

② gestion de « l'ancien frame pointer » :

Le frame pointer de la fonction appelante (le contenu de **I6** correspond à ce moment là à celui de la fonction appelante : on le nomme pour cela « l'ancien frame pointer ») est sauvegardé dans **R2**. (par une instruction assembleur du type **R2 = I6 ;**). L'état de la pile est inchangé.

③ gestion du « nouveau frame pointer » :

Le pointeur de pile **I7** est recopié dans **I6** (par une instruction assembleur du type **I6 = I7 ;**).

Le registre **I6** devient ainsi « le nouveau frame pointer », c'est-à-dire celui de la fonction appelée. Comme il contient la valeur de **I7** à ce moment là, on voit qu'il pointe lui aussi sur le sommet de la pile à cet instant là.

L'état de la pile est toujours inchangé mais maintenant **I6** (le nouveau frame pointer) pointe sur le sommet de la pile à cet instant), comme indiqué sur la figure VII.3.

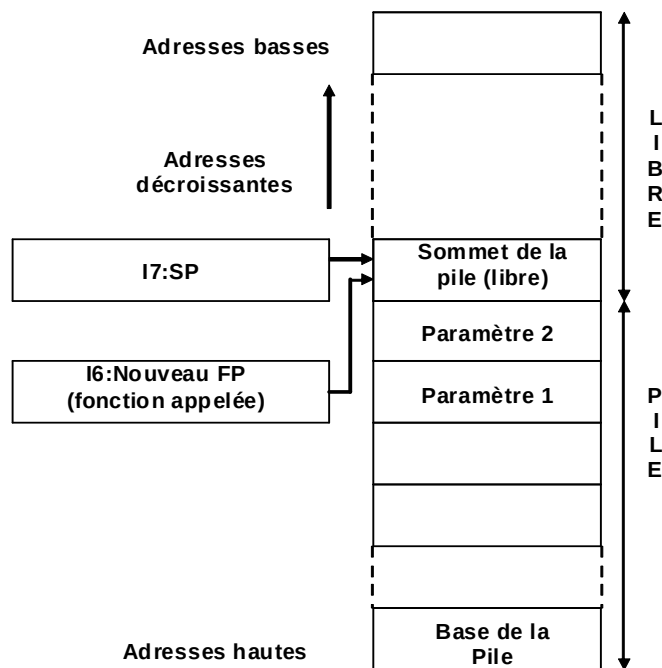


Figure VII.3.

④ appel de la fonction :

Le contrôle est passé à la fonction appelée par un saut vers la première instruction de cette fonction contenu à l'adresse **nom_de_la_fonction_appelée**.

(par une instruction assembleur du type **JUMP(PC, nom_de_la_fonction_appelée) (DB);**)

Le compilateur utilise un saut relatif au Program Counter PC pour faciliter la relogéabilité du code. (programmes **PIC : Position Indépendant Code**) associé à un retard de deux instructions représenté par le symbole **DB** (= **Delayed Branch**=saut « retardé »). C'est-à-dire que les deux instructions suivantes de ce code seront exécutées avant le saut proprement dit. Pourquoi cela ? Tout simplement pour vider le pipe du décodage de l'instruction (k+1) et de la recherche de l'instruction suivante (k+2) associé à cette instruction (k) de saut (cf. figure I.4.).

Quelles seront donc ces deux instructions suivantes ?

⑤ empilement de « l'ancien frame pointer » :

Le contenu de **R2** (et donc « l'ancien frame pointer » cf. étape ②) est empilé dans la pile.
(par une instruction assembleur du type **DM (I7 , M7) = R2;**)

Notons que le C garde dans **M7** la valeur **-1**.

Cette instruction a pour effet de sauvegarder **R2** à l'adresse pointée par **I7**, et donc au sommet de la pile puis de faire **I7 = I7 + M7** ce qui permet de décrémenter **I7** pour qu'il pointe toujours sur la première position libre de la pile (la pile grandit vers les adresses basses !!!).

Rappelons ici qu'il existe deux possibilités de codage de l'instruction **DM** :

- **DM (Mx , Ix)** ou **pré-modification** sans modifier **Ix**,
- **DM (Ix , Mx)** ou **post-modification** avec mise à jour de **Ix**.

Dans le premier cas (**Mx** avant **Ix** dans **DM**), l'adresse concernée pour la sauvegarde est obtenue avec la somme de **Ix** et de **Mx** mais **Ix** ne change pas de valeur.

Par contre, dans le second cas (**Ix** avant **Mx** dans **DM**), l'adresse concernée pour la sauvegarde est celle contenue dans **Ix** qui est ensuite modifiée par rajout de **Mx**.

Ici, par l'instruction **DM (I7 , M7) = R2;** et comme on l'a dit précédemment, on commencera donc à sauver **R2** au sommet de la pile pointée par **I7**, puis le contenu de **I7** sera remplacé par **I7+M7**, et donc sera décrémenté de 1 car **M7 = -1** pour que **I7** pointe toujours sur le sommet de la pile.

Suite à cette étape, la pile sera donc dans l'état de la figure VII.4.

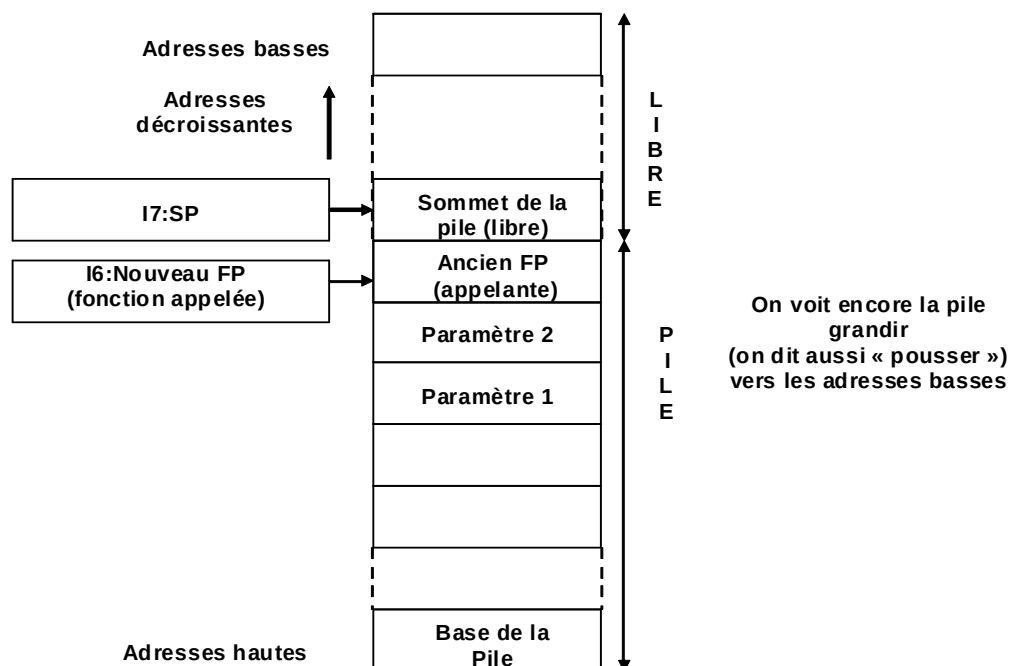


Figure VII.4.

⑥ gestion de l'adresse de retour :

Si on se rappelle de l'étape ④, le contrôle était donné à la fonction appelée par un simple saut. On voit donc qu'ici, contrairement à ce qui était fait dans le cas du **PIC** où l'adresse de retour était **automatiquement** sauvegardée sur la pile par une instruction du type **CALL**, avec un saut il n'y a pas de sauvegarde de cette adresse de retour. Pourtant, c'est obligatoire pour qu'à la fin de la fonction appelée, le programme appelant puisse continuer à s'exécuter. Il faut donc rajouter la sauvegarde de l'adresse de retour. (par une instruction du type **DM (I7 , M7) = PC;**).

Cette instruction sauvegarde la valeur courante du **PC** (c'est-à-dire à cet instant précis l'adresse de retour) sur le sommet de la pile et décrémente le pointeur de pile **I7** de 1 pour qu'il pointe toujours sur la prochaine case mémoire libre sur le sommet de la pile.

Et voici sur la figure VII.5. l'état de la pile après cette dernière étape.

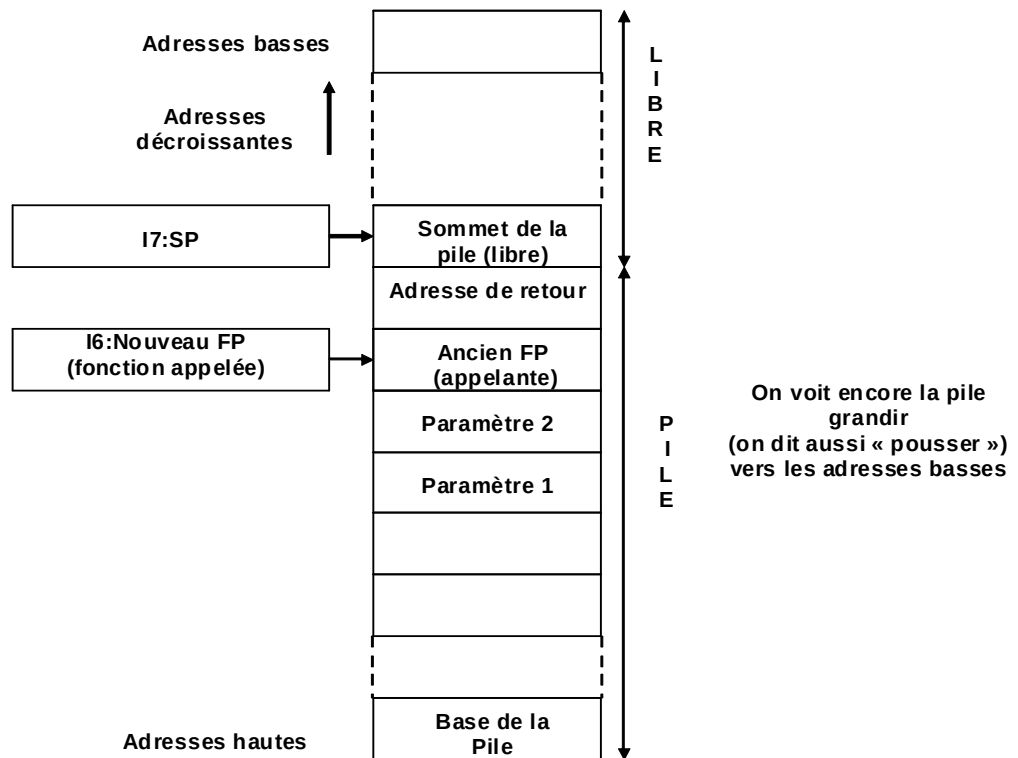


Figure VII.5.

A partir de cet instant là, c'est la première instruction de la fonction **nom_de_la_fonction_appelée** qui peut s'exécuter.

Au final et en résumé, voici l'état de la pile juste avant l'appel de la fonction (figure VII.6.a.) ou si vous voulez juste avant son accolade ouvrante { de son code en C et son état juste après cette même accolade ouvrante (figure VII.6.b.) une fois que les étapes précédentes ont été effectuées. On a supposé une fonction avec 2 paramètres : paramètre 1 et paramètre 2.

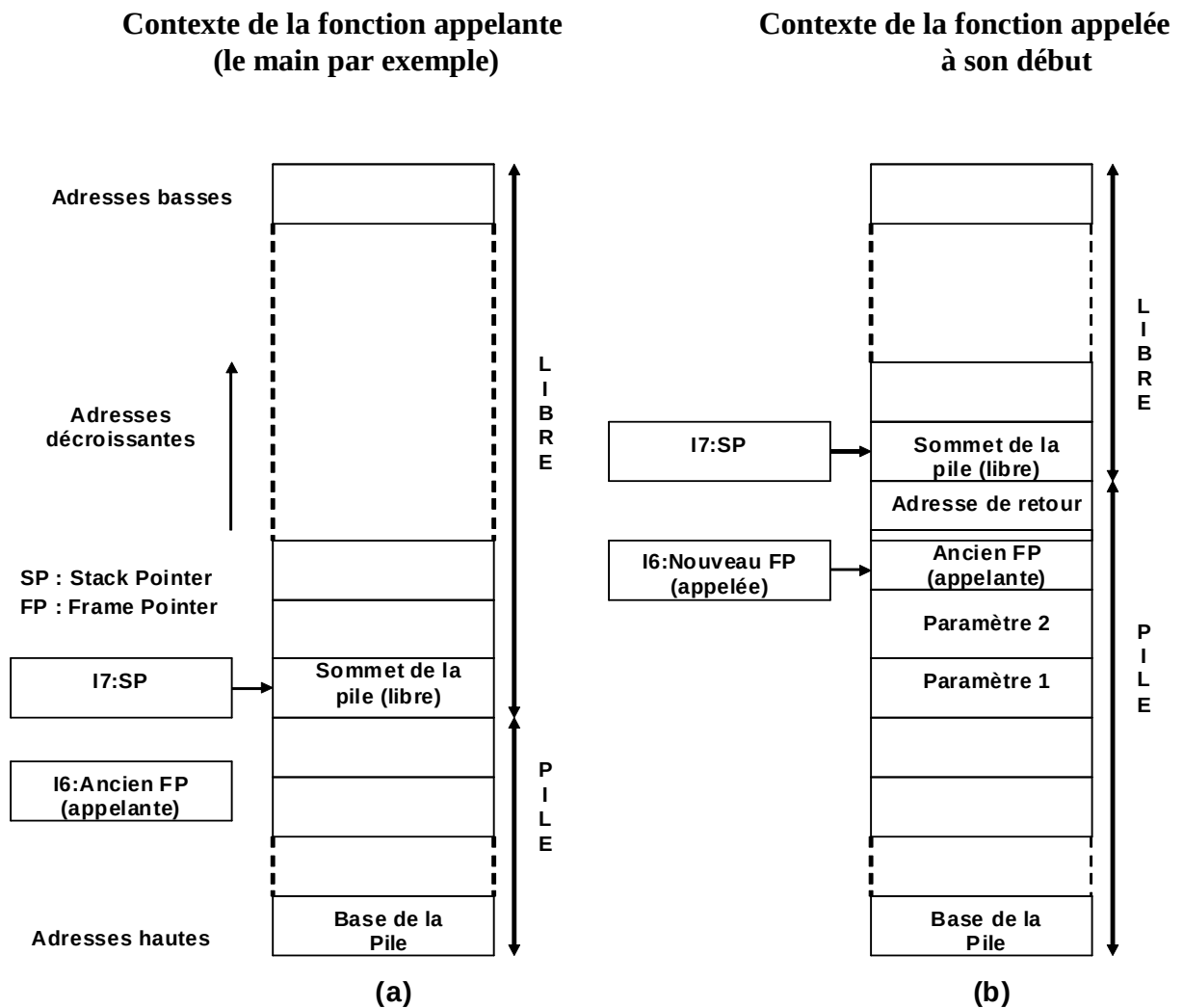


Figure VII.6.

A partir de ce moment là, si la fonction utilise des variables locales, celles-ci seront créées sur la pile juste au-dessus de l'adresse de retour et dans leur ordre de déclaration. On pourra utiliser par exemple une instruction de la forme `modify (I7, -n)` (voir plus loin) où `n` est la taille de la variable locale créée dynamiquement en haut de la pile.

Il apparaît clairement sur la figure précédente que la fonction appelée pourra accéder :

- à ses paramètres en utilisant un offset **positif** par rapport au contenu de **I6**,
- à ses variables locales en un offset **négatif** par rapport au contenu de **I6**.

Ainsi, comme on l'avait dit au début de ce chapitre, le nouveau Frame Pointer **I6** joue donc un rôle très important pour accéder au contexte de la fonction appelée.

Il ne reste maintenant plus qu'à préciser ce qui se passe à la fin de la fonction appelée, avant qu'elle ne rende le contrôle à la fonction appelante.

Ce qui incombe à la fonction appelée correspond aux étapes suivantes :

⑦ récupération de l'adresse de retour :

Ceci est fait par le compilateur via le registre **I12**.

(par une instruction du type : **I12 = DM (M7 , I6);**)

Notez bien qu'ici **I6** n'est pas modifié (pré-modification) et l'utilisation du nouveau frame pointer **I6** décalé de -1 (valeur de **M7**) permet bien de pointer sur l'adresse de retour préalablement sauvegardée et de la récupérer dans **I12**. Rappelons que le C garde dans **M7** la valeur -1.

L'état de la pile n'est pas modifié par cette étape.

⑧ retour à la fonction appelante :

Et plus exactement à l'instruction qui suit l'appel de la fonction appelée par la fonction appelante par un saut décalé (**DB**) à l'adresse de retour +1.

(par une instruction **JUMP (M14 , I12) (DB);**)

Notons que le C garde dans **M14** la valeur +1.

L'état de la pile n'est également pas modifié par cette étape.

Les deux instructions suivantes, à cause du **DB**, seront exécutées avant le retour proprement dit.

⑨ gestion du pointeur de pile :

Il est rechargé (par **I7 = I6;**) pour pointer sur le haut de la pile et libérer ainsi la place occupée par les éventuelles variables locales de la fonction appelée et l'adresse de retour.

Ce qui donne l'état de la figure VII.7. ci-dessous pour la pile.

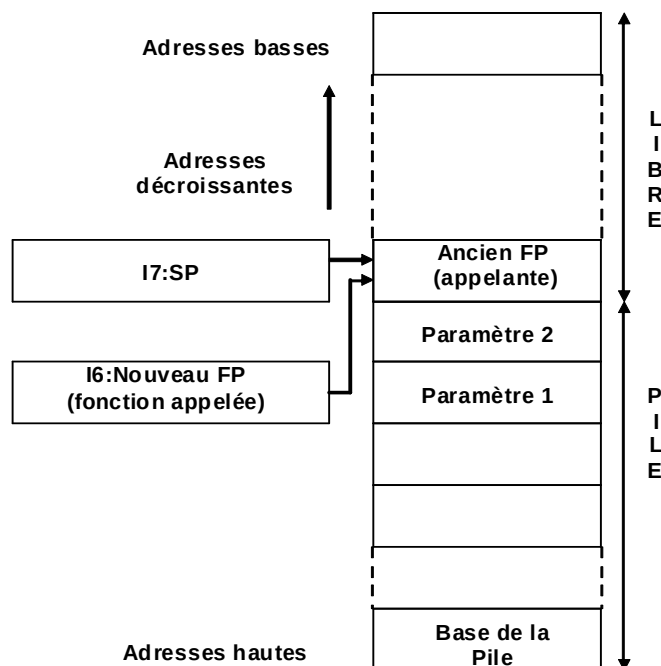


Figure VII.7.

De plus, s'il y a eu passage de paramètres par la pile, il faut aussi incrémenter **I7** d'une valeur égale à la taille des paramètres passés, toujours dans l'optique de libérer la place occupée dans la pile par ces paramètres. Et que pour **I7** pointe toujours sur la première place disponible sur la pile.

⑩ gestion de « l'ancien frame pointer » :

L'ancien frame pointeur (qui donne accès au contexte de la fonction appelante) doit enfin être récupéré (par une instruction **I6 = DM (0 , I6);**).

Par cette instruction, la nouvelle valeur de **I6** devient ainsi le frame pointer de la fonction appelante.

Ce qui donne pour la pile l'état de la figure VII.8.

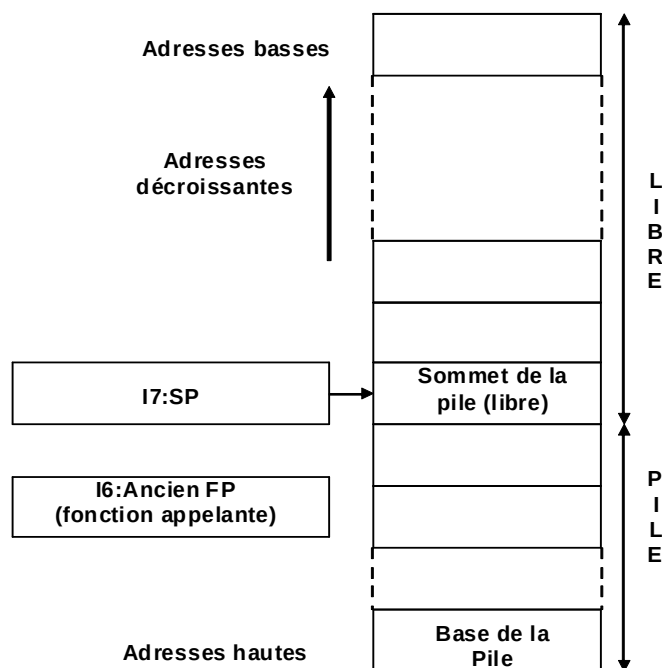


Figure VII.8.

Au final, après ces 4 étapes pour le retour, on retrouvera la pile exactement dans l'état où elle était avant l'appel de la fonction, ce qui était bien le but recherché : une fonction n'est qu'une simple « parenthèse » dans un programme, parenthèse toutefois importante puisqu'elle effectue tout de même une tâche utile au programme. Une fois exécutée, elle ne laisse plus aucune trace dans la pile ...

Le dernier point à préciser concerne la valeur retournée par une fonction par une instruction en C du type **return**.

La valeur retournée par une fonction (sauf si cette fonction est déclarée en **void**) est par convention le contenu de **R0** (si la taille de cette valeur est ≤ 32 bits).

Sinon pour une valeur comprise entre 32 bits et 64 bits la valeur retournée le sera :

- dans **R0** (pour son **MSDW** : **M**ost **S**ignificant **D**ouble **W**ord, soit ses 32 bits de poids forts),
- et dans **R1** (pour son **LSDW** : **L**ess **S**ignificant **D**ouble **W**ord, soit ses 32 bits de poids faibles).

Pour des structures de tailles plus importantes, on devra passer en paramètre un pointeur vers ces structures pour que la fonction appelée puisse les modifier à sa guise.

Remarque sur la taille des variables standards définies par le C :

La figure VII.9. résume la taille des variables standards utilisées par le C.

<i>C data type</i>	<i>Underlying type</i>	<i>Representation</i>
int	int	32-bit two's complement number
long int	int	32-bit two's complement number
short int	int	32-bit two's complement number
unsigned int	unsigned int	32-bit unsigned magnitude number
unsigned long int	unsigned int	32-bit unsigned magnitude number
char	int	32-bit two's complement number
unsigned char	unsigned int	32-bit unsigned magnitude number
float	float	32-bit IEEE single-precision number
double	float	32-bit IEEE double-precision number
long double	double	64-bit IEEE double-precision number
complex int	int	Two 32-bit two's complement numbers
complex float	float	Two 32-bit IEEE single-precision numbers

Figure VII.9.

Voici une liste de macros (écrites en assembleur) et décrites dans le fichier **asm_sprt.h** qui permettent de faciliter les différentes opérations précédentes et qui faciliteront l'écriture de fonctions en assembleur et appelables depuis un programme écrit en C.

Macros pour les protocoles d'entrée et de sortie de fonctions :

```
#define entry      /* nothing : en entrée de toute fonction assembleur : améliore la lisibilité */

#define leaf_entry /* nothing : en entrée de toute fonction assembleur : améliore la lisibilité */

#define ccall(x)   /* appel de fonction depuis l'assembleur correspondant à */
                  cjump (x) (DB);      /* étapes ②, ③ et ④ (cf. ci-dessous) */
                  dm(I7,M7)=R2;         /* avec gestion du frame pointer cf. étape ⑤ */
                  dm(I7,M7)=PC          /* et de l'adresse de retour cf. étape ⑥ */
```

	cjump (x) ; /* correspond en fait aux <u>3</u> instructions suivantes: */ R2 = I6 ; /* étape ② */ I6 = I7 ; /* étape ③ */ jump (PC,x) /* étape ④ */
---	--

```
#define leaf_exit /* en sortie de fonction écrite en assembleur correspondant à */
                  I12=dm(M7,I6);      /* récupération adresse de retour cf. étape ⑦ */
                  jump (M14,I12) (db); /* retour à l'appelant cf. étape ⑧ */
                  nop;                /* pour vider le pipe instruction*/
                  rframe ;             /* étapes ⑨ et ⑩ (cf. ci-dessous) */
```

	rframe; /* correspond en fait aux <u>2</u> instructions suivantes : */ I7 = I6 ; /* étape ⑨ */ I6 = dm(0,I6) / /* étape ⑩ */
---	---

```
#define exit      leaf_exit /* en sortie de fonction écrite en assembleur */
```

Macros pour la gestion de la pile (accès aux paramètres et aux variables locales ...):

```
#define reads(x)   dm(x, I6) /* accès au paramètre d'offset x par rapport à i6 */
                  /* i6 n'est pas modifié */

#define puts       dm(I7, M7) /* empilement et mise à jour du stack pointer par I7=I7+M7 */
                  /* rappelons que le C garde dans M7 la valeur -1 */
                  /* c'est en quelque sorte l'équivalent d'un push */
                  /* exemple : */
                  /* puts = R3 ; place R3 sur le haut de la pile et décrémente I7 */

#define gets(x)    dm(x, I7) /* lecture de la pile à l'offset x par rapport à I7 */
                  /* sans modification du stack pointer I7 */

/* exemple : */
R15=gets(1) ;      /* récupère dans R15 le dernier élément rangé en pile */

#define alter(x)   modify(I7, x) /* modifie la valeur de I7 pour libérer la place occupée */
                  /* par les paramètres dans la pile au retour de la fonction */
```

	modify(I7,x); /* correspond en fait à l'instruction suivante: */ I7 = I7+x ;
---	---

Ce qui permet de construire les macros suivantes pour sauver ou restituer tous les registres de travail :

```
#define save_reg    puts=R0;          /* empile les registres R0 à R15 et dans cet ordre !!!*/
                  puts=R1;
                  puts=R2;
                  puts=R3;
                  puts=R4;
                  puts=R5;
                  puts=R6;
                  puts=R7;
                  puts=R8;
                  puts=R9;
                  puts=R11;
                  puts=R12;
                  puts=R13;
                  puts=R14;
                  puts=R15

#define restore_reg R15=gets(1);      /* restaure les registres R0 à R15 en sens inverse !!!*/
                  R14=gets(2);
                  R13=gets(3);
                  R12=gets(4);
                  R11=gets(5);
                  R10=gets(6);
                  R9 =gets(7);
                  R8 =gets(8);
                  R7 =gets(9);
                  R6 =gets(10);
                  R5 =gets(11);
                  R4 =gets(12);
                  R3 =gets(13);
                  R2 =gets(14);
                  R1 =gets(15);
                  R0 =gets(16);
                  alter(16)          /* avec mise à jour du stack pointer I7 */
```

Remarques sur les registres du DSP et leur utilisation par le C :

On a vu que certains registres étaient utilisés par le C. En voici une liste plus complète :

- **I6 (B6, M6 et L6)** et **I7 (B7, M7 et L7)** : on l'a dit pour le cadre de pile et le pointeur de pile,
- les couples (**M5, M13**), (**M6, M14**) et (**M7, M15**) contiennent des valeurs fixes et initialisées par le C à respectivement 0, +1 et -1. A ne pas utiliser par l'assembleur,
- les registres **Lx** (sauf L6 et L7), s'ils sont utilisés par l'assembleur, doivent être remis à 0 avant de rendre la main au C,
- les registres **R0, R1, R2, R4, R8, R12, I4, B4, L4, M4, I12, B12, L12** et **M12** sont des “ **scratch** ” **registers**, c'est-à-dire que le compilateur C considère que ces registres peuvent être modifiés par une fonction et donc perdus. S'ils contiennent des valeurs importantes, il faut les sauvegarder avant d'appeler une fonction et les restituer ensuite,
- les autres registres sont des “ **compiler** ” **registers** : ils doivent être préservés après l'appel d'une fonction. Donc, si la fonction les utilise (c'est autorisé), elle doit les sauvegarder au début puis les restituer à la fin.

Pour mettre en pratique toutes ces notions, voici un exemple de code C appelant une fonction et sa transcription en assembleur par le compilateur. Vous le lirez attentivement pour retrouver les principaux éléments que l'on vient de décrire (bien lire et **comprendre** les commentaires !!!).

```
/* fichier source en C */

int carre(int j)          /* fonction retournant le carré de l'entier passé en paramètre */
{
    j=j*j;
    return(j);
}

int main()                /* la fonction principale = point d'entrée du programme */
{
    int i=3;
    i=carre(i);
}
```

```
/* traduction en assembleur par le compilateur C du fichier source précédent */

.global _carre;
_carre:
    modify(I7,-1);        /* réservation d'une place en pile pour la variable locale j */

    dm(-2,I6)=R4;         /* initialisation de j avec le paramètre dans R4 */

    R2=dm(-2,I6);         /* préparation du calcul du carré de j via R2*/
    R4=dm(-2,I6);         /* et R4 */
    R2=R2*R4;             /* calcul du carré dans R2 */
    dm(-2,I6)=R2;         /* et mise à jour de la locale j */

    R0=dm(-2,I6);         /* que l'on retourne dans R0 */

    I12=dm(-1,I6); /      /* récupération de l'adresse de retour */
    jump (M14,I12) (DB); /* et retour au main */
    nop;
    rframe;               /* avec gestion du pointeur de pile i7 et récupération du frame pointeur */
                          /* du main (ancien frame pointer) */

.global _main;
_main:
.extern _exit;            /* fonction de sortie du main : retour au système !!!*/

    modify(I7,-1);        /* réservation d'une place en pile pour la variable locale i */

    R2=3;
    dm(-2,I6)=R2;         /* initialisation de la locale i à 3 */

    R4=dm(-2,I6);         /* passage de paramètre par le registre R4 */
    cjump (pc, _carre) (DB); /* appel de la fonction carre avec gestion du frame pointer */
    dm(I7,M7)=R2;
    dm(I7,M7)=PC;         /* et de l'adresse de retour */

    dm(-2,I6)=R0;         /* récupération de la valeur retournée par carre dans r0 */
                          /* et modification de la locale i */

    I12=dm(-1,I6);
    jump (PC, _exit) (DB); /* retour du main */
    I7=I6;                /* avec gestion du pointeur de pile */
    I6=dm(0,I6);          /* et récupération de l'ancien frame pointer (celui du système)*/
```

VII.2.2. Les interruptions matérielles en C.

Il peut être intéressant de donner maintenant quelques éléments sur la gestion des interruptions matérielles par le C, d'autant plus que la majorité des traitements que vous effectuerez en TP se feront sous interruptions, pour les raisons d'efficacité que l'on a vues dans ce polycopié.

De façon très générale :

- elles correspondent à des événements matériels (et donc liés à l'environnement extérieur) pouvant avoir lieu **n'importe quand** (on dit qu'elles sont **asynchrones**) par rapport au déroulement normal d'un programme (comme par exemple l'arrivée d'un caractère sur une liaison, la fin d'un intervalle de temps mesuré par le timer ...),

- elles ne peuvent en conséquence retourner une valeur directement au programme (on ne peut pas mettre des instructions pour traiter une valeur de retour, car le retour d'une interruption peut-être à n'importe quel endroit d'un programme) et sont du type **void**.

Elles peuvent cependant modifier des variables globales (comme par exemple un flag indiquant qu'elles ont eu lieu ou l'incrément d'un compteur) : le main vient traiter cette information quand il en a le temps.

Il est conseillé alors de déclarer ces globales (et modifiées par la routine d'interruption) en **volatile**, ce qui évite au compilateur C d'exécuter une optimisation du code (comme par exemple celle qui supprimerait le test dans une boucle sur une variable qui n'est pas modifiée par cette boucle !!!),

- elles ne possèdent pas de paramètres (pour les mêmes raisons que précédemment),

- les interruptions peuvent être imbriquées (avec notion de priorité),

- une routine de traitement d'interruption doit être la plus courte possible en temps de traitement : ne pas oublier que c'est une situation exceptionnelle,

● les principales fonctions du C, par l'intermédiaire d'un dispatcher d'interruptions, qui permettent de gérer les interruptions sont :

➔ **void (*interrupt (int sig, void (*func) (int))) (int);**

Méthode standard et générale pour installer la routine d'interruption **func**.

Une fois installée, **chaque fois** que le signal **sig** se produira, la fonction **func** s'exécutera.

En pratique, plutôt que la nommer fonction, on préférera la nommer routine d'interruption.

Toutes les causes d'interruption sont codées dans **signal.h** et sont données sur le tableau de la figure VII.10. ci-dessous classées du plus haut au plus bas niveau de priorité :

sig value	Meaning
SIG_SOVF	Status stack or Loop stack overflow or PC stack full
SIG_TMZ0	Timer = 0 (high priority option)
SIG_VIRPTI	Vector Interrupt
SIG_IRQ2	Interrupt 2
SIG_IRQ1	Interrupt 1
SIG_IRQ0	Interrupt 0
SIG_SPR0I	SPORT0 receive DMA channel
SIG_SPR1I	SPORT1 receive (or LBUF0) DMA channel
SIG_SPT0I	SPORT0 transmit DMA channel
SIG_SPT1I	SPORT1 transmit (or LBUF1) DMA channel
SIG_LP2I	Link buffer 2 (LBUF2) DMA channel
SIG_LP3I	Link buffer 3 (LBUF3) DMA channel
SIG_EP0I	External Port DMA channel 0 (or LBUF4)
SIG_EP1I	External Port DMA channel 1 (or LBUF5)
SIG_EP2I	External Port DMA channel 2
SIG_EP3I	External Port DMA channel 3
SIG_LSRQ	Link service request
SIG_CB7	Circular buffer 7 overflow
SIG_CB15	Circular buffer 15 overflow
SIG_TMZ	Timer = 0 (low priority option)
SIG_FIX	Fixed-point overflow
SIG_FLTO	Floating-point overflow exception
SIG_FLTU	Floating-point underflow exception
SIG_FLTI	Floating-point invalid exception
SIG_USR0	User interrupt 0
SIG_USR1	User software interrupt 1
SIG_USR2	User software interrupt 2
SIG_USR3	User software interrupt 3

Figure VII.10.

➔ **void (*signal (int sig, void (*func) (int))) (int);**

Une autre méthode d'installation de la routine d'interruption **func**.

Cette routine **func** ne s'exécutera **qu'une seule fois**, quand le signal **sig** se produira pour la première fois.

Si on veut une exécution de **func** à chaque occurrence de **sig**, il faut réutiliser la fonction **signal** dans la routine d'interruption **func** ou alors plus simplement utiliser la fonction **interrupt** précédente.

Toutes les causes d'interruption sont aussi codées dans **signal.h** et sont identiques au cas précédent.

➔ **int raise (int sig);**

envoie le signal **sig** pour exécuter la routine d'interruption (au préalable installée).

Ce qui permet de **simuler** une interruption et son traitement sous contrôle logiciel.

Cela peut être utile dans les phases de mise au point d'un programme utilisant les interruptions matérielles.

➔ **int clear_interrupt (int sig);**

permet d'empêcher la prise en compte de l'interruption **sig en attente** (à utiliser avant la phase d'installation de la routine d'interruption: si une demande d'interruption en attente est présente avant l'installation, elle ne sera pas servie, les suivantes le seront une fois la routine installée).

Voici, à la page suivante, un exemple de programme en C, issu de la documentation technique de Analog Devices, qui permet de gérer et de traiter la durée d'expiration du timer (routine **timer_handler**) et l'arrivée d'une interruption extérieure sur la broche IRQ0 (routine **irq0_handler**).

J'ai rajouté en gras quelques commentaires permettant de faciliter la compréhension du code et de mieux assimiler le principe des interruptions matérielles.

```

#include <signal.h>      /* declaration des différentes causes d'interruption */

/* These are function that do useful work. */
/* Elles seront exécutées quand les flags timer_expired=1 ou irq0_occurred=1 */
/* Ces flags globaux sont mis à 1 par leurs routines d'interruption respectives */
extern void do_timer_things(void);
extern void do_irq0_things(void);
/* flags globaux déclarés en volatile pour éviter une optimisation du code */
volatile int timer_expired;
volatile int irq0_occurred;

/* Be sure to have the correct prototype for an ISR */
/* routine d'interruption exécutée chaque fois que le timer atteint 0 */
void timer_handler(int signal)
{
    timer_expired = 1;          /* flag global mis à 1 pour indiquer que le timer */
                                /* a expiré */
}

/* routine d'interruption exécutée chaque fois que le IRQ0 vaut 0 */
void irq0_handler(int signal)
{
    irq0_occurred = 1;          /* flag global mis à 1 pour indiquer que IRQ0=0 */
}

main()
{
    /* Set timer expired to zero at the beginning */
    /* les 2 flags sont initialises à 0 */
    timer_expired = 0;
    irq0_occurred = 0;
    /* Set up the routines to respond to interrupts */
    /* installation des 2 routines d'interruption par la fonction du C interrupt */
    interrupt(SIG_IRQ0, irq0_handler);
    interrupt(SIG_TMZ, timer_handler);
    /* Set up the timer */
    /* initialisation et mise en route du timer */
    timer_set((unsigned int)10000, (unsigned int)10000);
    timer_on();
    /* Loop continuously and respond to interrupts */
    while(1) /* boucle sans fin */
        /* remarquez que dans cette boucle rien ne vient modifier les */
        /* flags timer_expired et irq0_occurred et donc s'il n'y */
        /* avait pas les interruptions timer_handler et irq0_handler */
        /* ces flags resteraient à 0 et on ne pourrait pas exécuter */
        /* les fonctions do_timer_things() et do_irq0_things */
        {
            /* If the timer has expired, clear flag and do something*/
            if( timer_expired )
            {
                /* exécuté chaque fois que le timer expire */
                timer_expired = 0;          /* remise à 0 du flag pour prendre en compte */
                                            /* la prochaine expiration du timer */
                do_timer_things();          / fait le traitement adequat */
            }
            /* If irq0 has occurred, clear flag and do something*/
            if( irq0_occurred )
            {
                /* exécuté chaque fois que IRQ0 = 0 */
                irq0_occurred = 0;          /* remise à 0 du flag pour prendre en compte */
                                            /* le prochain passage de IRQ0 à 0 */
                do_irq0_things();          / fait le traitement adequat */
            }
        }
}

```

VII.2.3. Traitement numérique du signal en C.

Nous allons fournir maintenant quelques prototypes de fonctions utiles en traitement du signal, en détaillant pour certaines leur documentation.

Précisons tout d'abord quelques fichiers d'inclusion utiles :

Macros et définitions de types :

Purpose	Header
Error Handling	errno.h
Standard Definitions	stddef.h
Limits	limits.h
Floating Point	float.h

Fonctions ANSI :

Purpose	Header
Mathematics	math.h
Signal Handling	signal.h
Variable Arguments	stdarg.h
Character Handling	ctype.h
String Handling	string.h
Standard Library	stdlib.h
Localization	locale.h
Diagnostics	assert.h
Non-Local Jumps	setjump.h
Input/Output	stdio.h

Fonctions non ANSI fournies par Analog Devices :

Et pour faciliter l'implantation d'algorithmes de traitement du signal, Analog Devices fournit une bibliothèque de fonctions non ANSI très utiles. Voici la liste de leurs *.h :

Purpose	Header
Assembly Language Support	asm_sprt.h
Circular Buffers/Assembly	
Language Macros	macros.h
Matrix Operations	matrix.h
DSP Filters	filters.h
Fast Fourier Transforms	trans.h
ADSP-21020 Processor-Specific	21020.h
ADSP-2106x SHARC Processor-Specific	21060.h
Complex Mathematics	complex.h
Numerical C Extensions	dspc.h
Statistics	stats.h
Communication	comm.h
Bit Definitions for 21020 System Registers	def21020.h
Bit Definitions for ADSP-2106x	
SHARC System Registers	def21060.h

Il est conseillé, dans chaque cas, de se reporter à la documentation de chacune des fonctions fournies. Voici quelques exemples de fonctions (ainsi que leurs documentations en anglais) que nous mettrons en œuvre en TP.

FUNCTION

fir—finite impulse response (FIR) filter

SYNOPSIS

```
#include <filters.h>
float fir(float sample,\
         float pm coeffs[],\
         float dm state[], int taps);
```

DESCRIPTION

This function is an Analog Devices extension to the ANSI standard.

The `fir()` function implements a finite impulse response (FIR) filter defined by the coefficients and delay line that are supplied in the call of `fir`. The function produces the filtered response of its input data. This FIR filter is structured as a sum of products. The characteristics of the filter (passband, stop band, etc.) are dependent on the coefficient values and the number of taps supplied by the calling program.

The floating-point input to the filter is `sample`. The integer `taps` indicates the length of the filter, which is also the length of the array `coeffs`. The `coeffs` array holds one FIR filter coefficient per element. The coefficients are stored in reverse order; for example, `coeffs[0]` holds the $(taps - 1)$ coefficient. The `coeffs` array is located in program memory data space to use the single-cycle dual-memory fetch of the processor.

The `state` array contains a pointer to the delay line as its first element, followed by the delay line values. The length of the `state` array is therefore 1 greater than the number of taps. Each filter has its own `state` array, which should not be modified by the calling program, only by the `fir` function. The `state` array should be initialized to zeros before the `fir` function is called for the first time.

ERROR CONDITIONS

The `fir` function does not return an error condition.

EXAMPLE

```
#include <filters.h>
float y;
float pm coeffs[10]; /* coeffs array must be */
/* initialized and in */
/* PM memory*/
float state[11];
int i;
for (i=0; i < 11; i++)
state[i]=0; /* initialize state array */
y = fir(0.775,coeffs, state, 10);
/* y holds the filtered output */
```

SEE ALSO

iir

FUNCTION

iir—infinite impulse response (IIR) filter

SYNOPSIS

```
#include <filters.h>
float iir(float sample,
          float pm a_coeffs[],
          float pm b_coeffs[],
          float dm state[], int taps);
```

DESCRIPTION

This function is an Analog Devices extension to the ANSI standard. The `iir` function implements an infinite impulse response (IIR) filter defined by the coefficients and delay line that are supplied in the call of `iir`. The function produces the filtered response of its input data. The IIR filter implemented in this function is based on the Oppenheim and Schaffer Direct Form II. The characteristics of the filter are dependent on the coefficient values supplied by the calling program.

The floating-point input to the filter is `sample`. The integer `taps` indicates the length of the filter, which is the length of the arrays `a_coeffs`, `b_coeffs` and `delay`. The `a_coeffs` and `b_coeffs` arrays hold one IIR filter coefficient per element. The coefficients are stored in reverse order; for example, `a_coeffs[0]` holds the `taps - 1` (the last coefficient).

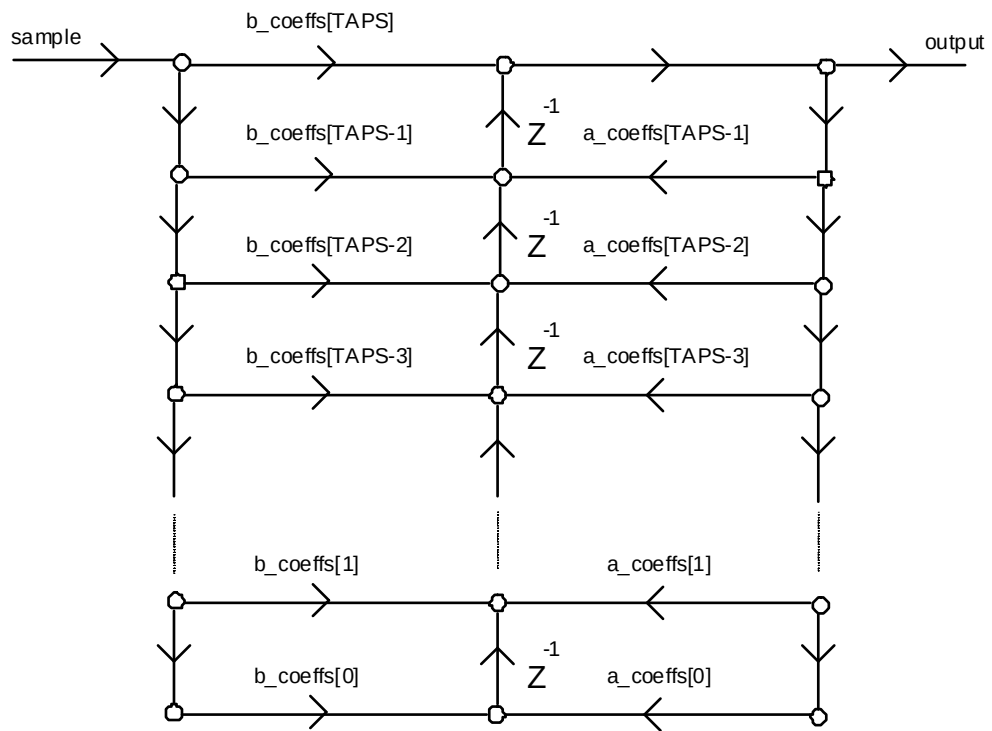
The `state` array contains a pointer to the delay line as its first element, followed by the delay line values. The length of the `state` array is therefore 1 greater than the number of taps. Each filter has its own `state` array, which should not be modified by the calling program, only by the `iir` function. The `state` array should be initialized to zeros before the `iir` function is called for the first time. The flow graph on the next page corresponds to the `iir()` routine as part of the C Runtime Library.

ERROR CONDITIONS

The `iir` function does not return an error condition.

EXAMPLE

```
#include <filters.h>
#define TAPS 10
int i;
float pm a_coeffs[TAPS], b_coeffs[TAPS+1];
float in_sample, output, state[TAPS+1];
for (i=0; i < TAPS+1; i++)
state[i]=0; /* initialize state array */
output = iir(in_sample, a-coeffs, b-coeffs, state,
TAPS);
```



The b_coeffs and state arrays should equal length $TAPS+1$
The a_coeffs array should equal length $TAPS$

FUNCTION

rfftN—N-point real input fast Fourier transform

SYNOPSIS

```
#include <trans.h>
```

```
float *rfft65536(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft32768(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft16384(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft8192(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft4096(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft2048(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft1024(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft256(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft128(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft64(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft32(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft16(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

```
float *rfft8(float dm real_input[],  
float dm real_output[],  
float dm imag_output[]);
```

DESCRIPTION

These functions are Analog Devices extensions to the ANSI standard. They are similar to the `cfftN` functions, except that they only take real inputs.

Each of these 14 `rfftN` functions computes the N-point radix-2 fast Fourier transform (RFFT) of its fractional input (where N is 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768 or 65536).

There are fourteen distinct functions in this set. They all perform the same function with same type and number of arguments. Their only difference is the size of the arrays on which they operate.

Call a particular function by substituting the number of points for N, as in

```
rfft8(r_inp, r_outp, i_outp);  
/* Correct */  
not  
rfftN(r_inp, r_outp, i_outp);  
/* WRONG */
```

The input to `rfftN` is a floating-point array of N points. If there are fewer than N actual data points, you must pad the array with zeros to make N samples. Better results occur with less zero padding, however.

The input data should be windowed (if necessary) before calling the function because no preprocessing is performed on the data.

`rfftN()` returns a pointer to its real floating-point output array.

ERROR CONDITIONS

The `rfftN` functions do not return any error conditions.

EXAMPLE

```
#include <trans.h>  
#define N 2048  
int i;  
float real_input[N], imag_output[N];  
float real_output[N];  
/* Real input array is filled from a converter or  
other source */  
rfft2048 (real_input, real_output, imag_output) ;  
/* Arrays are filled with FFT data */
```

SEE ALSO

`ifftN`, `cfftN`

SOMMAIRE

I. INTRODUCTION : LE DSP (ou Digital Signal Processor).....	02
I.1. Généralités : qui fait quoi ?.....	02
I.2. Les DSP chez Analog Devices.....	09
 II. LA FAMILLE 21xx	 10
II.1. Eléments sur l'architecture et sur les bus internes.....	10
II.2. Les unités de calculs internes.....	11
II.2.1. ALU : Arithmetic and Logic Unit.....	11
II.2.2. MAC : Multiplier / Accumulator Unit.....	12
II.2.3. SHIFTER : Shifter Unit.....	14
II.3. Les DAG : Data Address Generator.....	15
II.4. Le séquençement du programme.....	18
II.5. En résumé.....	23
II.6. Les différents éléments de la famille 21xx.....	24
 III. LA FAMILLE 21xxx.....	 26
 IV. LE DSP UTILISE SUR LA CARTE EZKIT	 32
IV.1. Le cœur du processeur.....	32
IV.2. La mémoire interne et le mapping mémoire.....	33
IV.3. Interfaçage avec l'extérieur.....	36
IV.3.1. Les bus externes.....	38
IV.3.2. Multi-processeurs.....	38
IV.3.3. Le processeur hôte.....	38
IV.4. Un processeur d'entrées / sorties.....	38
IV.4.1. Les registres de contrôle pour les IO.....	38
IV.4.2. Les deux ports série SPORT0 et SPORT1.....	39
IV.4.3. Un contrôleur de DMA (Direct Memory Access).....	39
IV.4.3.1. Transfert par « pooling » ou scrutation.....	41
IV.4.3.2. Transfert par interruption.....	43
IV.4.3.3. Transfert sous DMA.....	45
 V. LE CODEC AD1847.....	 51
V.1. Présentation générale.....	51
V.2. Format des données gérées par le CODEC.....	55
V.2.1. 16 bits signé et linéaire.....	55
V.2.2. 8 bits non signé et linéaire.....	56
V.2.3. 8 bits comprimés.....	57
V.3. Protocole de communication TDM entre le DSP et le Codec.....	60
V.3.1. Bilan des échanges.....	60
V.3.2. Solutions hardware.....	61
V.3.2.1. Liaison à un seul fil.....	62
V.3.2.2. Liaison à deux fils.....	63

VI. LE SYSTEME D'EVALUATION EZ-KIT UTILISE DANS LE TP.....	64
VI.1. Architecture matérielle.....	64
VI.2. Les fichiers et les logiciels utilisés.....	66
VI.2.1. Le fichier d'architecture matérielle EZKIT.ACH	66
VI.2.2. Les différentes phases de développement d'un programme....	69
VI.2.2.1. La phase d'écriture du programme.....	70
VI.2.2.2. La phase de compilation du programme.....	70
VI.2.2.3. La phase d'édition de liens ou de linkage.....	72
VI.2.2.4. La phase de simulation du programme.....	73
VI.2.2.5. La phase de téléchargement, de mise au point.....	73
VI.2.2.6. La phase de claquage d'une EPROM.....	76
VI.2.3. Autres utilitaires et librairies.....	77
VI.2.3.1. Un serveur de fonctions C.....	77
VI.2.3.2. Gestion d'une librairie.....	79
VI.2.3.3. Quelques sources assembleur d'Analog Devices.....	80
 VII. MISE EN ŒUVRE EN LANGAGE C.....	 81
VII.1. Un exemple commenté et détaillé en C.....	81
VII.2. Quelques précisions sur le C fourni par Analog Devices.....	87
VII.2.1. Couplage C/Assembleur.....	87
VII.2.2. Les interruptions matérielles en C.....	99
VII.2.3. Traitement numérique du signal en C.....	103